OpenBao 2.7.1 — The Complete Course

From absolute zero to production-grade enterprise secrets management

Mavis

October 2026

Contents

OpenBao 2.7.1 — The Complete Course **4**
Who this is for . . . 4
Version target — read this first . . . 4
Course map . . . 5
How to use this course . . . 6
Why OpenBao and not HashiCorp Vault? . . . 6
Canonical sources used to build this course . . . 6
✓ How this course was verified (and how to re-verify it) . . . 6
The one-paragraph summary . . . 8

Level 0 — Foundations **8**
0.1 The problem, stated brutally . . . 8
0.2 What OpenBao actually is (and is not) . . . 9
0.3 The one architectural diagram you need to remember . . . 9
0.4 Ten vocabulary words that make everything else obvious . . . 10
0.5 Threat model — what is OpenBao actually protecting you from? . . . 10
0.6 Install it: Docker first (as you prefer) . . . 11
0.7 Your very first authenticated session . . . 13
0.8 The ”3 requests” that teach you the whole product . . . 14
0.9 Level 0 checkpoint . . . 15
0.10 Scratchpad . . . 15

Level 1 — The Core Concepts **15**
1.1 The server, the barrier, the storage backend . . . 15
1.2 Seal / Unseal — the lifecycle . . . 17
1.3 Policies — the authorization model . . . 18
1.4 Tokens — identity, capability and time . . . 20
1.5 Auth methods — proving who you are . . . 22
1.6 Secrets engines — storing, generating, encrypting . . . 23
1.7 Leases — how secrets die on schedule . . . 25
1.8 Audit devices — the ledger . . . 25
1.9 Identity — entities, aliases, groups . . . 26
1.10 Namespaces — multi-tenancy in one instance . . . 26
1.11 The `sys/` namespace — the control plane . . . 27
1.12 Agent and Proxy — the client-side daemons . . . 28
1.13 The request lifecycle, end to end . . . 28
1.14 Level 1 checkpoint . . . 29
1.15 Scratchpad . . . 29

Level 2 — Hands-On: From Zero to a Working OpenBao **29**
2.0 The plan for this level . . . 29

2.1 Step 1 — The Docker Compose stack . . . 30
2.2 Step 2 — TLS (do this from day one, not ”in production later”) . . . 31
2.3 Step 3 — Initialise and unseal . . . 33
2.4 Step 4 — Audit FIRST (before you store anything) . . . 34
2.5 Step 5 — KV v2 and a secret . . . 35
2.6 Step 6 — Policies (write them like a reviewer would) . . . 36
2.7 Step 7 — A human login path (userpass) . . . 38
2.8 Step 8 — A machine login path (AppRole) . . . 38
2.9 Step 9 — Wire up a FastAPI app . . . 39
2.10 Step 10 — Read the audit log . . . 42
2.11 Now break it on purpose . . . 43
2.12 Clean up and reset (repeatable practice) . . . 43
2.13 Level 2 checkpoint . . . 43
2.14 Scratchpad . . . 44

Level 3 — Policies, Tokens & Identity Deep Dive **44**
3.1 The authorisation model, in one picture . . . 44
3.2 Policy syntax — the complete reference . . . 45
3.3 EGP and RGP (a note of caution) . . . 49
3.4 Tokens — the complete strategy . . . 49
3.5 Identity — the graph that scales authorisation . . . 51
3.6 Designing the authorisation model for a real org . . . 53
3.7 Auditing who did what . . . 54
3.8 Level 3 checkpoint . . . 55
3.9 Scratchpad . . . 55

Level 4 — The Secrets Engines **55**
4.0 Choosing the right engine . . . 55
4.1 KV v2 — the workhorse . . . 56
4.2 Transit — encryption as a service . . . 58
4.3 PKI — your internal certificate authority . . . 60
4.4 Database — dynamic credentials (the highest-value engine) . . . 64
4.5 TOTP — step-up MFA inside OpenBao . . . 66
4.6 SSH — certificates instead of authorized_keys . . . 66
4.7 Cubbyhole — per-token private storage . . . 67
4.8 Response Wrapping — hand a secret to someone else, not to yourself . . . 67
4.9 Redis / Valkey — what actually exists . . . 68
4.10 Engine tuning and lifecycle . . . 69
4.11 Level 4 checkpoint . . . 69
4.12 Scratchpad . . . 69

Level 5 — Production Security Standards **69**
5.0 The baseline, at a glance . . . 69
5.1 TLS everywhere — no exceptions . . . 70
5.2 Swap, core dumps, and memory protection . . . 71
5.3 Auto-unseal — designing the seal story . . . 73
5.4 Root token hygiene . . . 76
5.5 Audit strategy . . . 77
5.6 Network exposure . . . 79
5.7 Quotas, rate limits and lockout . . . 80
5.8 Policy hygiene . . . 81
5.9 Secret lifecycle — rotation without downtime . . . 82
5.10 Supply chain — plugins are code you execute . . . 82
5.11 Patch cadence . . . 84
5.12 The complete hardening checklist . . . 84
5.13 Level 5 checkpoint . . . 85
5.14 Scratchpad . . . 85

Level 6 — High Availability & Production Architecture **86**
6.1 The HA mental model . . . 86
6.2 The reference architecture: 5-node Raft in Docker Compose . . . 87
6.3 The PostgreSQL storage backend (the alternative) . . . 91
6.4 Consistency — the 2.7.0 headers . . . 93
6.5 Snapshots, backup and restore . . . 94
6.6 Upgrades . . . 95
6.7 Observability . . . 97
6.8 Disaster recovery scenarios . . . 98
6.9 Infrastructure as Code . . . 99
6.10 Level 6 checkpoint . . . 100
6.11 Scratchpad . . . 101

Level 7 — Integration With Your Stack **101**
7.0 The integration map . . . 101
7.1 Keycloak → OpenBao (OIDC) . . . 102
7.2 Machines: AppRole in Docker Compose . . . 104
7.3 FastAPI — the production integration . . . 107
7.4 LangChain — sourcing API keys safely . . . 110
7.5 PostgreSQL — dynamic credentials end to end . . . 112
7.6 Redis — the honest version . . . 114
7.7 GitHub Actions → OpenBao (OIDC, no stored secrets) . . . 115
7.8 The complete stack in one Compose file . . . 119
7.9 The anti-patterns . . . 122
7.10 Level 7 checkpoint . . . 122
7.11 Scratchpad . . . 123

Level 8 — Expert: The Advanced Surface **123**
8.1 Namespaces at scale . . . 123
8.2 The plugin system . . . 125
8.3 Workflows (2.6.0) . . . 127
8.4 External Keys (2.7.0) — keys in an HSM, not in OpenBao . . . 129
8.5 Post-quantum cryptography (2.7.0) . . . 130
8.6 CEL validation programs . . . 131
8.7 Control groups (2.7.0) — the two-person rule . . . 132
8.8 Password policies (advanced) . . . 134
8.9 Other expert features worth knowing . . . 134
8.10 Kubernetes (brief, but you should know it exists) . . . 136
8.11 Level 8 checkpoint . . . 136
8.12 Scratchpad . . . 136

Level 9 — Reference & Cheatsheet **137**
9.1 Every important bao command . . . 137
9.2 The API path map . . . 141
9.3 Production-grade HCL configuration (annotated) . . . 143
9.4 The full production runbook . . . 145
9.5 Version migration table (Vault → OpenBao, and 2.x changes) . . . 147
9.6 Troubleshooting . . . 147
9.7 The one-page mental model . . . 149
9.8 Where to go next . . . 150

Level 10 — Keycloak ⇄ OpenBao: The OIDC Deep Dive **151**
10.1 The `oidc` and `jwt` auth methods are the SAME plugin . . . 151
10.2 The configuration keys (and the two that bite) . . . 152
10.3 The role: where identity becomes policy . . . 153
10.4 The browser login flow . . . 156
10.5 The complete Keycloak → OpenBao setup . . . 157

10.6 Keycloak service accounts → OpenBao (machine auth) . . . 158
10.7 Keycloak → OpenBao for your own application (the reverse direction) . . . 159
10.8 token_policies_template_claims — one role, many policies . . . 160
10.9 The claim → OpenBao token pipeline . . . 161
10.10 Troubleshooting matrix . . . 161
10.11 The Keycloak-side cheat sheet . . . 162
10.12 Checkpoint . . . 162
10.13 Scratchpad . . . 162

Level 11 — GitHub Actions ⇄ OpenBao: Secrets-Free CI 163
11.1 The problem this solves . . . 163
11.2 The claims GitHub puts in the token . . . 164
11.3 OpenBao side — the complete configuration . . . 164
11.4 The GitHub Actions workflow . . . 166
11.5 Using the bao CLI in CI . . . 168
11.6 Hardening the pipeline . . . 168
11.7 The reverse direction: OpenBao as GitHub's OIDC provider . . . 170
11.8 Patterns for a real AI-harness repo . . . 171
11.9 Verified behaviour — what actually happens . . . 172
11.10 Troubleshooting . . . 172
11.11 The CI security checklist . . . 173
11.12 Run it for real . . . 173
11.13 Checkpoint . . . 174
11.14 Scratchpad . . . 174

OpenBao 2.7.1 — The Complete Course

From absolute zero to production-grade enterprise secrets management

---

Who this is for

You are building an **Enterprise AI harness** with this stack:

`Python · FastAPI · LangChain · JWT/OAuth/OIDC · Keycloak`
`OpenBao · PostgreSQL · Redis · Git/GitHub/GitHub Actions · Docker/Docker Compose`

You want everything containerised, you want open-source tools, you want production-grade security, and you want to understand OpenBao from the ground up.

This course takes you from "what even is a secret manager" to "I run a multi-namespace, HA, auto-unsealed OpenBao cluster with a PKI, dynamic database credentials, and CI that pulls secrets via OIDC."

Version target — read this first

Item	Value
OpenBao release taught	**v2.7.1** (released 1 Oct 2026)
CLI binary	`bao` (NOT `vault`)
Docs source	`openbao.org/docs` (2.7.x branch) + `github.com/openbao/openbao` tag `v2.7.1`
Language	Python 3.11+
Licence	MPL-2.0, Linux Foundation / OpenSSF

Everything in this course was verified against the shipped `v2.7.1` source tree and its `CHANGELOG.md`. Where 2.5/2.6/2.7 removed or deprecated something, you will see an explicit ⚠ **REMOVED** / **DEPRECATED** callout rather than the old way of doing things.

The "do not learn this" list (things people still teach that are now wrong)

Old thing	Status in 2.7.1
`vault` CLI	Renamed long ago
`storage "file"`	**REMOVED in 2.7.0**
`storage "pebbledb"`	Deprecated for `operator migrat`
Consul / DynamoDB / S3 / etcd storage	Removed
Built-in awskms/gcpckms/azurekeyvault/alicloudkms/ocikms/pkcs11 seals	**REMOVED from binary in 2.7.**
Built-in Kerberos / LDAP (auth) / RADIUS (auth) / LDAP (secrets)	**REMOVED from binary in 2.7.**
`openbao-hsm` container image / HSM distribution	**Discontinued in 2.7.0**
`sys/generate-root/attempt` (unauthenticated)	**Deprecated**; disabled by default
Legacy unauthenticated `/sys/rekey/*` endpoints	**Disabled by default** (`disable_`
`sys/health` → `performance_standby`, `last_wal` fields	Removed in 2.6.0
`stored_shares` param on `sys/init`	Removed in 2.6.0 (ignored)
Arm64-specific container tags (`2.7.0-arm64`)	Not published since 2.6.0
FQDNs with trailing dot (`example.com.`) in PKI	Rejected (Go 1.27 change)
`build_date` in `sys/seal-status` / `sys/version-history`	Renamed `commit_date` in 2.6.0
OpenBao-HSM `openbao-hsm:*` images	Discontinued

Course map

Each level builds directly on the previous one. Do them in order.

#	Chapter	You will learn
0	FOUNDATIONS.md	What OpenBao is, why you need it, the threat model, the 10 vocabulary w
1	CONCEPTS.md	The mental model: server → barrier → storage, seal/unseal, tokens, polici
2	HANDS-ON.md	Level 0 → working OpenBao in Docker. Init, unseal, KV v2, policies, userp
3	POLICIES-TOKENS-IDENTITY.md	Authorization deep dive: ACL syntax, capabilities, templating, groups, en
4	SECRET-ENGINES.md	KV v2, Transit (encryption-as-a-service), PKI (internal CA), Database (dyn
5	SECURITY-HARDENING.md	The production security baseline: TLS, mlock, auto-unseal design, root to
6	HA-PRODUCTION.md	Reference architecture: 5-node Raft HA, PostgreSQL backend, snapshots
7	INTEGRATION-STACK.md	Wiring OpenBao into your stack: Keycloak OIDC, FastAPI, LangChain, Post
8	EXPERT.md	Namespaces, plugin system + OCI distribution, workflows engine, Extern
9	REFERENCE.md	Cheatsheets: every important command, API path map, config reference
10	KEYCLOAK-OIDC.md	**Keycloak ⇄ OpenBao, deep dive.** The `oidc/jwt` mount anatomy, clain
11	GITHUB-ACTIONS-OIDC.md	**Secrets-free CI.** GitHub OIDC → OpenBao, `bound_claims` hardening, to

Labs

Runnable Docker Compose labs live in `labs/`:

```
labs/
├── lab01-dev-stack/        # OpenBao + Postgres + Redis + Keycloak, one command
├── lab02-ha-raft/          # 5-node Raft HA cluster with TLS
├── lab03-full-reference/   # The full production reference architecture
└── lab04-ci-oidc/          # Secrets-free CI: GitHub Actions OIDC → OpenBao
```

How to use this course

1. **Type every command.** Reading commands teaches you nothing. Copy/paste teaches you less.
2. **Break it on purpose** once per level. Rename a policy path, break a token, revoke a lease — then read the audit log to see what happened. The audit log is the debugger.
3. **Keep a scratch file.** Every level gives you a "scratchpad" section. Track your own `role_id`s, mount paths, policies. You will forget them.
4. **Never commit secrets.** Not to git, not to Slack, not to a `.env` file that ends up in the repo. Add `.env*` to `.gitignore` on day one.
5. **Read the audit log.** It is the single most underrated feature in this product.

Why OpenBao and not HashiCorp Vault?

	OpenBao	HashiCorp Vault
Licence	MPL-2.0 (OSI open source)	BSL 1.1 (source-available)
Governance	Linux Foundation / OpenSSF, vendor-neutral	Single vendor (IBM)
Namespaces (multi-tenancy)	**Free**, since 2.3.1	Enterprise only
HSM/PKCS#11 auto-unseal	**Free** (2.7: via KMS plugin)	Enterprise only
Standby nodes serving reads	**Free** (Raft since 2.5, Postgres since 2.7)	Enterprise only
Cross-cluster replication	Not available	Enterprise only
Advanced policy roles	CEL	Sentinel (Enterprise)
FIPS-validated builds	Not available	Enterprise
Managed cloud	None — self-host only	HCP Vault
Cost for you	**$0**	$$ (or Enterprise $$$)

OpenBao forked from Vault 1.14.x when HashiCorp relicensed to BSL in Aug 2023. The data-plane APIs are compatible, so hvac, Terraform/OpenTofu providers and existing scripts generally keep working. But the APIs **diverged** after 1.14 — do not assume Vault 1.15+ features exist. And note the CLI is `bao`, not `vault`.

Canonical sources used to build this course

- `https://openbao.org/docs/` — 2.7.x documentation
- `https://github.com/openbao/openbao` — source tree at tag `v2.7.1`, incl. `CHANGELOG.md`
- `https://github.com/openbao/openbao-plugins` — external plugin builds
- `https://github.com/openbao/openbao-helm` — Helm chart `0.30.x` for Kubernetes
- `https://openbao.org/community/release-notes/2-7-0/` — 2.7 release notes

✓ How this course was verified (and how to re-verify it)

This is not "I read the docs and wrote some commands". The course was checked against the **actual `bao` v2.7.1 binary** (`a5db72cef75c24b920ade02065b18dd8eb666bac`).

Check	Result
Every `bao …` command and subcommand in this course	✓ exists in v2.7.1
OIDC/JWT mount paths (`config`, `role/<name>`, `login`, `oidc/*`, `cel/*`)	✓ enumerated wi
`oidc_discovery_url` against a live mock IdP **and** against real GitHub	✓ `status: "vali`

Check	Result
Full OIDC authorization-URL generation (PKCE S256 + nonce + scopes)	✓ real URL produ
Full GitHub-Actions OIDC exchange (signed JWT → OpenBao token)	✓ token issued wi
Negative cases: wrong audience / wrong repo / wrong ref / tampered payload / missing `kid`	✓ all correctly rej
`token_policies_template_claims` deriving policies from claims	✓ `ci-prod` + `ci-`
Full CI flow end-to-end (OIDC token → OpenBao token → secret read → audit trail)	✓ run in `labs/la`
The audit trail showing `actor` / `repository` / `ref` / `workflow` / `environment`	✓ captured in the
Negative case: a token from a different repo	✓ refused — ”clai
Every built-in auth method type	✓ verified by ena
Every built-in secrets engine type	✓ verified by ena
Every server-config HCL block in this course	✓ passes `bao ope`
The three lab `openbao.hcl` files	✓ pass `bao opera`
KV v2 read/write/version/metadata	✓ run against a li
AppRole login, token creation, accessors	✓ run against a li
Transit encrypt/decrypt round trip	✓ run against a li
PKI root → intermediate → leaf chain	✓ run against a li
Namespaces, wrapping, quotas, lease revocation	✓ run against a li
Plugin OCI images on `ghcr.io/openbao`	✓ live tag list pull
All Compose files, shell scripts, JSON, Python	✓ syntax-validate

Things that verification caught (and that most tutorials get wrong)

These are the 2.7.1 realities that this course now teaches correctly:

Wrong (pre-2.7 / Vault habits)	Correct in 2.7.1
`disable_mlock = false` in the config	**No such option** — removed in 2.0.0; the s
`telemetry { prometheus = true }`	`telemetry { prometheus_retention_tim`
`bao audit enable file file_path=…`	**API audit creation is disabled by defau**
`bao kv get -metadata secret/x`	`bao kv metadata get -mount=secret x`
`bao policy fmt <policy-name>`	`bao policy fmt <local-file>` — it rewri
`bao logout`	**No such command.** Delete `~/.vault-to`
`~/.bao-token`	The default token file is `~/.vault-token`
`bao token create -field=accessor`	The accessor is under auth; use `-format=j`
`bao read -field=role_id auth/approle/role/x`	`bao read -field=role_id auth/approle`
`bao sys capabilities-self <path>`	No CLI command — it is an API endpoint. U
`bao write auth/oidc/oidc …`	The config path is **`auth/<mount>/config`**;
`oidc_discovery_url="…/.well-known/openid-configuration"`	**Rejected** — it must be the **base URL**; Ope
`client_id=` / `client_secret=` on the OIDC config	Silently ignored — the keys are **`oidc_clie**
`bound_claims=k=v` repeated on the CLI	`bound_claims` is a **map** — use a JSON writ
`bao status` inside a `set -e` script	It **exits 2** when sealed/uninitialised — capt
`jq -r '.sealed // "true"'`	jq’s `//` treats `false` as empty → reports an
`bao write -f transit/encrypt/...` expecting one bare value	`-f` is ”raw format”, not ”one field” — use -
`bao read` on `/v1/pki_int/ca`	That endpoint returns **raw PEM** — use cur
`bao write sys/quotas/rate-limit rate=…`	Needs a name: `sys/quotas/rate-limit/<`
`bao -namespace=x secrets list`	The flag goes **after** the subcommand: `bao`
`key_bits=ecdsa-p521` in PKI	`key_bits` is an **integer** (521), and `key_typ`
`pki/root/generate/internal` with `issuer_name="Corp Root"`	Issuer names cannot contain spaces — use
PKI CA TTL shorter than requested	**`bao secrets tune -max-lease-ttl=… fir**
`bao write pki/ca` to export a chain	The CA endpoint returns **raw PEM** — use c
`storage "file"`	**Removed in 2.7.0** — use `raft` or `postgre`
Two `listener` blocks to give the cluster port mTLS	The cluster listener **inherits** its parent liste

※ **Reproduce the checks yourself:**

```
# 1. Get the binary
curl -sL -o bao.tgz \
  https://github.com/openbao/openbao/releases/download/v2.7.1/openbao_2.7.1_linux_amd64.t
tar xzf bao.tgz && sudo install -m0755 bao /usr/local/bin/bao

# 2. Validate any config
bao operator validate-config -config=./openbao.hcl

# 3. Prove the engine/auth tables
bao server -dev &
for t in oidc jwt approle userpass kubernetes cert ldap kerberos radius github; do
  bao auth enable -path="t_$t" "$t" >/dev/null 2>&1 && echo "built-in: $t" || echo "plugi
done
```

The one-paragraph summary

OpenBao is a **secret manager**: it holds your passwords, API keys, certificates and encryption keys encrypted at rest, decides who may read which of them through code-reviewable policies, hands out **short-lived dynamic credentials** instead of long-lived ones, and writes **an immutable audit log** of every single request. Your applications never need to know a static secret; they ask OpenBao "give me a Postgres credential that expires in an hour" and OpenBao creates a real database user, hands it over, and deletes the user when the lease expires. That single loop — short-lived, auditable, automatically revoked — is the whole product. Everything else is detail.

Level 0 — Foundations

> **Goal of this level:** you finish it able to explain to a colleague, in plain English, what OpenBao does, what problem it solves, what it explicitly does not do, and you have OpenBao running in Docker on your laptop.

0.1 The problem, stated brutally

Picture a normal enterprise codebase. Where do secrets live?

```
.env file        → git-committed (oops), on every dev's laptop, in every image layer
K8s Secret         → base64, NOT encrypted, visible to anyone with get-secret RBAC
CI/CD variables    → masked in logs, but plaintext at rest in the CI vendor's DB
Config file        → committed, rotated never
Hardcoded          → in the git history forever
```

Every one of those is the same four problems:

1. **No expiry.** A leaked key from 2023 still works today.
2. **No attribution.** Who read the production DB password? The logs don't say.
3. **No blast-radius control.** The same key is in 40 services, so you can't rotate it.
4. **No revocation.** You need the app restarted to pick up a new value.

And a fifth, sneakier one:

5. **Static secrets scale linearly with risk.** Every replica of every service that holds a long-lived credential multiplies your attack surface.

OpenBao attacks exactly this. It is a centralised, encrypted, policy-controlled, audited place where secrets live, plus a machine that can mint short-lived credentials on demand so that step 5 stops being a problem.

0.2 What OpenBao actually is (and is not)

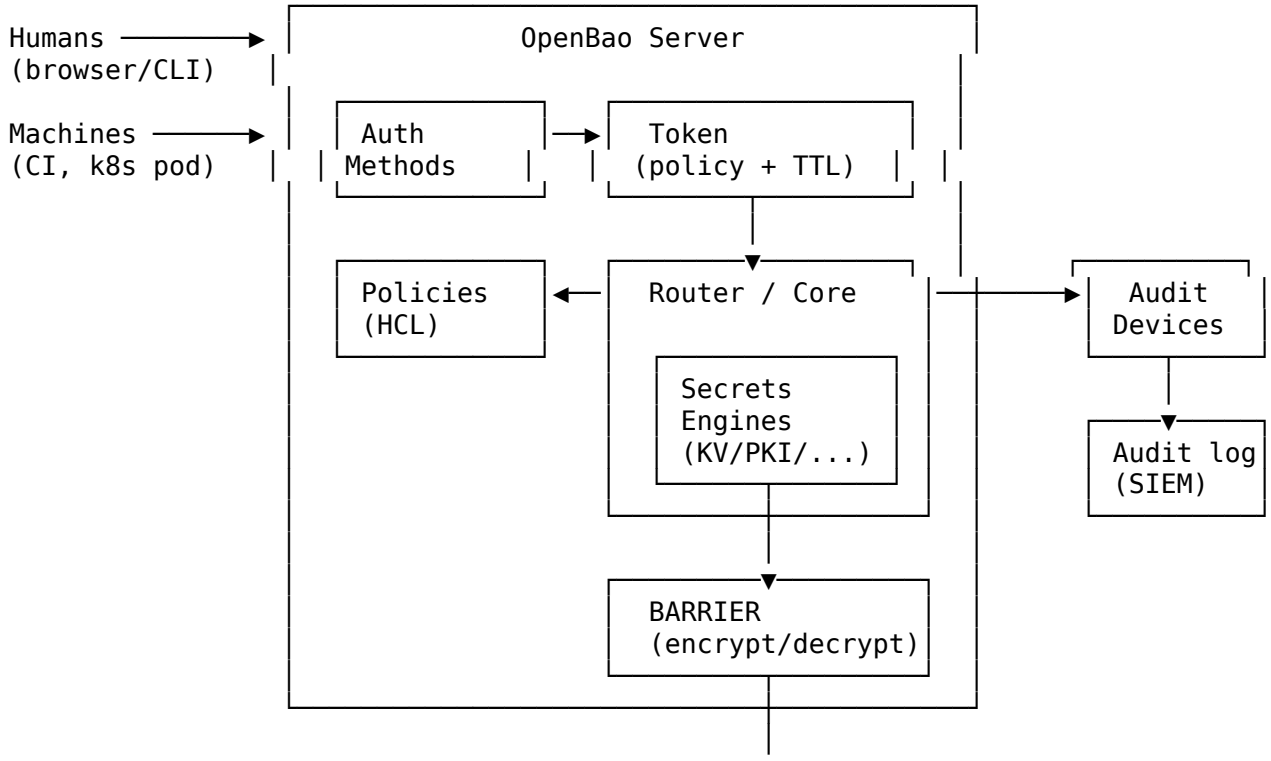
It IS

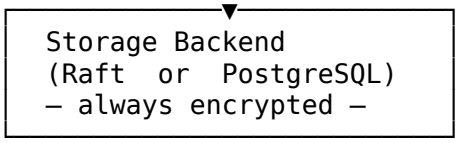
- An **encrypted key/value + policy store** with an HTTP API.
- An **identity broker**: it logs users in against your existing IdP (Keycloak, LDAP, GitHub, Kubernetes, cloud IAM) and maps them to OpenBao policies.
- A **dynamic credential factory**: it creates real credentials in your database, your cloud IAM, your SSH, and then destroys them on a timer.
- A **certificate authority** (PKI) that issues and rotates internal TLS certs.
- An **encryption service** (Transit) so apps never hold a raw encryption key.
- An **audit ledger**: every request and response is written to a tamper-evident log you ship to a SIEM.

It is NOT

- A general-purpose config store. Don't put non-secret config in it.
- A password manager for humans. It's for machines; humans get tokens from it.
- A database. It stores small things: passwords, keys, certs, connection strings.
- A replacement for your IdP. Keycloak stays Keycloak. OpenBao just trusts it.
- Automatically safe. **A secrets manager with bad policies is worse than no secrets manager**, because now everything routes through one place.

0.3 The one architectural diagram you need to remember





Read it bottom-up:

1. **Storage holds only ciphertext.** If someone steals your Postgres disk or your Raft snapshot, they get encrypted blobs. Useless without the root key.
2. **The barrier** is the crypto engine that encrypts/decrypts everything crossing that boundary. It lives in memory and its key is protected by the seal.
3. **The seal** is the master secret. If it's Shamir, humans hold shares. If it's auto-unseal, a KMS/HSM holds it.
4. **The router** takes a request path, asks the **policy engine** "is this token allowed to do this to this path?", and if yes dispatches to the right **auth method** (to verify identity) or **secrets engine** (to get data).
5. **Every request and response is written to every enabled audit device.**
6. **Tokens** are the currency: they carry identity + policies + TTL.

Everything in OpenBao is a **path** and a **capability** on that path. That single idea explains policies, plugins, namespaces, and why `sys/` is just another path.

0.4 Ten vocabulary words that make everything else obvious

Read this table twice. It's the densest value in the whole course.

Word	Plain English
Path	A URL-ish string. `secret/data/myapp`, `sys/health`, `auth/userpass/login/bob`.
Mount	Attaching a plugin to a path prefix. `bao secrets enable -path=secret kv-v2` attaches "t
Token	A session credential. Carries which policies and how long. Not a password — a lease.
Policy	HCL rules: "this token may `read` on path *X*". **Deny by default.**
Capability	The verb: `create`, `read`, `update`, `patch`, `delete`, `list`, `scan`, `sudo`, `deny`.
Lease	A timer attached to a dynamic secret or service token. When it expires → secret revoked.
Seal / Unseal	Locking / unlocking the master key. Sealed OpenBao serves almost nothing.
Auth method	A way to prove who you are (→ a token). `userpass`, `oidc`, `approle`, `jwt`, `kubernetes`.
Secrets engine	A way to store, generate or encrypt secrets. `kv`, `transit`, `pki`, `database`.
Audit device	A place every request/response gets written. `file`, `socket`, `syslog`, `http`.

Two more you'll meet fast:

Word	Plain English
Root token	A token with the `root` policy. Can do anything. Treat like the nuclear key.
Namespace	An isolated mini-OpenBao inside one instance (multi-tenancy). Free in OpenBao.

0.5 Threat model — what is OpenBao actually protecting you from?

This matters, because "security" without a threat model is just vibes.

In scope (OpenBao defends against these):

Threat	Defence
Stolen disk / stolen DB backup	Barrier encryption; storage holds only ciphertext
Stolen laptop with `.env`	Secrets live server-side, not on the laptop
Insider with prod read access	Path-scoped policies, per-token TTLs, audit trail
Leaked key in a git repo	Rotation; and if you use the PKI, certs rotate automatically
Compromised service A reading service B's secret	Separate paths + separate policies per app
Long-lived credential abuse	Dynamic creds that die in an hour
"Who did it?"	Audit log, requester identity, source IP
Insider tampering with logs	Ship audit logs to a separate immutable store (SIEM/WORM

Out of scope (you must handle these yourself):

Threat	Why OpenBao can't help
Someone with the **root token**	Root is root. It can read every secret.
Someone who **stole your unseal shares**	Threshold secrets: you need k of n.
A **malicious application** that legitimately has a token	Policies are your defence — write them well.
Runtime memory scraping of your app	A process with the secret in RAM can leak it. OpenBao
Supply-chain compromise of your app	If your app is malicious, its token is effectively the att
Compromised host root	A root attacker can read the unseal key from memory
OpenBao itself having a bug	Mitigated by patching. That's it.

> **The honest truth:** OpenBao moves the blast radius from "N services × M secrets scattered everywhere" to "one hardened, audited, encrypted service." That is an enormous improvement. It is not magic.

0.6 Install it: Docker first (as you prefer)

You said you like Docker over manual installs. Agreed — and OpenBao is designed for it. Two images matter:

Image	Use
`openbao/openbao:2.7.1`	Standard. Has a shell, useful for debugging.
`openbao/openbao-distroless:2.7.1`	Production-hardened: `distroless/static`, contains **only** the bao bina

> ⚠ **2.7.0 changed container behaviour you must know:**
>
> - The image now runs as the **`openbao` user**, not `root`. The entrypoint drops privileges automatically. If you mounted host paths, they must be writable by uid `100` / gid `1000` (check with `docker run --rm openbao/openbao:2.7.1 id`).
> - **All `VOLUME` instructions were removed** in 2.7.0 — you must declare your own volumes.
> - Architecture-specific tags (`2.7.1-arm64`) are **not published**. Use the multi-arch tag.

0.6.1 Dev mode (5 seconds, for learning)

```
docker run --rm -it \
  --name openbao-dev \
  -p 8200:8200 \
  -e BAO_DEV_ROOT_TOKEN_ID="dev-root-token" \
  -e BAO_DEV_LISTEN_ADDRESS="0.0.0.0:8200" \
  openbao/openbao:2.7.1 \
  server -dev
```

You should see:

```
==> OpenBao server configuration:
...
Unseal Key: ...
Root Token: dev-root-token
```

`server -dev` is a learning tool only. It is:

- in-memory storage (everything is lost on exit),
- unsealed automatically,
- root token printed in the logs and fixed to `dev-root-token`,
- running with a permissive setup.

Never ship `-dev`. In Level 2 we build the real thing.

While it's running, in another terminal:

```
export BAO_ADDR=http://127.0.0.1:8200
export BAO_TOKEN=dev-root-token

bao status          # -> Sealed: false, Initialized: true
bao secrets list
bao policy list
```

> ※ **`server -dev` already mounts a KV v2 engine at `secret/`.** If you run `bao secrets enable -path=secret kv-v2` you'll get `path is already in use at secret/`. That's expected, not a bug.

0.6.2 Production-shaped single node (the "real" dev box)

Even locally, run it the way you'd run it in prod. This is the single biggest habit difference between people who learn this tool and people who get paged.

```
mkdir -p ~/openbao-lab/{data,config,logs}
```

`~/openbao-lab/config/openbao.hcl`:

```
ui            = true
api_addr      = "127.0.0.1:8200"
cluster_addr  = "127.0.0.1:8201"

storage "raft" {
  path    = "/openbao/data"
  node_id = "openbao-node-1"
}

listener "tcp" {
  address         = "0.0.0.0:8200"
  cluster_address = "0.0.0.0:8201"
  tls_disable     = true          # dev only! see Level 5 for production TLS
}
```

> ⚠ **There is no `disable_mlock` option in OpenBao 2.x.** It was removed in **2.0.0** (the "mlock removal" RFC, GH-363) and the config parser will **refuse to start** if you leave it in:
>
> `OpenBao has dropped support for mlock. Please remove the line "disable_mlock" = false from your config and disable or encrypt swap instead.`
>
> OpenBao no longer calls `mlock()` at all. The hardening it now asks for is **swap**: disable swap for the OpenBao process, or encrypt it. Level 5 covers the full story (and the Docker equivalent is `--memory-swappiness=0`).

Run it:

```
docker run --rm -it \
  --name openbao-1 \
  -p 8200:8200 \
  -v ~/openbao-lab/data:/openbao/data \
  -v ~/openbao-lab/config:/openbao/config \
  openbao/openbao:2.7.1 \
  server -config=/openbao/config/openbao.hcl
```

0.6.3 The first interaction — read the unauthenticated surface

Before you authenticate, learn what OpenBao exposes to the world. This is a **security exercise you should do on every system you deploy**.

```
curl -s http://127.0.0.1:8200/v1/sys/health | jq
curl -s http://127.0.0.1:8200/v1/sys/seal-status | jq
curl -s http://127.0.0.1:8200/v1/sys/init | jq        # {"initialized":false}
```

`sys/health`, `sys/seal-status`, `sys/init`, `sys/leader` are the intended unauthenticated endpoints. They are not audited (see the audit chapter) and they leak version + seal state. **Do not expose them to the internet.**

Version fingerprinting note: `sys/seal-status` returns `version` and `commit_date`. That's useful for you and useful to an attacker. Restrict at the network layer.

0.6.4 The `bao` CLI without installing it

You don't need the binary on your host. Run it in a throwaway container that shares the network with the server:

```
docker run --rm -it --network host \
  -e BAO_ADDR=http://127.0.0.1:8200 \
  openbao/openbao:2.7.1 \
  status
```

Or install it locally for convenience (a single static binary — no deps):

```
# Linux
wget -qO- https://github.com/openbao/openbao/releases/download/v2.7.1/openbao_2.7.1_linux_amd
unzip openbao_2.7.1_linux_amd64.zip && sudo install -m 0755 bao /usr/local/bin/bao
bao version
```

```
# Windows (PowerShell) — you said Windows 11
winget install OpenBao.OpenBao     # or download the zip from the releases page
bao version
```

0.7 Your very first authenticated session

```
export BAO_ADDR=http://127.0.0.1:8200
export BAO_TOKEN=dev-root-token

# Who am I?
bao token lookup
```

```
{
  "data": {
```

```
    "accessor": "...",
    "creation_time": 1758...,
    "display_name": "token",
    "entity_id": "",
    "expire_time": null,
    "id": "dev-root-token",
    "meta": { "username": "dev-root-token" },
    "orphan": true,
    "policies": ["root"],
    "renewable": false,
    "ttl": 0
  }
}
```

Note `policies: ["root"]` and `ttl: 0` (never expires). That is the worst possible token to keep around, and in Level 5 you'll learn how to make sure it only ever exists for the 90 seconds you need it.

0.8 The "3 requests" that teach you the whole product

Do these three in order. They are the spine of the entire course.

```
# 1. STORE a secret
bao secrets enable -path=secret kv-v2
bao kv put -mount=secret myapp db_password="s3cr3t" api_key="abc123"
bao kv get -mount=secret myapp
```

```
{
  "data": {
    "data": { "api_key": "abc123", "db_password": "s3cr3t" },
    "metadata": { "created_time": "...", "version": 1 }
  }
}
```

```
# 2. CONTROL who can read it
bao policy write app-readonly - <<'EOF'
path "secret/data/myapp" {
  capabilities = ["read"]
}
EOF

# 3. GET a *temporary* credential, not a static one
bao secrets enable -path=db database
bao write db/config/postgres \
    plugin_name=postgresql-database-plugin \
    connection_url="postgresql://{{username}}:{{password}}@127.0.0.1:5432/app?sslmode=disable
    username="openbao_admin" \
    password="openbao_admin_pw" \
    allowed_roles="app"

bao write db/roles/app \
    db_name=postgres \
    creation_statements="CREATE ROLE \"{{name}}\" WITH LOGIN PASSWORD '{{password}}' VALID UN
    default_ttl="1h" max_ttl="24h"
```

```
bao read db/creds/app
```

```
{
  "data": {
    "username": "v-token-approle-abc123-2026-10-20",
    "password": "A1b2C3d4E5f6...",
    "lease_duration": 3600,
    "lease_id": "db/creds/app/..."
  }
}
```

That third one is the whole point of OpenBao. The password is for a database user that did not exist a second ago and will be deleted when the lease expires or is revoked. You never store it. You never rotate it. You just ask again.

0.9 Level 0 checkpoint

You are done with Level 0 when you can answer these without looking anything up:

1. What problem does OpenBao solve that a `.env` file does not?
2. What is stored on disk, and in what form?
3. What is a path? A capability? A lease?
4. What is the difference between an auth method and a secrets engine?
5. Which three things are **out** of OpenBao's threat model?
6. What is the difference between `server -dev` and a real server?

Next → **Level 1: CONCEPTS.md** — the full mental model, from the barrier up to namespaces.

0.10 Scratchpad

```
OpenBao version taught:  2.7.1
BAO_ADDR (dev):          http://127.0.0.1:8200
Dev root token:          dev-root-token
First KV mount:          secret/  (kv-v2)
First policy:            app-readonly
Lab dir:                 ~/openbao-lab/
```

> [SEC] **From this point on, every `export BAO_TOKEN=...` in your terminal history is a liability.** Use `bao login` and the token helper, or `BAO_TOKEN` only in throwaway shells. Level 5 covers the professional workflow.

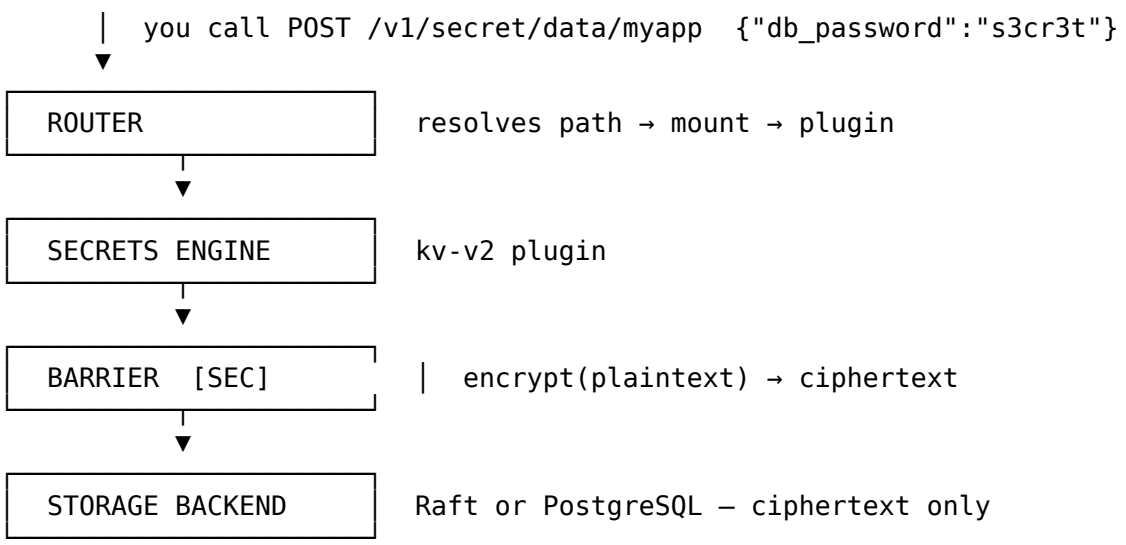
Level 1 — The Core Concepts

> **Goal:** you can now read any OpenBao documentation, any policy, and any config file and understand why it says what it says. This level is the foundation everything else is built on.

1.1 The server, the barrier, the storage backend

The chain of custody for a secret

```
  plaintext secret
```



Everything crosses the barrier. The barrier is a single in-memory key (the encryption key) that OpenBao holds only while unsealed. On disk, at rest, in backups, in snapshots, in the Postgres database — **only ciphertext**.

The three keys (get this right, it's a common confusion)

Key	Where it lives	What it protects
Encryption key ("keyring")	In memory, unsealed. Encrypted at rest.	All your data
Root key (barrier key)	Encrypted by the unseal key, stored next to the data	The encryption key
Unseal key	Split into N shares, given to humans (or held by a KMS)	The root key

So to read a secret: `unseal share(s) → root key → encryption key → plaintext`

Which is why **unseal is the crown jewels** and why auto-unseal moves that crown jewel into a KMS/HSM (Level 5) or a second OpenBao cluster (Level 6).

Storage backends — you have exactly two real choices

Backend	HA?	When to use
`raft` (integrated storage)	✓ Yes	Default choice. Self-contained, no external DB, snaps
`postgresql`	✓ Yes (`ha_enabled = true`)	You already run Postgres and want one backup/monit
`pebbledb`	✗ No	Non-HA durable, good perf, new in 2.7.0. Fine for sin
`in-memory`	✗	`-dev` mode only.
`file`	—	**REMOVED in 2.7.0.**

> ⚠ **2.7.0 removed the `file` storage backend.** If you see a tutorial using `storage "file" { path = "..." }`, that tutorial is pre-2.7 and is wrong. Use `storage "raft"`.

Raft config:

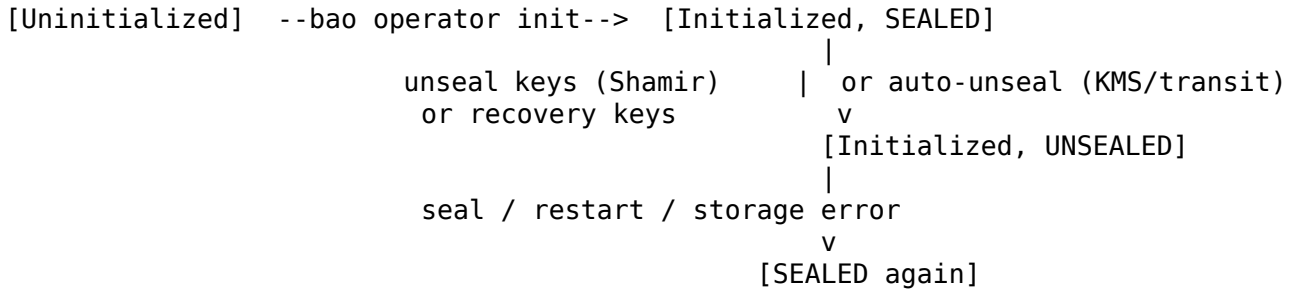
```
storage "raft" {
  path    = "/openbao/data"
  node_id = "openbao-node-1"
}
cluster_addr = "https://10.0.0.1:8201"   # REQUIRED with raft
```

PostgreSQL config (2.7 flavour):

```
storage "postgresql" {
  connection_url = "postgres://bao:bao@postgres:5432/openbao?sslmode=verify-full"
  ha_enabled     = "true"
  cluster_addr   = "https://10.0.0.1:8201"   # needed for 2.7 read-scaling
}
```

> **PostgreSQL + HA caveat (2.7.0):** read scalability on Postgres requires **physical replication** (streaming replication / Patroni), **not** logical replication. And it requires `ha_enabled = true` plus a reachable `cluster_addr` from standbys to the primary.

1.2 Seal / Unseal — the lifecycle



The four states you'll see

```
bao status
```

```
Sealed          true
Storage Type    raft
Initialized     true
```

State	Meaning	Can you log in?
Uninitialized	Never `bao operator init`'d	No — only `/sys/init` works
Initialized + Sealed	Keys exist but master key is locked	**No**
Initialized + Unsealed	Fully operational	Yes
Unsealed + Standby	Node is alive, leader is elsewhere	Yes (reads; writes forwarded)

Shamir seal (the default)

`bao operator init` splits the unseal key into N shares with a threshold T. **T of N** shares reconstruct it.

```
bao operator init -key-shares=5 -key-threshold=3
```

- 5 shares, any 3 unseal.
- Each share is a long base64 string.
- **Partial unsealing is per-node, not per-cluster.** Each node needs T shares entered on that node. (This trips up everyone once.)
- Shares are NOT stored in OpenBao. If you lose them, you're done (unless you have recovery keys).

Recovery keys (the emergency escape hatch)

With **auto-unseal**, `init` produces recovery keys instead of unseal keys. They are **not** unseal keys — they cannot decrypt the root key. They only authorize quorum operations (generating a new root

token, rotating keys).

```
bao operator init -recovery-shares=5 -recovery-threshold=3
```

> [ALERT] **The critical production warning from the docs, verbatim in spirit:** "If the seal mechanism or its keys are permanently deleted, then the OpenBao cluster cannot be recovered, even from backups." Auto-unseal creates a hard lifecycle dependency. Mitigate with IAM/SCP-style controls on the KMS key, and **rehearse the recovery-key path**.

Unsealing, step by step

```
bao status                      # -> Sealed: true
bao operator unseal <share-1>   # "Key 1 of 3 unsealed"
bao operator unseal <share-2>   # "Key 2 of 3 unsealed"
bao operator unseal <share-3>   # "Unseal complete"
bao status                      # -> Sealed: false
```

The state is **server-side**, so you can enter shares from different machines.

Sealing on purpose (incident response)

```
bao operator seal     # throws away the root key in memory
```

Useful if you suspect an active compromise: the cluster is instantly useless to the attacker until the threshold is met again. Requires root/sudo.

Generating a new root token when you're locked out

```
# Requires a quorum of recovery keys (auto-unseal) or unseal keys (Shamir)
bao operator generate-root -init
bao operator generate-root -format=json
```

> ✓ Since 2.6.0, `bao operator generate-root` uses the **authenticated** `/sys/generate-root-token` endpoints. The old **unauthenticated** `/sys/generate-root/attempt` endpoints are deprecated and disabled by default (2.7.1 also fixed a bug where they returned a response even when audit logging failed — `GHSA-q74w-hv5x-6hxf`). Never teach people the old endpoint.

Seal migration

You can move from Shamir → auto-unseal or KMS A → KMS B, but it **requires downtime** and both seal mechanisms must be available during the migration. Full procedure: Level 5. Always take a snapshot first.

1.3 Policies — the authorization model

Everything is a path, everything is a capability

OpenBao's model is a filesystem with verbs. A **policy** is a set of rules mapping paths → allowed capabilities.

```
# app-readonly.hcl
path "secret/data/myapp" {
  capabilities = ["read"]
}
```

Deny by default. An empty policy grants nothing. A token with only `app-readonly` cannot read `sys/health`, cannot list mounts, cannot write.

The nine capabilities

Capability	Means	HTTP verb
`create`	Create a new object at this path	POST/PUT
`read`	Read the value	GET
`update`	Overwrite an existing value	POST/PUT
`patch`	Partially modify	POST/PUT (merge)
`delete`	Delete	DELETE
`list`	List entries under a path	LIST/GET `?list=true`
`scan`	Recursively list	LIST/GET `?list=true` (recursive)
`sudo`	Perform the action as if holding root	any
`deny`	Explicitly deny (overrides any wildcard grant)	—

deny is special: it **takes precedence** over a broader wildcard grant. That's your "allow everything except X" tool.

Path globbing — the three patterns

```
path "secret/data/myapp"   { capabilities = ["read"] }   # exact
path "secret/data/*"       { capabilities = ["read"] }   # prefix, one segment+
path "secret/data/zip-*"   { capabilities = ["read"] }   # glob within a segment
path "secret/data/+/teamb" { capabilities = ["read"] }   # "+" = any chars in ONE segment
```

Pattern	Matches	Does NOT match
`secret/data/*`	`secret/data/a`, `secret/data/a/b`	`secret/dataX`
`secret/data/+/teamb`	`secret/data/foo/teamb`, `secret/data/bar/teamb`	`secret/data/foo/bar/teamb`
`secret/data/+/*`	`secret/data/foo/anything/deep`	—

> ⚠ **Case sensitivity:** inbound request paths are canonicalized (lower-cased) before ACL checks, but **the paths you write in a policy are NOT**. If your engine is case-insensitive, write your ACL paths in canonical (lower-case) form, or the policy silently won't match.

Parameter-level control

```
path "secret/data/restricted" {
  capabilities = ["create"]
  allowed_parameters = {
    "foo" = []                      # any value
    "bar" = ["zip", "zap"]          # only these values
  }
  denied_parameters = {
    "legacy_field" = []             # never allow
  }
}
```

Templating — policies that react to identity

```
path "secret/data/{{identity.entity.aliases.auth_userpass_xxx.metadata.username}}/*" {
  capabilities = ["read", "list"]
}
```

This is how you get ”each user sees only their own folder” with one policy.

> [SEC] **2.6.0 hardened templating (GHSA-59w7-v8rr-pr4p).** A template value containing `+`, `*`, `/` or ` ` is now **rejected by default** — because a malicious identity metadata value could otherwise escalate into a different path. The escape hatches `allow_wildcards_in_identity_templates`, `allow_slashes_in_identity_templates`, `allow_commas_in_identity_templates` and `allow_globs_in_identity_templates` re-enable the old behaviour. **Leave them off.** If a template breaks, fix the source data, don’t enable the flag.

Policy composition

A token’s effective policy is the **union** of all attached policies, with deny always winning. There’s no ”deny overrides allow” ordering — it’s pure union + explicit deny.

Writing policies well

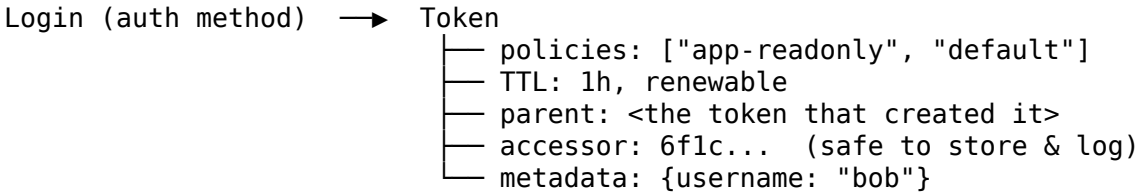
```
# ✗ Bad: too broad
path "secret/*" { capabilities = ["create","read","update","delete","list","scan"] }

# ✓ Good: explicit, minimal, per-app
path "secret/data/myapp"        { capabilities = ["read"] }
path "secret/data/myapp/config" { capabilities = ["read","update"] }
path "secret/metadata/myapp"    { capabilities = ["read","list"] }
path "secret/delete/myapp"      { capabilities = ["read"] }   # metadata only
path "secret/undelete/myapp"    { capabilities = ["update"] }
path "secret/destroy/myapp"     { capabilities = ["update"] }
```

> ※ **KV v2 needs four paths per secret**, not one. `data/`, `metadata/`, `destroy/`, `undelete/`. Forgetting `metadata/` breaks `bao kv metadata get` and version listing. Level 4 covers this in detail.

1.4 Tokens — identity, capability and time

The token model



Everything about access is on the token. Revoke the token → access gone.

Token formats (2.7.1)

Type	Prefix	Example
Service token	`s.`	`s.raPGTZdARXdY0KvHcWSpp5wWZIHNT`

Type	Prefix	Example
Batch token	`b.`	`b.n6keuKu5Q6pXhaIcfnC9cFNd`
Recovery token	`r.`	`r.JaKnR2AIHNk3fC4SGyyyDVoQ9O`

Regex: `[sbr]\.[a-zA-Z0-9]{24,}`

> ⚠ **Token internal structure is volatile.** Never write a script that parses the token string. Use the **accessor** instead.

Service vs batch tokens

	Service	Batch
Stored on disk	✓	✗ (encrypted blob)
Renewable	✓	✗
Revocable	✓	✗ (just stops working)
Can create children	✓	✗
Can be root	✓	✗
Has accessor	✓	✗
Has cubbyhole	✓	✗
Cost	Heavy (multiple storage writes)	Light (no storage)

Use case: batch tokens for high-volume, short-lived, read-only automation (they scale far better). Service tokens for everything else.

Token hierarchy and orphans

When a token creates a token, the child is a **child**. Revoking the parent revokes the whole tree. That is a security feature: you can't escape revocation by minting grandchildren.

An **orphan token** has no parent — it is the root of its own tree. You need this when a system issues tokens for other systems and must be able to revoke just its own subtree.

```
# Orphan token
bao token create -policy=app-readonly -ttl=1h -orphan=true

# Revoke a subtree but keep the accessor's token
bao token revoke -accessor <accessor>
```

Token accessors — the professional way to handle tokens

An accessor is a **safe handle** to a token. It can look up, renew, and revoke the token — but it **cannot be used to authenticate**. So you can store accessors in logs, databases and dashboards without leaking a credential.

```
# `-field=accessor` does NOT work here: the accessor lives under `auth`,
# and `-field` does not traverse into it. Use:
bao token create -policy=app-readonly -ttl=1h -format=json | jq -r .auth.accessor
# ...or for the token itself, `-field=token` works:
bao token create -policy=app-readonly -ttl=1h -field=token
```

> ※ **Production pattern:** the service that mints tokens for other services stores the **accessor** keyed by job ID. When the job finishes, it revokes by accessor — instant, and no credential ever touched disk.

TTLs, max TTLs, periodic tokens

- **TTL** — how long the token is currently valid.
- **Renew** — extend it. `bao token renew`.
- **Max TTL** — the hard ceiling on lifetime. Determined by:
  1. System `max_lease_ttl` (default 768h = 32 days)
  2. The mount's `max_lease_ttl` (can override, longer or shorter)
  3. The auth method's suggested max (can only be shorter)
- **Explicit max TTL** on the token = absolute limit, wins over everything.
- **Periodic token** — TTL resets on every renewal. The only way (besides root) to have an effectively unlimited token, but it dies if you stop renewing. Good for long-lived services that must hold a DB connection pool.

```
bao token create -policy=app -ttl=1h -renewable=true -explicit-max-ttl=24h
bao token create -policy=app -period=24h          # periodic
```

Batch tokens (2.7 has no batch token changes, but they're underused)

```
bao token create -policy=app-readonly -ttl=5m -batch=true
```

The `default` policy

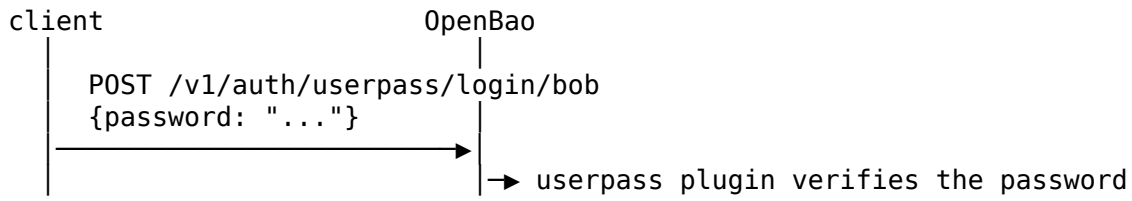
Every token gets the `default` policy attached automatically. It grants `lookup` on `auth/token/lookup-self` and `renew-self`. If you ever see "permission denied" on `bao token renew`, check that `default` wasn't removed from the token.

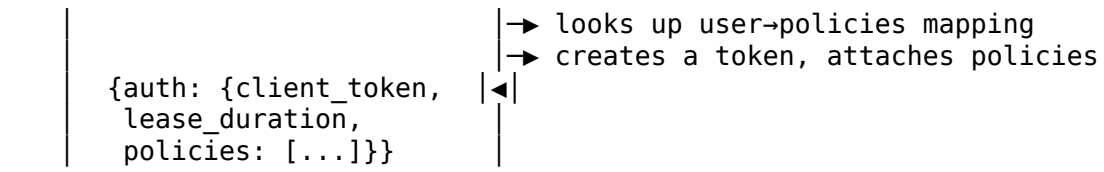
1.5 Auth methods — proving who you are

An auth method takes a credential, verifies it against an internal or external system, and returns a **token**.

Auth method	Best for	Human/Machine	Built-in in 2.7.1?
`userpass`	Dev, tiny setups, break-glass	Human	✓
`oidc`	**Your Keycloak** — browser login	Human	✓
`jwt`	JWTs from any issuer (incl. GitHub Actions)	Machine	✓
`approle`	Machines/apps in VMs or containers	Machine	✓
`kubernetes`	Pods in K8s	Machine	✓
`cert`	mTLS client certs	Machine	✓
`token`	The token store itself — **cannot be disabled**	—	✓ (always mounted)
`ldap`	AD/LDAP	Human	✗ **plugin now**
`kerberos`	AD/Kerberos	Human	✗ **plugin now**
`radius`	Network devices	Human	✗ **plugin now**
`aws` / `azure` / `gcp`	Cloud IAM	Machine	✗ **plugin now**
`github`	GitHub	Human	✗ **plugin** (`auth-github`)

How a login works





The **mapping from external identity → policies** is where the real design work happens. Two ways:

1. **Direct mapping** — the auth method's own config says "user `bob` gets policy `app-readonly`". Simple, brittle at scale.
2. **Identity groups** — create an OpenBao entity (an identity), an alias (that identity's login in a given auth method), and groups with policies. Then map the auth method's users to groups. This scales and is the right model for anything beyond a handful of people. (Level 3.)

AppRole — the machine auth method you'll actually use

AppRole gives you two values: a `role_id` (like a username) and a `secret_id` (like a password). Login with both, get a token.

```
bao auth enable approle

bao write auth/approle/role/myapp \
    secret_id_ttl=10m \
    secret_id_num_uses=1 \
    token_num_uses=0 \
    token_ttl=20m \
    token_max_ttl=1h \
    secret_id_bound_cidrs="10.0.0.0/8"

bao read -field=role_id auth/approle/role/myapp/role-id/role-id
bao write -f auth/approle/role/myapp/secret-id
```

Login:

```
bao write auth/approle/login role_id=... secret_id=...
```

The security pattern (this matters): the `role_id` is not secret. The `secret_id` is. In a proper design, the `role_id` is baked into the app image/config (it's not a secret), and the `secret_id` is **injected at deploy time by a trusted party** (CI, an orchestrator, an operator). If a `secret_id` is ever used twice, rotate it. `secret_id_num_uses=1` plus `secret_id_ttl=10m` means a leaked secret_id is worthless after 10 minutes.

1.6 Secrets engines — storing, generating, encrypting

Engine	Does	Typical use
`kv` / `kv-v2`	Stores arbitrary key/value, with versioning (v2)	API keys, connection strings
`transit`	Encrypt/decrypt/sign/verify as a **service**; keys never leave	App data at rest, envelope encryptio
`pki`	Issues X.509 certificates; acts as a CA	Internal TLS everywhere, mTLS
`database`	Creates **real** users in a DB, deletes them on revoke	Dynamic Postgres/MySQL creds
`totp`	Generates/validates TOTP codes	Step-up MFA
`ssh`	Issues SSH certificates	Replace `authorized_keys` with short
`cubbyhole`	Per-token private storage	Stash a credential during a multi-ste
`identity`	Identity store internals	—
`ldap`	LDAP secrets	✗ plugin in 2.7

KV v1 vs KV v2 — always use v2

	KV v1	KV v2
Versioning	✗	✓ (default 10 versions)
Soft delete / undelete / destroy	✗	✓
Check-and-Set (CAS)	✗	✓ (`cas` / `cas_required`)
Metadata separate from data	✗	✓
Storage overhead	Lower	Higher

```
bao secrets enable -path=secret kv-v2
```

KV v2 stores data at `secret/data/<key>` and metadata at `secret/metadata/<key>`. That's why policies need both paths.

Transit — encryption as a service

Your app encrypts; OpenBao encrypts. The key **never leaves OpenBao**.

```
bao secrets enable transit
bao write transit/keys/myapp type=aes256-gcm96
bao write transit/encrypt/myapp plaintext=$(echo -n "hello" | base64)
# {"ciphertext":"vault:v1:..."}
```

`vault:v1:<base64>` — note the `v1` is the **key version**. Rotating the key (`transit/keys/myapp/rotate`) keeps old ciphertext decryptable. That's the whole point of key versioning.

PKI — your internal CA

The production pattern is **offline root → online intermediate → short-lived leaves**:

```
offline root CA (air-gapped, 10-year cert)
        │ signs
        ▼
intermediate CA (in OpenBao, 1-year cert)  ← private key in OpenBao (or External Key / HSM in 2.7)
        │ signs
        ▼
leaf certs for apps (24h–7d, auto-renewed)
```

Short-lived leaves make revocation almost a non-issue: a stolen cert expires before it's useful.

Database — dynamic credentials

```
bao secrets enable -path=db database
```

OpenBao connects with a **privileged** account, then per-request:

1. Creates a unique username (`v-token-approle-xxx-2026-10-20`).
2. Creates a random password.
3. Sets `VALID UNTIL` = lease expiry.
4. Hands them out with a lease.
5. On revoke/expiry → `DROP` the role.

The privileged account only needs `CREATEROLE` (+ connect), never `SUPERUSER`.

1.7 Leases — how secrets die on schedule

```
bao read db/creds/app
→ { "lease_id": "db/creds/app/...", "lease_duration": 3600 }
                  │
                  ├── renew:  bao lease renew -increment=3600 db/creds/app/...
                  └── revoke: bao lease revoke db/creds/app/...
```

- Every **dynamic secret** and every **service token** has a lease.
- When it expires, OpenBao revokes the secret (drops the DB user, etc.).
- **Prefix-based revocation** is a gift during an incident:

```
bao lease revoke -prefix auth/userpass/     # kill every userpass login
bao lease revoke -prefix db/creds/           # kill every dynamic DB cred
```

- The KV engine does **not** issue leases (static data), though it may report a duration.
- `increment` on renew is **from now**, not from the end of the current lease.

1.8 Audit devices — the ledger

Every request and every response is written to **every enabled audit device**.

```
# 2.7.1: audit devices are declared in the SERVER CONFIG, not via the API.
#         See §5.5 for the declarative `audit "file" "file" { ... }` stanza.
```

> ⚠ **In OpenBao 2.7.1, audit devices cannot be created via the API by default.** `bao audit enable` returns "cannot enable audit device via API; use declarative, config-based audit device management instead". Define the device in the **server config file** instead (see §5.5):
>
> ```
> audit "file" "file" {
>   description = "Local audit log (declarative)."
>   options {
>     file_path = "/openbao/logs/audit.log"
>     mode      = "0600"
>     log_raw   = "false"
>   }
> }
> ```
>
> The temporary escape hatch is the `unsafe_allow_api_audit_creation` server option (toggle with `SIGHUP`).

The audit log is a structured record:

```
{
  "type": "response",
  "auth": { "client_token": "hmac-sha256:...", "policies": ["app-readonly"] },
  "request": { "id": "...", "client_token": "hmac-sha256:...", "operation": "read",
               "path": "secret/data/myapp" },
  "error": "",
  "response": { "data": { "data": { "api_key": "hmac-sha256:..." } } },
  "auth": { "display_name": "token", "policies": ["app-readonly"] }
}
```

Secrets are HMAC'd, not logged in plaintext. OpenBao replaces sensitive values with `hmac-sha256:<hex>` using a per-device key, so you can correlate values across logs without exposing them. `log_raw = true` disables that — **never in production**.

These paths are NOT audited (by design):

`sys/init`, `sys/seal-status`, `sys/seal`, `sys/unseal`, `sys/leader`, `sys/health`, `sys/storage/raft/bootstrap`, `sys/storage/raft/join`

plus, if the listener allows it: `sys/metrics`, `sys/pprof/*`, `sys/in-flight-req`.

Critical operational rule: OpenBao considers a request successful if it can log to **at least one** audit device. If all audit devices fail, **OpenBao refuses to serve requests.** That's the correct fail-closed behaviour for a security system — and it's why you must monitor the audit sink's disk space and alert on audit failures.

Run at least two audit devices (e.g. a local file + a remote socket) so you have redundancy and tamper-evidence.

1.9 Identity — entities, aliases, groups

The identity system is a graph, and it's how you scale authorisation.

```
  Entity (a person or machine, namespace-scoped)
    ├── Alias  (how it logs in: auth method + metadata)
    ├── Group  (a named bucket of entities)
    │     └── Policies
    └── Metadata
```

```
# Create an entity + alias
bao write identity/entity name="alice" \
    metadata=full_name="Alice Smith" \
    policies=app-readonly

bao write identity/entity-alias \
    name="alice" \
    canonical_id=$(bao read -field=id identity/entity/name/alice) \
    mount_accessor=$(bao auth accessor list -format=json | jq -r '.[]|select(.path=="auth/use
```

Groups:

```
bao write identity/group name=platform \
    policies=app-readonly,app-admin

bao write identity/entity/id/<id> \
    groups=platform
```

Then in the auth method, map a claim → group:

```
bao write auth/userpass/users/alice token_policies=app-readonly
# or for OIDC/JWT:
bao write auth/oidc/role/dev \
    user_claim=preferred_username \
    bound_claims=groups \
    claim_mappings=...
```

> [SEC] **Cross-namespace identity is off by default and should stay off.** `unsafe_cross_namespace_identity` exists only for Vault Enterprise compatibility and "allows for unconfined permission sharing." Do not enable it.

1.10 Namespaces — multi-tenancy in one instance

A namespace is a **mini-OpenBao** inside one instance. It has its own:

secret engines · auth methods · ACL/EGP/RGP policies · password policies
entities · identity groups · tokens · login paths

```
bao namespace create platform-team
export BAO_NAMESPACE=platform-team
bao secrets enable kv-v2
bao policy write mypolicy - <<'EOF'
path "secret/data/*" { capabilities = ["read","list"] }
EOF
```

Namespaces are **free** in OpenBao (since 2.3.1) — in HashiCorp Vault they're Enterprise-only. That alone is often the reason to choose OpenBao.

2.6.0 added namespace sealing: you can give a namespace its own Shamir seal and its own barrier keyring, so a tenant can revoke the operator's access to their namespace without touching other tenants.

Naming rules: no trailing `/`, no spaces, and not one of `.`, `..`, `root`, `sys`, `audit`, `auth`, `cubbyhole`, `identity`.

1.11 The sys/ namespace — the control plane

Everything under `sys/` is OpenBao's own machinery. Learn these:

Path	What it does
`sys/health`	Health check (unauthenticated)
`sys/seal-status`	Seal state, version (unauthenticated)
`sys/init`	Initialize the cluster (unauthenticated, once)
`sys/seal` / `sys/unseal`	Seal / unseal (unauthenticated, quorum-guarded)
`sys/mounts`	List/enable/disable secret engines
`sys/auth`	List/enable/disable auth methods
`sys/policies/acl`	ACL policy CRUD
`sys/policies/egp`	**E**ndpoint **G**overning **P**olicies (parameter rules)
`sys/policies/rgp`	**R**ole **G**overning **P**olicies (time-bound access)
`sys/audit`	Audit device CRUD
`sys/config/state/sanitized`	Current config with secrets redacted
`sys/tools/hash`	Hash arbitrary data (SHA-256 etc.)
`sys/tools/random`	Generate cryptographically random bytes
`sys/capabilities-self`	"What can I do on this path?"
`sys/capabilities`	"What can that token do on this path?"
`sys/leases/lookup`	Look up a lease
`sys/leases/revoke`	Revoke a lease or a prefix
`sys/quotas/config`	Rate limits and lease-count limits
`sys/leader`	Who is the leader
`sys/metrics`	Prometheus metrics (if enabled)
`sys/wrapping/unwrap`	Unwrap a response-wrapping token
`sys/external-keys`	2.7: map mounts to HSM/KMS-backed keys
`sys/workflows`	2.6: operator-defined multi-request workflows
`sys/generate-root-token`	2.6+: **authenticated** root token generation

`sys/capabilities-self` is your best debugging friend:

```
# There is no `bao sys capabilities-self` CLI command — it is an API
# endpoint. Use the CLI equivalent, or call the API:
bao token capabilities <token> secret/data/myapp
```

```
# → read

curl -s -X POST -H "X-Vault-Token: $BAO_TOKEN" \
  -d '{"path":"secret/data/myapp"}' \
  "$BAO_ADDR/v1/sys/capabilities-self"
# → {"capabilities":["root"],"secret/data/myapp":["root"],...}
```

1.12 Agent and Proxy — the client-side daemons

`bao agent` and `bao proxy` are separate binaries that run **next to your application** and handle authentication + caching for you.

	`bao agent`	`bao proxy`
Purpose	Auto-auth + render templates to files	Auto-auth + cache + local API
Talks to OpenBao	Yes	Yes
Talks to your app	No (writes files/env)	Yes (local HTTP API)
Best for	Sidecar, writing secrets to disk	Any app that speaks the OpenBao API

`bao agent` supports a `file` sink with templating, so your app reads `/secrets/db_password` from disk and the agent keeps it fresh.

```
pid_file           = "/tmp/bao-agent.pid"
vault { address = "https://openbao:8200" }

auto_auth {
  method "approle" {
    mount_path = "auth/approle"
    config = { role_id = "..." }
  }
  sink "file" {
    config = {
      path = "/secrets/token"
      mode = 0640
    }
  }
}

template {
  source      = "/secrets/db_password"
  destination = "/secrets/db_password"
  error_on_missing_key = true
  wait { min_duration = "5s" }
}
```

Level 7 wires this into Docker Compose for your FastAPI service.

1.13 The request lifecycle, end to end

1. TLS handshake
2. Listener accepts, HTTP request parsed
3. Token extracted (X-Vault-Token header, or BAO_TOKEN)

```
 4. Router resolves path → mount → plugin
 5. Policy engine: token policies ∩ path capabilities  → allow / deny
 6. (if login) auth method plugin verifies credential
 7. Plugin executes → result
 8. Result passes through the barrier (decrypt)
 9. Audit device writes request+response  ← happens for every step
10. HTTP response
```

If step 9 fails for all devices → **the request fails**. (Fail closed.)

1.14 Level 1 checkpoint

1. Explain the three keys (encryption / root / unseal) and how they chain.
2. What's the difference between unseal keys and recovery keys? Which one is useless if the KMS dies?
3. Write a policy that lets a token read `secret/data/myapp` but not list it.
4. Why does KV v2 need both a `data/` and a `metadata/` path in your policy?
5. When do you want a batch token instead of a service token?
6. What's the difference between a token and its accessor, and which is safe to write in a log?
7. Which paths are NOT audited, and what does that mean for your threat model?
8. What happens to OpenBao if all audit devices fail?

Next → Level 2: HANDS-ON.md — you build a real, working, policy-controlled OpenBao in Docker from scratch.

1.15 Scratchpad

```
Barrier chain:    unseal share → root key → encryption key → plaintext
Seal types:       Shamir (default) | auto-unseal (KMS/transit) | static
Policy verbs:     create read update patch delete list scan sudo deny
Token prefixes:   s. (service)  b. (batch)  r. (recovery)
KV v2 paths:      <mount>/data/<key>  <mount>/metadata/<key>  <mount>/destroy/<key>
Unaudited paths:  sys/init sys/seal-status sys/seal sys/unseal sys/leader
                  sys/health sys/storage/raft/{bootstrap,join}
Default policy:   attached to every token (lookup-self + renew-self)
```

Level 2 — Hands-On: From Zero to a Working OpenBao

> **Goal:** by the end of this level you will have a **real** (non-dev) OpenBao in Docker, with storage, TLS-ready config, an audit device, KV v2, a properly scoped policy, a human login path, a machine login path, and an app that pulls its own secrets. Everything below is a live command.

2.0 The plan for this level

```
Step 1  Build the Docker Compose stack (OpenBao + Postgres + Redis)
Step 2  Generate an internal TLS CA and server certificates
Step 3  Initialise + unseal
Step 4  Enable audit BEFORE anything else
Step 5  Enable KV v2 and write a secret
Step 6  Write policies (the good, minimal kind)
```

```
Step 7  Enable userpass, log in as a human
Step 8  Enable AppRole, log in as a machine
Step 9  Wire a FastAPI app to pull its secret
Step 10 Read the audit log and understand what you see
```

2.1 Step 1 — The Docker Compose stack

Create the lab directory:

```
mkdir -p ~/openbao-lab/{certs,config,data,logs,scripts}
cd ~/openbao-lab
```

docker-compose.yml

```
services:
  openbao:
    image: openbao/openbao:2.7.1
    container_name: openbao
    command: server -config=/openbao/config/openbao.hcl
    ports:
      - "127.0.0.1:8200:8200"
    volumes:
      - ./config:/openbao/config:ro
      - ./certs:/openbao/certs:ro
      - ./data:/openbao/data
      - ./logs:/openbao/logs
    environment:
      BAO_ADDR: https://127.0.0.1:8200
      BAO_CACERT: /openbao/certs/ca.crt
    # Swap protection (OpenBao 2.x does not use mlock — see Level 5, §5.2)
    memswap_limit: 0
    security_opt:
      - no-new-privileges:true
    healthcheck:
      test: ["CMD-SHELL", "BAO_CACERT=/openbao/certs/ca.crt bao status >/dev/null 2>&1 || exi
      interval: 10s
      timeout: 5s
      retries: 12
      start_period: 10s
    restart: unless-stopped

  postgres:
    image: postgres:16-alpine
    container_name: openbao-postgres
    environment:
      POSTGRES_USER: openbao
      POSTGRES_PASSWORD: ${POSTGRES_PASSWORD:?set POSTGRES_PASSWORD in .env}
      POSTGRES_DB: openbao
    volumes:
      - pgdata:/var/lib/postgresql/data
    healthcheck:
      test: ["CMD-SHELL", "pg_isready -U openbao"]
      interval: 5s
      timeout: 3s
```

```
      retries: 10

  redis:
    image: redis:7-alpine
    container_name: openbao-redis
    command: ["redis-server", "--requirepass", "${REDIS_PASSWORD:?set REDIS_PASSWORD in .env}
    volumes:
      - redisdata:/data
    healthcheck:
      test: ["CMD", "redis-cli", "-a", "${REDIS_PASSWORD}", "ping"]
      interval: 5s
      timeout: 3s
      retries: 10

volumes:
  pgdata:
  redisdata:
```

.env (git-ignored!):

```
POSTGRES_PASSWORD=change-me-postgres
REDIS_PASSWORD=change-me-redis
```

[SEC] **Create `.gitignore` right now** if this lives in a repo:

```
.env
.env.*
data/
logs/
certs/
```

[PG] **Note on the Postgres service here:** in this lab, Postgres is the **application database** for your FastAPI app, not OpenBao's storage (we're using Raft in this lab). Level 6 shows you the alternative where Postgres is OpenBao's storage backend.

2.2 Step 2 — TLS (do this from day one, not "in production later")

Self-signed internal CA. Five minutes now, no regrets later.

```
cd ~/openbao-lab/certs

# 1. CA key + cert
openssl genrsa -out ca.key 4096
openssl req -x509 -new -nodes -key ca.key -sha256 -days 3650 \
  -subj "/CN=openbao-lab-ca" -out ca.crt

# 2. Server key + CSR
openssl genrsa -out openbao.key 4096
openssl req -new -key openbao.key -subj "/CN=openbao" -out openbao.csr

# 3. SAN file — the SAN is what modern TLS actually checks
cat > san.cnf <<'EOF'
[req]
distinguished_name = req_distinguished_name
req_extensions     = v3_req
[req_distinguished_name]
[v3_req]
```

```
basicConstraints = CA:FALSE
keyUsage = digitalSignature, keyEncipherment
extendedKeyUsage = serverAuth
subjectAltName = @alt_names
[alt_names]
DNS.1 = openbao
DNS.2 = localhost
DNS.3 = openbao.local
IP.1  = 127.0.0.1
EOF

# 4. Sign
openssl x509 -req -in openbao.csr -CA ca.crt -CAkey ca.key -CAcreateserial \
  -out openbao.crt -days 825 -sha256 -extfile san.cnf -extensions v3_req

# 5. Restrict permissions (OpenBao will refuse world-readable plugin/key material)
chmod 600 ca.key openbao.key
chmod 644 ca.crt openbao.crt
```

config/openbao.hcl

```
ui            = true
api_addr      = "https://openbao:8200"
cluster_addr  = "https://openbao:8201"
cluster_name  = "openbao-lab"

# Never use the "file" storage backend: removed in OpenBao 2.7.0
storage "raft" {
  path    = "/openbao/data"
  node_id = "openbao-node-1"
}

listener "tcp" {
  address         = "0.0.0.0:8200"
  cluster_address = "0.0.0.0:8201"
  tls_cert_file   = "/openbao/certs/openbao.crt"
  tls_key_file    = "/openbao/certs/openbao.key"
  tls_min_version = "tls12"
}

# ⚠ There is NO `disable_mlock` option in OpenBao 2.x.
#    It was removed in 2.0.0 (mlock-removal RFC) and the server
#    REFUSES TO START if it is present. See §5.2 for the swap-based
#    hardening OpenBao expects instead.

telemetry {
  prometheus_retention_time = "30s"
  disable_hostname = false
}

# Tighten request handling
default_max_request_duration = "60s"
```

> ⚠ **`api_addr` and `cluster_addr` must be reachable by clients.** In a Compose network, `https://openbao:8200` is the service name, but your laptop can't resolve it. Two options:

- Set `api_addr = "https://127.0.0.1:8200"` (reachable from the host) and `cluster_addr = "https://openbao:8201"` (reachable between nodes), **or**
- Keep the service name and set `BAO_ADDR` on the host to `https://127.0.0.1:8200` while adding `127.0.0.1 openbao` to `/etc/hosts`.

For a single-node lab the simplest correct choice is: `api_addr = "https://127.0.0.1:8200"`, `cluster_addr = "https://openbao:8201"`.

Start it:

```
cd ~/openbao-lab
docker compose up -d
docker compose logs -f openbao
```

You'll see it start **sealed** and **uninitialized**.

2.3 Step 3 — Initialise and unseal

```
export BAO_ADDR=https://127.0.0.1:8200
export BAO_CACERT=~/openbao-lab/certs/ca.crt

bao status
```

```
{
  "sealed": true,
  "initialized": true,     // ← in your case this is false on first boot
  "storage_type": "raft",
  ...
}
```

Initialise

```
bao operator init -key-shares=5 -key-threshold=3
```

Output (abridged):

```
Unseal Keys:
Unseal Key 1: <base64...>
Unseal Key 2: <base64...>
Unseal Key 3: <base64...>
Unseal Key 4: <base64...>
Unseal Key 5: <base64...>

Initial Root Token: hvs.CAESIJ...
```

Do this properly, even in a lab. In a real engagement you would:

- `-pgp-keys="alice.asc,bob.asc,carol.asc"` so the shares are encrypted to individual GPG keys,
- `-root-token-pgp-key="admin.asc"` for the root token.

The point of the exercise is to build the habit.

Unseal

Each node needs its own threshold of shares entered **on that node**:

```
bao operator unseal '<Unseal Key 1>'
bao operator unseal '<Unseal Key 2>'
bao operator unseal '<Unseal Key 3>'

bao status    # -> "sealed": false
```

Log in with the root token

```
export BAO_TOKEN='hvs.CAESIJ...'
bao token lookup
```

※ **Better: use the token helper** so the token isn't in your shell history.

```
bao login -token-only -no-store   # prints the token, doesn't save it
# or store it in the helper (macOS/Linux):
bao login -method=userpass -no-store
```

Level 5 covers root token rotation and the "no root token at rest" pattern.

2.4 Step 4 — Audit FIRST (before you store anything)

This is the habit that separates professionals from hobbyists: **enable audit before there is anything to audit.**

⚠ **Important 2.7.1 change: audit devices can no longer be created via the API by default.** `bao audit enable` now fails with:

```
cannot enable audit device via API; use declarative, config-based
audit device management instead
```

The **right** way is the **declarative audit stanza in the server config file** — which has the additional benefit that a node can never come up with audit missing. Add this to `config/openbao.hcl` and restart:

```
audit "file" "file" {
  description = "Local audit log (declarative, 2.4.0+)."
  options {
    file_path = "/openbao/logs/audit.log"
    mode      = "0600"
    log_raw   = "false"
  }
}
```

Note the **two labels**: the first is the device type, the second is the mount path. `options` is a block, and its values are **strings** (hence `log_raw = "false"` with quotes). Validate before you restart:

```
bao operator validate-config -config=config/openbao.hcl
```

The escape hatch (for one-off API creation, e.g. adding a second device in an emergency) is the `unsafe_allow_api_audit_creation` server option, which can be toggled at runtime with `SIGHUP`:

```
unsafe_allow_api_audit_creation = true
```

Enable it, create the device, then set it back to `false` and `SIGHUP` again. The option name says everything you need to know about it.

Once the config-based device is up, verify it:

```
bao audit list -detailed
```

```
{
  "file/": {
    "type": "file",
    "options": {
      "file_path": "/openbao/logs/audit.log",
      "log_raw": "false",
      "hmac_accessor": "true"
    }
  }
}
```

> ⚠ **`log_raw = "false"` is mandatory in production.** With it `true`, secret values are written to the audit log in plaintext.
>
> ⚠ `hmac_accessor = "true"` means token accessors are HMAC'd too (safer, but you can't use them for emergency revocation from the log). Default is now `true` — good. Set it `false` only if you have a separate, secure revocation runbook.

Now store the unseal shares and root token somewhere safe (password manager, offline). Then, ideally, **revoke the root token** — Level 5 shows how.

2.5 Step 5 — KV v2 and a secret

```
# Enable the KV v2 engine at "secret"
bao secrets enable -path=secret kv-v2
bao secrets list
```

```
{
  "cubbyhole/": { "type": "cubbyhole", "description": "per-token private secret storage", "ac
  "identity/":  { "type": "identity",  ... },
  "secret/":    { "type": "kv", "options": { "version": "2" }, ... },
  "sys/":       { "type": "sys", ... }
}
```

Write a secret

```
bao kv put -mount=secret myapp \
  db_host=postgres \
  db_port=5432 \
  db_user=app \
  db_password='S3cr3t!' \
  llm_api_key='sk-live-xxxxx'
```

```
{
  "request_id": "...",
  "lease_id": "",
  "renewable": false,
  "lease_duration": 0,
  "data": {
    "created_time": "2026-10-20T12:00:00Z",
    "deletion_time": "",
    "destroyed": false,
```

```
    "version": 1
  },
  "wrap_info": null,
  "warnings": null,
  "auth": null
}
```

Read it back

```
bao kv get -mount=secret myapp
bao kv get -mount=secret -field=db_password myapp
bao kv metadata get -mount=secret myapp
```

The KV v2 lifecycle (know these before you need them)

```
bao kv put -mount=secret myapp db_password='new'      # v2 now
bao kv get -mount=secret -version=1 myapp              # read old version
bao kv delete -mount=secret myapp                      # soft delete (v3 marked deleted)
bao kv get -mount=secret myapp                         # -> error: data deleted
bao kv undelete -mount=secret myapp                    # bring it back
bao kv destroy -mount=secret myapp                     # permanently destroy current version
bao kv metadata delete -mount=secret myapp             # delete all versions + metadata
```

Sub-path (folder) organisation

```
bao kv put -mount=secret myapp/api llm_api_key='sk-live-xxxxx'
bao kv put -mount=secret myapp/db  db_password='S3cr3t!'

bao kv get -mount=secret myapp
# -> { "api": {...}, "db": {...} }   (folder listing)
```

> ※ **Design tip:** one secret per path (not one big blob) makes policies far more granular. `secret/data/myapp/api` and `secret/data/myapp/db` can have different policies.

2.6 Step 6 — Policies (write them like a reviewer would)

The ”reader” policy for the app

```
bao policy write app-readonly - <<'EOF'
# Read the app's secrets
path "secret/data/myapp" {
  capabilities = ["read"]
}

# Read version metadata (needed for `bao kv metadata get` / version listing)
path "secret/metadata/myapp" {
  capabilities = ["read", "list"]
}
EOF

bao policy read app-readonly
```

The ”writer” policy (for a deploy pipeline, not the app)

```
bao policy write app-writer - <<'EOF'
path "secret/data/myapp" {
  capabilities = ["create", "read", "update", "patch"]
}
path "secret/metadata/myapp" {
  capabilities = ["read", "list", "delete"]
}
EOF
```

※ **The `metadata` path for KV v2 needs `list`** to enumerate versions. Forgetting it causes a confusing ”permission denied” on `bao kv metadata get`.

The ”version” policy (does the secret exist, and what’s its latest version)

```
bao policy write app-version-reader - <<'EOF'
path "secret/metadata/myapp" {
  capabilities = ["read"]
}
EOF
```

Test the policy before you give it to anyone

```
bao token create -policy=app-readonly -ttl=10m -no-default-policy -field=token
```

Then, in a second shell with that token:

```
export BAO_TOKEN=<the token>
bao kv get -mount=secret myapp             # ✓ works
bao kv put -mount=secret myapp x=1         # ✗ permission denied
bao secrets list                     # ✗ permission denied
bao token capabilities <token> secret/data/myapp
```

[SEC] **`-no-default-policy`**: without it, every token also gets `default` (lookup-self + renew-self). Usually you want it; for a strict test, drop it.

The ”audit auditor” policy

```
bao policy write audit-reader - <<'EOF'
path "sys/audit" {
  capabilities = ["read", "list"]
}
path "auth/token/accessors" {
  capabilities = ["read", "list"]
}
EOF
```

Check what a token can do — the debugging superpower

```
bao token capabilities <token> secret/data/myapp
bao sys capabilities <other-token> secret/data/myapp
```

2.7 Step 7 — A human login path (userpass)

```
bao auth enable userpass

bao write auth/userpass/users/alice \
  password='Correct-Horse-Battery-Staple' \
  token_policies=app-readonly \
  token_ttl=1h \
  token_max_ttl=8h
```

Log in:

```
bao login -method=userpass username=alice
```

This writes the token to `~/.vault-token` (2.6.0+ lets you override with `BAO_TOKEN_PATH`).

```
bao token lookup
bao kv get -mount=secret myapp
```

Logout:

```
# There is NO `bao logout` command in OpenBao 2.7.1. To "log out", delete
# the token file that the default token helper wrote:
rm -f ~/.vault-token          # or $BAO_TOKEN_PATH, if you set it
```

> ※ userpass is **not** for production humans — you have Keycloak. It is here for (a) learning, (b) break-glass. Level 7 replaces it with OIDC.

2.8 Step 8 — A machine login path (AppRole)

```
bao auth enable approle

bao write auth/approle/role/myapp \
  secret_id_ttl=10m \
  secret_id_num_uses=1 \
  token_num_uses=0 \
  token_ttl=20m \
  token_max_ttl=1h \
  secret_id_bound_cidrs="10.0.0.0/8"

ROLE_ID=$(bao read -field=role_id auth/approle/role/myapp/role-id)
SECRET_ID=$(bao write -f -field=secret_id auth/approle/role/myapp/secret-id)

echo "role_id:    $ROLE_ID"
echo "secret_id:  $SECRET_ID"     # inject this at deploy time, never commit it
```

Login:

```
bao write auth/approle/login role_id="$ROLE_ID" secret_id="$SECRET_ID"
```

```
{
  "auth": {
    "client_token": "hvs.CAES...",
    "accessor": "...",
    "policies": ["app-readonly", "default"],
    "lease_duration": 1200,
    "renewable": true
```

```
  }
}
```

Test that a used secret_id is dead

```
bao write auth/approle/login role_id="$ROLE_ID" secret_id="$SECRET_ID"
# -> Error: invalid secret id   ✓ (num_uses=1 worked)
```

> [SHIELD] **2.7.1 security fix (GHSA-7m59-mp95-w6ph):** AppRole secret IDs now cannot be used for authentication after expiry, even before the `tidy` job runs. This was a real bug — **keep OpenBao patched**. That is the whole reason to run `2.7.1` and not `2.6.x`.

2.9 Step 9 — Wire up a FastAPI app

Directory:

```
~/openbao-lab/app/
├── Dockerfile
├── requirements.txt
└── main.py
```

`requirements.txt`:

```
fastapi==0.115.6
uvicorn[standard]==0.34.0
hvac==2.3.0
pydantic==2.10.4
```

`main.py`:

```
"""
Minimal, production-shaped FastAPI + OpenBao integration.

Key ideas:
  * Secrets are fetched at startup, not at import time.
  * The Vault token is renewed in the background.
  * Secrets are never logged.
  * Fail fast if OpenBao is unreachable at boot (fail closed).
"""
from __future__ import annotations

import asyncio
import contextlib
import logging
import os
from typing import Any

import hvac
from fastapi import FastAPI, HTTPException

log = logging.getLogger("app")
logging.basicConfig(level=logging.INFO)

BAO_ADDR = os.environ["BAO_ADDR"]
BAO_CACERT = os.environ.get("BAO_CACERT")
ROLE_ID = os.environ["APPROLE_ROLE_ID"]
```

```
SECRET_ID = os.environ["APPROLE_SECRET_ID"]
SECRET_PATH = os.environ.get("SECRET_PATH", "secret/data/myapp")

client = hvac.Client(url=BAO_ADDR, verify=BAO_CACERT, timeout=30)
_cache: dict[str, Any] = {}


def login() -> str:
    """Authenticate with AppRole and return the client token."""
    resp = client.auth.approle.login(role_id=ROLE_ID, secret_id=SECRET_ID)
    token = resp["auth"]["client_token"]
    client.token = token
    return token


def renew_loop() -> None:
    """Renew the Vault token in the background so the app never loses access."""
    while True:
        try:
            client.auth.token.renew_self(token=client.token, increment="30m")
            log.info("renewed OpenBao token")
        except Exception as exc:  # noqa: BLE001
            log.error("token renewal failed: %s", exc)
            with contextlib.suppress(Exception):
                login()
        import time
        time.sleep(300)


def get_secret() -> dict[str, Any]:
    """Read the secret from KV v2 with a tiny in-process cache."""
    key = "value"
    now = asyncio.get_event_loop().time() if False else 0  # simple TTL below
    cached = _cache.get(key)
    if cached and cached["expires"] > now:
        return cached["data"]
    resp = client.secrets.kv.v2.read_secret_version(
        path=SECRET_PATH.replace("secret/data/", ""),
        mount_point="secret",
    )
    data = resp["data"]["data"]
    _cache[key] = {"data": data, "expires": 0}  # refresh each call in this example
    return data


app = FastAPI(title="AI Harness", version="1.0.0")


@app.on_event("startup")
def startup() -> None:
    login()
    asyncio.create_task(asyncio.to_thread(renew_loop))
    log.info("OpenBao authenticated and secret cache warm")


@app.get("/health")
```

```python
def health() -> dict[str, str]:
    return {"status": "ok"}


@app.get("/config-check")
def config_check() -> dict[str, str]:
    """Prove the app can read its secret — WITHOUT leaking it."""
    secret = get_secret()
    return {
        "db_host": secret.get("db_host", "missing"),
        "llm_key_present": bool(secret.get("llm_api_key")),
    }


@app.get("/llm-key")
def llm_key() -> str:
    """Example of a *bad* endpoint. Do not do this in production."""
    return get_secret()["llm_api_key"]
```

Dockerfile:

```dockerfile
FROM python:3.11-slim

WORKDIR /app
COPY requirements.txt .
RUN pip install --no-cache-dir -r requirements.txt
COPY main.py .

RUN useradd -m -u 10001 appuser
USER appuser

EXPOSE 8000
CMD ["uvicorn", "main:app", "--host", "0.0.0.0", "--port", "8000"]
```

Run it:

```
docker build -t harness-app ./app

docker run -d --name harness-app \
  --network openbao-lab_default \
  -e BAO_ADDR=https://openbao:8200 \
  -e BAO_CACERT=/certs/ca.crt \
  -e APPROLE_ROLE_ID="$ROLE_ID" \
  -e APPROLE_SECRET_ID="$SECRET_ID" \
  -v ~/openbao-lab/certs:/certs:ro \
  harness-app
```

```
curl -s localhost:8000/config-check | jq
```

```
{ "db_host": "postgres", "llm_key_present": true }
```

※ **Notice what the app does not do:** it never writes the secret to a file, never puts it in a command-line argument, never prints it. It holds it in memory, briefly, and uses it.

⚠ **A note on the "never log the secret" rule:** the `/llm-key` endpoint above is deliberately labelled bad. In production, you would expose actions that use the key, never the key.

2.10 Step 10 — Read the audit log

This is the most important 5 minutes in the course.

```
docker compose exec openbao sh -c 'tail -f /openbao/logs/audit.log' | jq .
```

You'll see a stream of JSON objects. Let me annotate one:

```
{
  "type": "response",
  "auth": {
    "client_token": "hmac-sha256:9c2a...",   // ← HMAC'd, not plaintext
    "accessor": "hmac-sha256:1f3b...",
    "display_name": "token",
    "policies": ["app-readonly", "default"],
    "token_ttl": 3600,
    "token_num_uses": 0
  },
  "request": {
    "id": "0e4b...",
    "operation": "read",
    "client_token": "hmac-sha256:9c2a...",
    "client_token_accessor": "hmac-sha256:1f3b...",
    "namespace_path": "root/",
    "path": "secret/data/myapp",
    "remote_address": "172.18.0.5",
    "client_capabilities": ["read"],
    "request_priority": 0
  },
  "error": "",
  "response": {
    "data": {
      "data": {
        "db_password": "hmac-sha256:7a4e...",   // ← secret value, HMAC'd
        "llm_api_key": "hmac-sha256:2b9f..."
      }
    }
  }
}
```

What to look for in an incident

```
# All operations on a specific path
jq 'select(.request.path=="secret/data/myapp") | {t:.time, op:.request.operation, ip:.request

# All failed requests
jq 'select(.error != "") | {t:.time, path:.request.path, err:.error}' audit.log

# Everything a given accessor did
jq 'select(.auth.accessor=="hmac-sha256:1f3b...")' audit.log

# How many reads happened, by path
jq -r 'select(.type=="response") | .request.path' audit.log | sort | uniq -c | sort -rn
```

> ※ **The HMAC trick:** because the same plaintext always produces the same HMAC (for a given audit device), you can prove "this is the same secret value" across logs and across time, without ever seeing the value. That's how you detect value reuse after a supposed

rotation.

⚠ Because the HMAC key is per-device, **HMACs are not comparable across different audit devices.**

2.11 Now break it on purpose

The best way to internalise policies is to violate them.

```
# 1. A token that should only read, tries to write
BAO_TOKEN=<app-readonly-token> bao kv put -mount=secret myapp hacked=yes
# → Error making API request. ... Code: 403. Errors: * 1 error occurred: * permission denied

# 2. Try to read a different app's secret
BAO_TOKEN=<app-readonly-token> bao kv get -mount=secret otherapp
# → 403 permission denied

# 3. Try to list secrets
BAO_TOKEN=<app-readonly-token> bao secrets list
# → 403

# 4. Try to escalate
BAO_TOKEN=<app-readonly-token> bao token create -policy=root
# → 403

# 5. Try to read the audit log
BAO_TOKEN=<app-readonly-token> bao audit list
# → 403
```

Now check the audit log — did the **denied** attempts get recorded?

```
jq 'select(.error != "") | {path:.request.path, op:.request.operation, err:.error}' audit.log
```

Yes. Denied requests are audited. That's how you detect credential stuffing and permission-probing.

2.12 Clean up and reset (repeatable practice)

```
# Revoke the root token (Level 5 does this properly — do it by hand once here)
bao token revoke <root-token>

# Or nuke the whole thing
docker compose down -v
rm -rf data/*
docker compose up -d
```

2.13 Level 2 checkpoint

1. Why must you enable the audit device before storing secrets?
2. What does `bao operator init -key-shares=5 -key-threshold=3` mean, and can two different people unseal with shares 1 and 2?
3. Which four paths does a KV v2 policy need per secret, and which one do people forget?
4. What happens to a secret_id after `secret_id_num_uses=1` is used once?

5. Where does the AppRole `role_id` belong, and where does the `secret_id` belong? Why the difference?
6. How do you check what a token is allowed to do before giving it to someone?
7. How do you find "who read this secret" in the audit log?

Next → Level 3: POLICIES-TOKENS-IDENTITY.md — the authorization deep dive: advanced policy syntax, templating, identity groups, token strategy at scale.

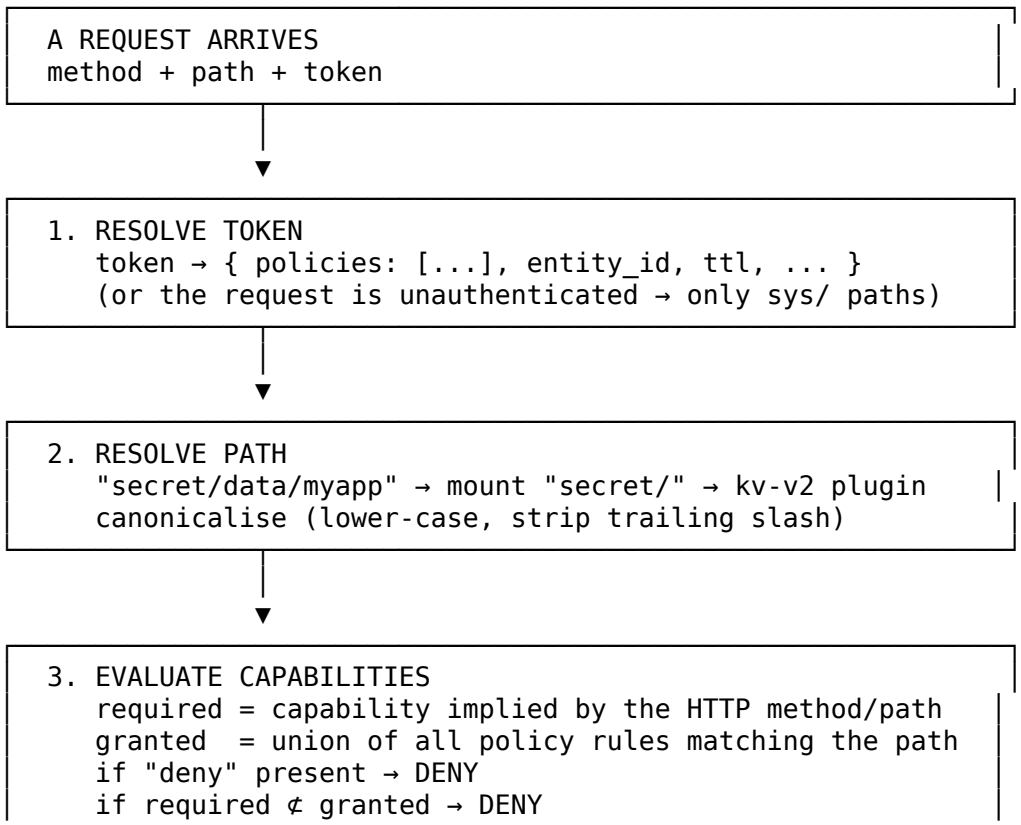
2.14 Scratchpad

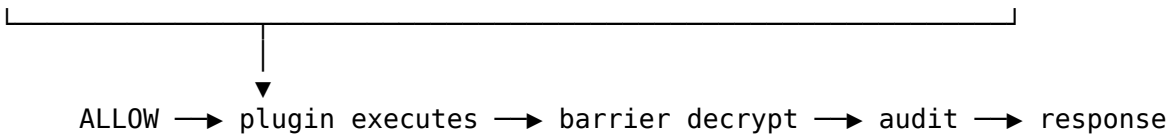
```
BAO_ADDR:              https://127.0.0.1:8200
BAO_CACERT:            ~/openbao-lab/certs/ca.crt
Mounts:                secret/ (kv-v2)
Policies:             app-readonly, app-writer, app-version-reader, audit-reader
Auth methods:         userpass/ (human), approle/ (machine)
AppRole role_id:     <from bao read -field=role_id auth/approle/role/myapp/role-id>
Audit log:             /openbao/logs/audit.log  (inside container)
```

Level 3 — Policies, Tokens & Identity Deep Dive

> **Goal:** you can design an authorisation model for a real organisation — dozens of people, dozens of services, multiple teams — and you understand the exact mechanics of how a token ends up allowed or denied.

3.1 The authorisation model, in one picture





Three consequences you should internalise:

1. **Everything is a path**, including `sys/`. There is no ”special case” for admin endpoints.
2. **The union of policies is the grant.** Two policies both granting `read` on the same path = read. There’s no ”highest priority wins” — only `deny` is special.
3. **Denial is decided before the plugin runs.** A denied request never touches the secrets engine, and (crucially) is still audited.

3.2 Policy syntax — the complete reference

The shape

```
path "<glob>" {
  capabilities   = ["read", "list"]
  allowed_parameters = { "key" = ["value"] }
  denied_parameters  = { "key" = [] }
  min_wrapping_ttl  = "60s"
  max_wrapping_ttl  = "15m"
  control_group = { ... }          # 2.7.0 feature
}
```

Path patterns — every form that matters

```
# 1. Exact path
path "secret/data/myapp" { capabilities = ["read"] }

# 2. Prefix glob: everything below
path "secret/data/myapp/*" { capabilities = ["read"] }

# 3. Wildcard within one segment
path "secret/data/app-*" { capabilities = ["read"] }

# 4. "+" = any characters within a SINGLE segment
path "secret/data/+/config" { capabilities = ["read"] }
#    matches secret/data/foo/config
#    matches secret/data/bar/config
#    does NOT match secret/data/foo/bar/config

# 5. Combining
path "secret/data/+/+/secrets" { capabilities = ["read"] }

# 6. Explicit deny always wins over a broader grant
path "secret/data/myapp/root-keys" { capabilities = ["deny"] }
```

> ※ **The ”one path per secret” rule.** If you have path `"secret/data/*"` with read, then path `"secret/data/myapp/root-keys"` with deny — the deny wins for that exact path. This lets you write a broad base policy and carve out exceptions, which is far more maintainable than 200 narrow policies.

Capabilities — precise meanings

Capability	Use it for
`create`	First write to a path; also KV v2 metadata create
`read`	Read a value; also KV v2 `data/` reads
`update`	Overwrite; also `metadata/`, `undelete/`, `destroy/`
`patch`	Merge-patch a JSON object
`delete`	Delete an object; KV v2 `metadata/` deletion
`list`	List direct children (needs `list` on the parent path)
`scan`	Recursive list
`sudo`	Perform the operation as the root user. Extremely powerful — e.g. `sudo` on `sys/raw` lets a token re
`deny`	Hard deny, overrides wildcard grants

`sudo` deserves a warning. A policy with `sudo` on `sys/raw` can decrypt the whole storage barrier. Treat `sudo` grants like root.

`list` needs `list` on the parent

Listing `secret/data/myapp` requires `list` on `secret/data/myapp` (and `read` on `secret/metadata/myapp` for KV v2 metadata). A common gotcha:

```
# ✗ This does NOT let you list
path "secret/data/myapp" { capabilities = ["read"] }

# ✓ This does
path "secret/data/myapp" {
  capabilities = ["read", "list"]
}
path "secret/metadata/myapp" {
  capabilities = ["read", "list"]
}
```

Parameter control

```
path "secret/data/app-config" {
  capabilities = ["create", "update"]
  allowed_parameters = {
    "environment" = ["dev", "staging", "prod"]
    "replicas"    = []
  }
  denied_parameters = {
    "debug" = []
  }
}
```

※ This is a genuinely useful, under-used feature: it lets a deployment pipeline set environment and replica count, but **not** flip on debug mode.

Response wrapping TTL control

```
path "secret/data/super-secret" {
  capabilities = ["read"]
  min_wrapping_ttl = "60s"
```

```
  max_wrapping_ttl = "5m"
}
```

Prevents an operator from wrapping a response in a token that lives for a year.

Policy templates — the identity-aware bit

```
# Each user gets a folder named after their own username
path "secret/data/users/{{identity.entity.aliases.auth_userpass_XXXX.metadata.username}}/*" {
  capabilities = ["read", "list"]
}
```

The template is evaluated **at request time** against the token's identity.

Available template roots:

Root	What it is
`identity.entity.id`	The entity UUID
`identity.entity.name`	Entity name
`identity.entity.aliases.<mount_accessor>.metadata.<key>`	Alias metadata
`identity.entity.aliases.<mount_accessor>.name`	Alias name (e.g. username)
`identity.entity.groups.<id>.name`	Group names
`identity.entity.policies`	Policies on the entity

> [SEC] **2.6.0 security change (GHSA-59w7-v8rr-pr4p):** template values containing `+`, `*`, `/` or `,` are **rejected by default**. This prevents an attacker who controls a metadata field (e.g. a username) from escaping the intended path. The escape hatches `allow_wildcards_in_identity_templates`, `allow_slashes_in_identity_templates`, `allow_commas_in_identity_templates` and `allow_globs_in_identity_templates` restore the old behaviour.
>
> **Keep them off.** If a template fails, the fix is sanitising the source claim, not enabling a bypass.

A realistic, production-shaped policy set

```
# ─────────────────────────────────────────────────────────────
# app-readonly.hcl   — the application itself
# ─────────────────────────────────────────────────────────────
path "secret/data/myapp" {
  capabilities = ["read"]
}
path "secret/metadata/myapp" {
  capabilities = ["read", "list"]
}

# ─────────────────────────────────────────────────────────────
# deploy-writer.hcl   — the CI pipeline
# ─────────────────────────────────────────────────────────────
path "secret/data/myapp" {
  capabilities = ["create", "read", "update", "patch"]
}
path "secret/metadata/myapp" {
  capabilities = ["read", "list", "delete"]
}
```

```
path "secret/undelete/myapp" {
  capabilities = ["update"]
}
path "secret/destroy/myapp" {
  capabilities = ["update"]
}

# ─────────────────────────────────────────────────────────────────
# breakglass-admin.hcl — humans, only via a protected auth path
# ─────────────────────────────────────────────────────────────────
path "secret/data/*" {
  capabilities = ["create", "read", "update", "patch", "delete", "list", "scan"]
}
path "sys/audit" {
  capabilities = ["read", "list"]
}
path "sys/mounts" {
  capabilities = ["read", "list"]
}
path "sys/auth" {
  capabilities = ["read", "list"]
}
path "sys/policies/acl" {
  capabilities = ["list"]
}
path "auth/token/lookup-self" {
  capabilities = ["read"]
}
path "auth/token/renew-self" {
  capabilities = ["update"]
}
```

> ⚠ Notice `breakglass-admin` has **no** sudo, no `sys/raw`, no `auth/token/create`. It can manage secrets but **cannot mint new tokens or escalate**. That's a deliberately constrained admin.

Writing and testing policies safely

```
# Write (with a heredoc so the policy is reviewable in git)
bao policy write app-readonly - <<'EOF'
path "secret/data/myapp" { capabilities = ["read"] }
EOF

# Format / lint — NOTE: `policy fmt` operates on a LOCAL FILE and
# overwrites it in place. It does NOT take a policy name, and there is
# no `-check` flag.
bao policy fmt ./policies/app-readonly.hcl
#   (then re-upload: bao policy write app-readonly - < ./policies/app-readonly.hcl)

# Read back
bao policy read app-readonly

# List
bao policy list
```

```
# IMPORTANT: policy updates take effect IMMEDIATELY for all attached tokens
bao policy delete app-readonly
```

Dry-run against a real token:

```
bao token capabilities <token> secret/data/myapp
# Capabilities: [create read update]
```

`bao write -output-policy` shows you what a policy would need to be:

```
bao kv put -output-policy -mount=secret myapp value=x
# path "secret/data/myapp" { capabilities = ["create", "update"] }
```

3.3 EGP and RGP (a note of caution)

OpenBao's policy surface has three families:

Family	Path	Purpose
ACL	`sys/policies/acl/:name`	Path → capabilities. The 99% case.
EGP — Endpoint Governing Policy	`sys/policies/egp/:name`	Governs who may call which endpoint with wh
RGP — Role Governing Policy	`sys/policies/rgp/:name`	Time-bound, just-in-time access to a role.

Honest guidance: learn ACL thoroughly first — it covers essentially everything you will build for an AI harness. EGP/RGP are for very specific "this API must only ever be called with these exact arguments" and "grant production admin for 30 minutes, then it evaporates" requirements. Read them directly from the API reference before using them, and **verify against your specific build** with `bao sys/policies/ -detailed` style calls. I am not going to invent exact JSON schemas for you and have you discover they're wrong at 2am.

For the "time-bound just-in-time access" need, the **2.7.0 Control Groups** feature (Level 8) is the modern, documented way to require a second pair of eyes.

3.4 Tokens — the complete strategy

Creating tokens properly

```
bao token create \
  -policy=app-readonly \
  -ttl=1h \
  -renewable=true \
  -explicit-max-ttl=8h \
  -display-name=myapp-prod \
  -meta=env=prod \
  -meta=owner=platform-team \
  -orphan=true
```

Useful flags:

Flag	Why
`-orphan=true`	Root of its own tree; revoking the parent doesn't kill it (and vice versa)
`-display-name`	Human-readable label; shows up in the audit log

Flag	Why
`-meta k=v`	Custom metadata; also shows in the audit log. **Great for attribution.**
`-explicit-max-ttl`	Hard ceiling regardless of mount/system max
`-period`	Periodic token (TTL resets on renew)
`-batch`	Batch token (no storage, non-renewable)
`-no-default-policy`	Drop the implicit `default` policy
`-num-uses=N`	Token can be used N times then dies
`-cidr="10.0.0.0/8"`	Bind the token to source IP ranges
`-explicit-orphan`	(API)

The token lifecycle in practice

```
login ──▶ token (ttl=1h, renewable)
          │
          ├── renew ──▶ extends TTL (bounded by max TTL)
          │
          ├── create child ──▶ child token (inherits parent's tree)
          │
          └── revoke ──▶ token + all children + all leases die
```

```
bao token lookup                      # current token
bao token lookup <token>              # specific token
bao token renew                       # renew self
bao token renew -increment=2h         # ask for 2h from NOW
bao token renew -orphan               # renew as an orphan (no parent)
bao token revoke <token>
bao token revoke -accessor <accessor>     # revoke via accessor
bao token revoke -accessor <accessor> -revoke-lease   # also drop its leases
bao token revoke -orphan <accessor>      # revoke only that node of the tree
bao token capabilities <token> secret/data/myapp
```

Accessors in anger

```
# 1. Create a token, keep only the accessor
ACC=$(bao token create -policy=app-readonly -ttl=1h -orphan=true -format=json | jq -r .auth.a
TOKEN=$(bao token create ...)   # give the token to the workload

# 2. Later, in an emergency, kill it — without ever having stored the token
bao token revoke -accessor "$ACC"
```

> ※ This is the pattern that makes secret distribution safe. The workload gets the token; your control plane stores the accessor. A leaked control plane can't authenticate. A leaked token can be revoked instantly.

Batch tokens for scale

```
bao token create -policy=app-readonly -ttl=5m -batch=true
```

- Encrypted blob, **no storage write** → extremely cheap.
- Cannot renew, cannot have children, cannot be root, no accessor, no cubbyhole.
- Leases created by a batch token are bounded by the batch token's TTL.

Use for: CI jobs, high-frequency read-only automation, anything issuing thousands of tokens an hour.

Don't use for: long-running services (you can't renew), or anything needing revocation precision.

Periodic tokens for long-lived services

```
bao token create -policy=app -period=24h -orphan=true
```

The TTL resets to 24h on each renewal. If the service stops renewing, the token dies within 24h. This is how you give a service a ”permanent” credential that still self-destructs.

> ⚠ Use a **short** period (1h, not 30d) and renew frequently. The period is your detection window.

CIDR binding

```
bao token create -policy=app-readonly -ttl=1h -cidr="10.0.0.0/8"
```

If the token is used from outside the CIDR → denied. Great for a service that should only ever talk to OpenBao from its own subnet. (Root tokens with a TTL are also affected; non-expiring root tokens are not.)

Token storage — where does the CLI put it?

Mechanism	Path
Default token file	`~/.vault-token` (override with `BAO_TOKEN_PATH`)
Token helper	`bao config -h` → set `BAO_TOKEN_HELPER` to a helper binary
Env var	`BAO_TOKEN` (or legacy `VAULT_TOKEN`)

```
# Inspect and change the helper
bao config -h
# or
bao config -output-policy   # see effective CLI config
```

> [KEY] **Production rule:** never use `BAO_TOKEN` in a long-lived shell or in CI logs. Use the token helper (e.g. `bao` + `pass`/`vault` helper) or `bao login -no-store` and re-login per session.

3.5 Identity — the graph that scales authorisation

Entities, aliases, groups

```
ENTITY (uuid)  ── represents a person or a machine
   │
   ├── ALIAS  ── "how this entity logs in"
   │             e.g. auth method = userpass, name = "alice"
   │             e.g. auth method = oidc,    name = "alice@corp.com"
   │
   ├── GROUP   ── a named bucket of entities, carries policies
   │             e.g. "platform-engineers" → [app-readonly, app-deploy]
   │
   ├── METADATA ── arbitrary key/values
   │             e.g. full_name = "Alice Smith", team = "platform"
   │
   └── POLICIES ── extra policies granted on top of the token's
```

Why this matters

Without identity, you hardcode `user → policy` in the auth method. With 500 people across 3 teams and 2 IdPs, that's a nightmare of config. With identity:

- the auth method maps a claim to a group name,
- OpenBao's identity store maps group → policies,
- and a person in both Keycloak and (say) userpass resolves to **one entity**.

Building it

```
# 1. Create a group with policies
bao write identity/group name=platform-engineers \
    policies=app-readonly,app-deploy,audit-reader

# 2. Create an entity (or let OpenBao create it on first login)
bao write identity/entity \
    name="alice" \
    metadata=full_name="Alice Smith" \
    metadata=team="platform" \
    policies=app-readonly

# 3. Get the entity id
ENTITY_ID=$(bao read -field=id identity/entity/name/alice)

# 4. Add the entity to the group
bao write identity/entity/id/$ENTITY_ID groups=platform-engineers
```

Aliases — linking an IdP identity to an entity

An **alias** binds "a login in a specific auth method" to an entity. The alias is unique by `(mount_accessor, alias_name)`.

```
# Get the accessor of the userpass mount
ACCESSOR=$(bao auth accessor list -format=json | \
  jq -r '.[] | select(.path=="auth/userpass/") | .accessor')

bao write identity/entity-alias \
    name="alice" \
    canonical_id="$ENTITY_ID" \
    mount_accessor="$ACCESSOR"
```

※ The **alias name** each auth method reports:

Auth method	Alias name
AppRole	Role ID
JWT/OIDC	configurable via `user_claim` (no default)
Kubernetes	service account UID (default) or name
userpass / LDAP / RADIUS / Kerberos	username
TLS cert	Subject CommonName
Token	`entity_alias` if provided

Entity policies are evaluated at request time

This is the important shift. A token's policy names are fixed at creation, but the **contents** of policies, and any policies granted via the entity, are resolved **on every request**.

```
Token has policies: ["default"]
Token has entity_id: <alice's entity>
Entity has policies: ["app-readonly"]
Group "platform" has policies: ["app-deploy"]

Effective capabilities = default ∪ app-readonly ∪ app-deploy
```

So you can **change what a person can do without re-issuing their token** — just edit the entity or the group. Revoke access instantly.

> [ALERT] **Warning from the docs, and it is serious:** ”Be careful in granting permissions to non-readonly identity endpoints. If a user can modify an entity, they can grant it additional privileges through policies. If a user can modify an alias they can login with, they can bind it to an entity with higher privileges. If a user can modify group membership, they can add their entity to a group with higher privileges.”
>
> Practical rule: **never let an application write to `identity/entity-alias`, `identity/entity`, or `identity/group/members` with a capability beyond what it strictly needs.** A service that can write group membership is an escalation engine.

Identity in namespaces

Entities and groups are **namespace-scoped**. A child namespace can inherit policies/identities from its parent. Cross-namespace identity references are **disabled by default** — and `unsafe_cross_namespace_identity` should stay disabled (it ”allows for unconfined permission sharing”).

3.6 Designing the authorisation model for a real org

Here is a concrete, defensible design for a mid-size enterprise.

Layer 0 — the root namespace holds only infrastructure

```
root/
├── auth/oidc/           (Keycloak — humans)
├── auth/approle/        (machines)
├── auth/kubernetes/     (K8s workloads)
├── identity/            (entities, groups, policies for humans)
├── sys/                 (cluster config)
└── secret/              (shared platform secrets: db, redis, kafka...)
```

Layer 1 — namespaces per team

```
bao namespace create platform
bao namespace create payments
bao namespace create ai-harness
```

Each team gets its own namespace with its own admin, its own policies, its own secrets. Root-namespace admins are not automatically admins inside a namespace (they need a policy that grants it).

Layer 2 — three policy tiers

Tier	Who	TTL	Example policies
Read	Apps	1h	`app-readonly`
Operate	Humans/SRE	4h	`app-readonly` + `app-writer` + `audit-reader`
Break-glass	On-call	30m	`breakglass-admin` (no sudo)

Layer 3 — bind tiers to groups

```
bao write identity/group name=app-services   policies=app-readonly
bao write identity/group name=sre-team        policies=app-readonly,app-writer,audit-reader
bao write identity/group name=breakglass      policies=breakglass-admin
```

Layer 4 — bind groups to IdP claims (Keycloak, Level 7)

```
bao write auth/oidc/role/sre \
    user_claim=preferred_username \
    bound_claims=groups \
    claim_mappings="{\"sre-team\":{\"groups\":[\"sre-team\"]}}"
```

The review checklist for any new policy

```
□ Is the path as narrow as possible?
□ Does it grant the minimum capabilities needed?
□ Does it need list/scan? (they leak the shape of your data)
□ Does it need sudo? (almost never)
□ Does it need create/update on a shared path? (someone else can overwrite you)
□ Is there a `deny` carve-out for the sensitive sub-paths?
□ Would this policy still make sense if the app were fully compromised?
□ Is it scoped to one namespace?
```

> ※ **The "fully compromised" test** is the real bar. If the app is popped, what can the attacker reach? With `app-readonly` scoped to `secret/data/myapp`, they get one app's secrets. With path `"secret/data/*"`, they get everything. Design for the worst case, because the worst case is the case.

3.7 Auditing who did what

```
# 1. List all active token accessors
bao auth accessor list

# 2. Look up one
bao token lookup -accessor <accessor>

# 3. Revoke one
bao token revoke -accessor <accessor>
```

In the audit log, the fields that matter:

Field	Use
`request.path`	What was touched
`request.operation`	read / update / delete / list
`request.remote_address`	Where from

Field	Use
`request.client_token_accessor`	Which token (HMAC'd)
`auth.display_name`	Token's display name
`auth.policies`	Effective policies at the time
`auth.entity_id`	Which identity
`time`	When
`error`	Empty = success

[DEBUG] **In 2.7.0, `X-Vault-Index` is returned on write operations**, giving you an opaque index you can use to force read-after-write consistency (see Level 6, "Consistency"). Also note: 2.7.0 fixed a bug where malformed request data could leak into audit logs in plaintext (GHSA-8xxq-mq9m-xmhw) — another reason to run 2.7.1.

3.8 Level 3 checkpoint

1. Why is the effective policy a union rather than a priority system? What is the one exception?
2. You have `path "secret/data/*" { capabilities=["read"] }` and `path "secret/data/x" { capabilities=["deny"] }`. Can you read `secret/data/x`?
3. What does `sudo` on `sys/raw` let a token do?
4. When do you want `-orphan=true`?
5. Why is the accessor safer to store than the token?
6. What's the difference between a batch token and a periodic token?
7. Your team has 300 engineers. Why would you use identity groups instead of `token_policies` in the auth method?
8. Why is write access to `identity/group/members` dangerous?
9. What happens to an already-issued token's effective permissions when you add a policy to a group its entity is in?

Next → **Level 4: SECRET-ENGINES.md** — KV v2 in depth, Transit, PKI, dynamic database credentials, TOTP, SSH, cubbyhole, wrapping.

3.9 Scratchpad

```
Policy verbs:      create read update patch delete list scan sudo deny
Templating roots:  identity.entity.{id,name,aliases,groups,metadata}
Accessors:         safe to store/log; can lookup+renew+revoke, cannot auth
Batch token:       b.  no storage, no renew, no revoke, no children
Periodic token:    period=1h → TTL resets on renew; dies if you stop
Entity policies:   evaluated at REQUEST time (not token creation time)
DANGER:            identity/entity*, entity-alias, group/members = escalation
```

Level 4 — The Secrets Engines

Goal: you can pick the right engine for every job, configure it properly, and use it in a way that survives contact with a real production system.

4.0 Choosing the right engine

```
"I need to store an API key"                    →  KV v2
```

```
"I need to encrypt data my app stores in S3"     →  Transit
"I need internal TLS certs for my services"      →  PKI
"I need a Postgres user that expires"            →  Database
"I need step-up MFA codes"                       →  TOTP
"I need to replace authorized_keys"              →  SSH
"I need to pass a secret through a workflow"     →  Cubbyhole
"I need a credential but must not see it"        →  Response Wrapping
"I need to encrypt it myself and OpenBao holds
 the key in an HSM"                              →  Transit + External Keys (2.7)
```

4.1 KV v2 — the workhorse

Enable and configure

```
bao secrets enable -path=secret kv-v2

# Tune
bao secrets tune -default-lease-ttl=24h -max-lease-ttl=168h secret
```

> ※ KV v2 doesn't use leases, but `max-lease-ttl` **does** bound how long a deleted-but-not-destroyed version can be undeleted, and it bounds the cas window. Set it deliberately.

The path anatomy

```
secret/data/myapp          ← the VALUE
secret/metadata/myapp      ← versions, created_time, deletion_time, custom_metadata
secret/delete/myapp        ← mark a version deleted
secret/undelete/myapp      ← restore a deleted version
secret/destroy/myapp       ← permanently destroy a version
secret/config              ← engine-wide config (max_versions, cas_required)
```

Versioning in practice

```
bao kv put -mount=secret myapp password=v1
bao kv put -mount=secret myapp password=v2
bao kv put -mount=secret myapp password=v3

bao kv get -mount=secret -version=1 myapp    # → v1
bao kv get -mount=secret myapp               # → v3 (latest)
bao kv get -mount=secret -version=2 myapp    # → v2

bao kv metadata get -mount=secret myapp
# → { "current_version": 3, "versions": { "1": {...}, "2": {...}, "3": {...} } }
```

Deletion semantics (this confuses everyone)

```
bao kv delete -mount=secret myapp          # soft delete: marks version deleted
bao kv get -mount=secret myapp             # → Error: data deleted
bao kv undelete -mount=secret myapp        # → version is back
bao kv destroy -mount=secret myapp         # → data permanently gone (current version)
bao kv metadata delete -mount=secret myapp # → ALL versions + metadata gone
```

Each of these operations is a **separate path** and therefore a **separate capability**. That is a feature: you can let a service read values, let a pipeline soft-delete, and forbid permanent destruction except for a break-glass role.

Check-and-Set (CAS) — optimistic concurrency

CAS prevents a lost update when two writers race.

```
# Read the current version number
CURRENT=$(bao kv get -mount=secret -field=metadata.version myapp)

# Write only if nobody else has written since
bao kv put -mount=secret myapp password=v4 cas=$CURRENT
# → if someone else wrote first, this FAILS

# Or require CAS on every write to the engine
bao write secret/config cas_required=true
```

> ※ CAS is essential for "the deploy pipeline updates the secret" workflows where a human might also be editing it.

`max_versions` and pruning

```
bao write secret/config max_versions=5
```

Old versions beyond the limit are pruned automatically. If you need long audit history of secret values, use a separate audit trail — don't keep hundreds of KV versions around.

Custom metadata

```
bao kv put -mount=secret myapp password=v1 \
  custom_metadata=owner=platform \
  custom_metadata=rotated_at=2026-10-20
```

The sub-key pattern (recommended)

```
bao kv put -mount=secret myapp/db     db_host=postgres db_port=5432 db_user=app db_password=.
bao kv put -mount=secret myapp/llm    provider=anthropic api_key=sk-ant-...
bao kv put -mount=secret myapp/jwt    signing_key=... issuer=...

bao kv get -mount=secret myapp        # → folder listing
bao kv get -mount=secret myapp/db     # → just the db bundle
```

This gives you **per-sub-path policies**, which means the LLM service can read `secret/data/myapp/llm` and cannot read `secret/data/myapp/db`.

KV v1 vs v2 — final word

Use **v2**. Always. v1 has no versioning, no CAS, no metadata separation, and you cannot retrofit it. If you inherit a v1 mount, migrate it:

```
# v1 → v2 migration is supported via the upgrade guide
# (read it, do not improvise — the mount type must change)
bao secrets tune -default-lease-ttl=0 secret    # v1 is non-versioned
```

⚠ Actually migrating v1→v2 requires the documented upgrade procedure (read /docs/guides/upgrade/), not a casual enable. Do it in a lab first.

4.2 Transit — encryption as a service

The principle: your application never holds the encryption key.

```
app:  plaintext ──▶ bao transit/encrypt ──▶ ciphertext (stored in S3/DB)
app:  ciphertext ──▶ bao transit/decrypt ──▶ plaintext
                          │
                   key never leaves OpenBao
```

Setup

```
bao secrets enable -path=transit transit

# Create a key
bao write transit/keys/app-data type=aes256-gcm96
bao write transit/keys/app-data \
    type=aes256-gcm96 \
    convergent_encryption=true \
    min_decryption_version=0 \
    deletion_allowed=false
```

Key type	Use
aes128-gcm96, aes256-gcm96	General purpose, authenticated encryption
chacha20-poly1305	Fast, no AES-NI requirement
ed25519	Signatures
ecdsa-p256 / ecdsa-p384	Signatures
rsa-2048 / rsa-4096	Signatures + encryption (wrap/unwrap)
mldsa-44/65/87	**2.7.0** — post-quantum signatures (NIST FIPS 204)

Encrypt / decrypt

```
# Encrypt (input is base64)
CIPHER=$(bao write -format=json transit/encrypt/app-data \
          plaintext=$(echo -n "sensitive" | base64) | jq -r .data.ciphertext)
# → vault:v1:<base64>

# Decrypt
bao write -format=json transit/decrypt/app-data ciphertext="$CIPHER" | jq -r .data.plaintext
# → (base64) → decode with `base64 -d`

# Deterministic (convergent) encryption — same input → same ciphertext
bao write -format=json transit/encrypt/app-data \
    plaintext=$(echo -n "sensitive" | base64) | jq -r .data.ciphertext
# → vault:v1:<same ciphertext>   (because convergent_encryption=true)
```

※ **Convergent encryption** is what lets you do a searchable encrypted field: you can encrypt a column and still index it, because equal plaintexts produce equal ciphertexts. Trade-off: it leaks equality (which is inherent to searchable encryption).

Key versioning and rotation

```
bao read transit/keys/app-data
# → { "latest_version": 3, "min_decryption_version": 0, ... }

bao write -format=json transit/keys/app-data/rotate   # → latest_version: 4
bao write transit/keys/app-data/min_decryption_version=2   # archive v0,v1
```

Key rotation is non-destructive: old ciphertext still decrypts because it's tagged `vault:v1:...` (v1 = key version 1).

```
# Re-encrypt everything with the latest key (batch rewrap)
bao write transit/rewrap/app-data
```

Datakey generation — the "envelope encryption" pattern

```
bao write -format=json transit/datakey/plaintext/app-data bits=256
```

```
{
  "plaintext": "AAAAB3NzaC1yc2EAAAADAQABAAABAQ...",   // the actual key
  "ciphertext": "vault:v1:..."                          // the key, encrypted
}
```

The app encrypts data with the plaintext datakey (kept only in memory), stores only the ciphertext, and stores the encrypted datakey alongside. To decrypt, it asks OpenBao to decrypt the datakey. **The key is never persisted in the clear**, and OpenBao never sees the data.

Other transit powers

```
bao write transit/hash sha256      # hash input
bao write transit/hmac/sha256      # keyed HMAC
bao write transit/sign/mykey       # sign
bao write transit/verify/mykey     # verify
bao write transit/random/32        # 32 random bytes
bao write transit/export/mykey     # export key material (needs strict policy!)
bao write transit/backup/mykey     # backup key (for disaster recovery)
bao write transit/restore/mykey    # restore
bao write transit/trim/mykey       # permanently trim key versions
```

> [SEC] **`transit/export` and `transit/backup` are the most dangerous paths in OpenBao.** Export gives you the raw key. Backup gives you key material that can restore the key on another cluster. Both should be on a separate, tightly controlled policy — ideally break-glass only, and always audited.

Transit for your AI harness

Your LangChain app might need to encrypt prompts, RAG documents, or cached embeddings before writing them to Postgres/Redis. Transit is the right tool:

```
import base64, hvac

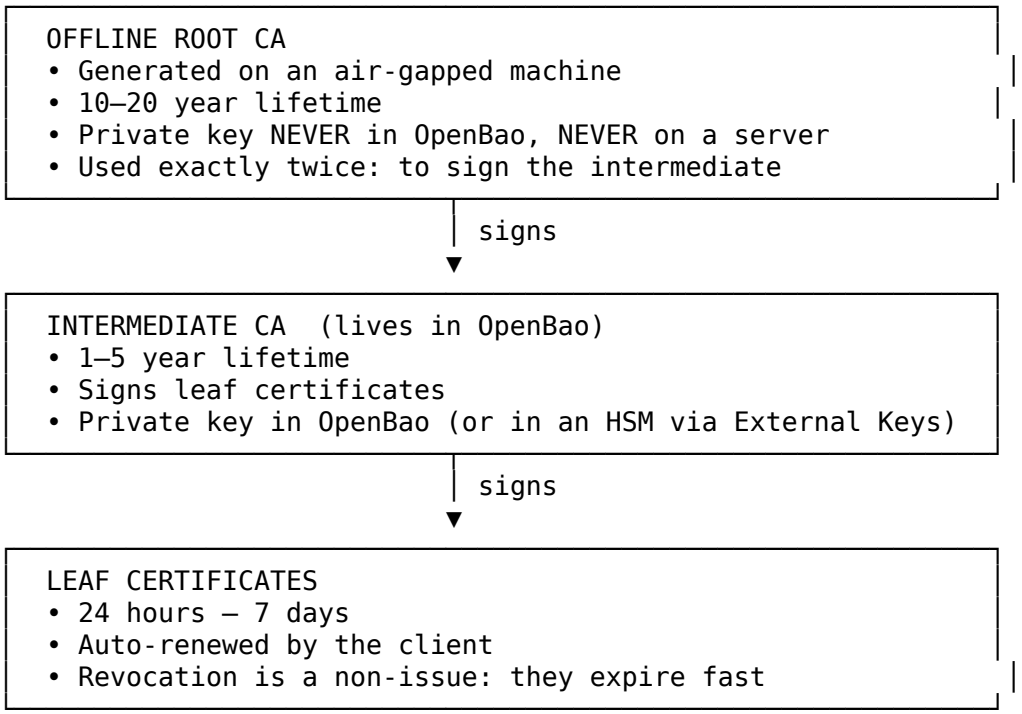
client = hvac.Client(url=os.environ["BAO_ADDR"], token=os.environ["BAO_TOKEN"])

def encrypt(plaintext: str) -> str:
    b64 = base64.b64encode(plaintext.encode()).decode()
    return client.secrets.transit.encrypt_data(name="rag-documents", plaintext=b64)["cipherte
```

```
def decrypt(ciphertext: str) -> str:
    b64 = client.secrets.transit.decrypt_data(name="rag-documents", ciphertext=ciphertext)["p
    return base64.b64decode(b64).decode()
```

4.3 PKI — your internal certificate authority

The three-tier model (the only sane production model)



Build it

⚠ **Three PKI gotchas that will bite you** (all verified against 2.7.1):

1. **`bao secrets tune -max-lease-ttl=... FIRST.`** The CA's lifetime is capped by the mount's max lease TTL (default 768h = 32 days). If you generate a 10-year root before tuning, OpenBao caps it and warns: "TTL '87600h0m0s' is longer than permitted maxTTL '768h0m0s', so maxTTL is being used". The certificate is then only 32 days old.
2. **`key_bits` is an INTEGER, not a string.** `key_bits=ecdsa-p521` fails with "error converting input for field 'key_bits': " cannot parse value as 'int'". Use `key_type=ec` + `key_bits=521`.
3. **`issuer_name` cannot contain spaces.** Use hyphens: `Corp-Offline-Root`. Or just omit `issuer_name` — the official quick-start does.

Step 1 — the root CA (run once, offline):

```
bao secrets enable -path=pki_root pki
bao secrets tune -max-lease-ttl=87600h pki_root     # 10 years — DO THIS FIRST

bao write pki_root/root/generate/internal \
    common_name="Corp-Offline-Root" \
    issuer_name="Corp-Offline-Root" \
```

```
    key_type=ec \
    key_bits=521 \
    ttl=87600h
```

※ The root becomes the **default issuer** automatically once generated, so you can immediately issue from it. You can check with:

```
bao read pki_root/issuer/default
```

Export the root cert (public, safe) and the private key (**very sensitive**):

```
# The generated cert + key come back in the write response:
bao write -format=json pki_root/root/generate/internal \
  | python3 -c "import json,sys;print(json.load(sys.stdin)['data']['certificate'])" > root-ca

# The CA chain endpoint serves raw PEM (not JSON) — use curl:
curl -s --cacert ca.crt https://openbao:8200/v1/pki_root/cert/ca > ca-chain.crt

# The private key (⚠ CONTAINS THE PRIVATE KEY)
bao read pki_root/root/export > root-ca-bundle.json
```

[SEC] The root private key must be moved **off** the OpenBao server immediately and stored offline. If it stays in OpenBao, your "offline root" is a lie.

Step 2 — the intermediate CA (in OpenBao):

```
bao secrets enable -path=pki_int pki
bao secrets tune -max-lease-ttl=43800h pki_int      # 5 years — DO THIS FIRST

# Generate the intermediate's CSR (writes csr + key_id)
bao write -format=json pki_int/intermediate/generate/internal \
    common_name="Corp-Intermediate" \
    issuer_name="Corp-Intermediate" \
    key_type=ec \
    key_bits=384 \
    ttl=43800h > csr.json

# Extract just the CSR to a file (the `csr=@file` syntax needs a real file)
python3 -c "import json;print(json.load(open('csr.json'))['data']['csr'])" > csr.pem

# Sign it with the root (this is the ONLY time the root key is used)
bao write -format=json pki_root/root/sign-intermediate \
    csr=@csr.pem \
    format=pem_bundle \
    ttl=43800h > signed.json

# Extract the signed certificate
python3 -c "import json;print(json.load(open('signed.json'))['data']['certificate'])" > signe
```

Step 3 — import the signed intermediate:

```
bao write pki_int/intermediate/set-signed \
    certificate=@signed-intermediate.crt
```

Step 4 — a role for your services:

```
bao write pki_int/roles/myapp \
    allowed_domains="myapp.internal" \
    allowed_subdomains="true" \
    allow_bare_domains="true" \
```

```
    allow_localhost="true" \
    max_ttl="72h" \
    key_type="ec" \
    key_bits=256 \
    server_flag="true" \
    client_flag="true" \
    use_csr_common_name="true" \
    use_csr_sans="true" \
    key_usage="DigitalSignature,KeyAgreement" \
    ext_key_usage="ServerAuth,ClientAuth"
```

Step 5 — issue:

```
# Returns certificate, private_key, ca_chain, serial_number, expiration, ...
bao write -format=json pki_int/issue/myapp \
    common_name="myapp.internal" \
    ttl="24h" > issued.json

python3 -c "import json;d=json.load(open('issued.json'))['data'];print(d['certificate'])" > c
python3 -c "import json;d=json.load(open('issued.json'))['data'];print(d['private_key'])" > k
```

※ `bao write -f` (raw format) does **not** reliably give you a single bare value for multi-field responses. Use `-format=json` and parse it — that's what the scripts in this course do.

Or let OpenBao generate the whole thing (key + CSR + cert) in one call — which is what the command above already does.

Step 6 — publish the issuing URLs and grab the chain:

```
bao write pki_int/config/urls \
    issuing_certificates="$BAO_ADDR/v1/pki_int/ca" \
    crl_distribution_points="$BAO_ADDR/v1/pki_int/crl"

# The CA chain endpoint serves raw PEM, not JSON — use curl:
curl -s --cacert ca.crt "$BAO_ADDR/v1/pki_int/ca" > ca-chain.crt
```

※ **Distribute `ca-chain.crt` (intermediate + root) to every client** as the trust anchor. Clients validate the leaf against it. The root cert alone is NOT enough — you need the chain. And the chain endpoint returns **raw PEM**, so `bao read` (which expects JSON) will fail on it — use `curl`.

PKI roles — the important knobs

Parameter	Meaning	Recommendation
`allowed_domains`	Exact list of permitted CNs	Always set it. Empty = any dor
`allowed_subdomains`	Allow `*.domain`	Only if you need it
`allow_bare_domains`	Allow `domain` without prefix	Usually `true` alongside subdom
`allow_localhost`	Allow `localhost`	Dev only
`allow_ip_sans`	Allow IP SANs	Usually `true` for k8s
`allow_any_name`	⚠ Allow **any** common name	**Never in production**
`enforce_hostnames`	Reject CNs not matching allowed list	`true`
`allow_bare_domains` + `use_csr_sans`	SAN handling	Understand before enabling
`server_flag` / `client_flag`	EKU	Set both for mTLS
`max_ttl`	Max leaf lifetime	72h–168h

[ALERT] **`allow_any_name = true` is the PKI equivalent of `path "*" { capabilities = ["*"] }`.** It lets anyone request a cert for `*.corp.internal` and MITM your services.

Never enable it outside a lab.

PKI in 2.7.0 — two big upgrades

ML-DSA (post-quantum, NIST FIPS 204):

```
bao write pki_int/roles/pqc-app \
    key_type="mldsa-44" \
    signature_bits=0 \
    max_ttl="24h"
```

⚠ The 2.7.0 notes carry an explicit warning: "Go's OCSP implementation does not support ML-DSA so will be unusable with ML-DSA typed issuers." If you rely on OCSP/CRL revocation checking, ML-DSA roles will not work with it.

External Keys — PKI private key in an HSM (2.7.0):

```
# Map a mount to an HSM/KMS-backed key
bao write sys/external-keys/pki_int \
    keys=my-hsm-key \
    backend=kms \
    # ... backend-specific parameters
```

The PKI engine then signs with the HSM key and **never holds the private key in OpenBao at all**. See Level 8.

Cert revocation

```
# Revoke a specific serial
bao write pki_int/revoke serial=<serial>

# Build and publish a CRL
bao write pki_int/crl/config expiration=72h
bao write pki_int/crl/rotate
```

※ **The real answer to revocation is short lifetimes.** If your leaves live 24h, a compromised cert is worthless tomorrow. Build CRL/OCSP distribution anyway, but don't depend on it.

ACME (for browsers/Let's Encrypt-style clients)

```
bao write pki_int/config/acme \
    enabled=true \
    directory="https://acme-v02.api.letsencrypt.org/directory" \
    email="pki@corp.internal" \
    dns_resolver="10.0.0.53:53" \
    http_01_port=5002
```

⚠ 2.7.0 security fix: "Forbid issuance of non-validated SANs through ACME" (GHSA-x8fg-h69x-p28f) — the ACME path now refuses to issue certs with SANs it hasn't validated. Also 2.7.0 added optional ACME client port/host configuration so you can run non-privileged behind a load balancer.

4.4 Database — dynamic credentials (the highest-value engine)

Why dynamic creds change everything

```
  BEFORE:  app ──► postgres_user "app" (password in every env file, forever)
  AFTER:   app ──► OpenBao ──► postgres_user "v-token-approle-x-2026-10-20"
                               (exists for 1 hour, then deleted)
```

Benefits: no shared password, per-request attribution in the DB's own logs, automatic revocation, no rotation job.

Step 1 — the OpenBao admin account in Postgres

```
-- Run as a superuser, ONCE
CREATE ROLE openbao_admin WITH LOGIN PASSWORD 'strong-password' CREATEROLE;
GRANT CONNECT ON DATABASE appdb TO openbao_admin;
```

CREATEROLE is enough. **Never give it SUPERUSER.** OpenBao needs to create roles; it does not need to own the database.

Step 2 — configure the connection

```
bao secrets enable -path=db database

bao write db/config/postgres \
    plugin_name=postgresql-database-plugin \
    connection_url="postgresql://{{username}}:{{password}}@postgres:5432/appdb?sslmode=requir
    username="openbao_admin" \
    password="strong-password" \
    verify_connection=true \
    allowed_roles="app,readonly"
```

※ The {{username}}:{{password}} placeholders in connection_url are substituted from the username/password fields. Don't put the real credentials in the URL.

Step 3 — define a role (a "dynamic role")

```
bao write db/roles/app \
    db_name=postgres \
    creation_statements="CREATE ROLE \"{{name}}\" WITH LOGIN PASSWORD '{{password}}' VALID UN
    revocation_statements="DROP ROLE IF EXISTS \"{{name}}\";" \
    default_ttl="1h" \
    max_ttl="24h" \
    renew_statements="ALTER ROLE \"{{name}}\" VALID UNTIL '{{expiration}}';" \
    username_template="v-token-{{identity.entity.id}}-{{random 8}}-{{timestamp}}"
```

Template variables available in creation_statements:

Variable	Value
{{name}}	The generated username
{{password}}	The generated password
{{expiration}}	Lease expiry timestamp (RHS of VALID UNTIL)
{{creation_timestamp}}	Issue time
{{identity.entity.id}}	Requesting entity
{{random 8}}	Random string

※ **username_template matters for attribution.** Include the identity ID so you can map a DB login back to an OpenBao entity in an incident. The default (`v-token-approle-...`) is fine but less useful.

Step 4 — request credentials

```
bao read db/creds/app
```

```
{
  "data": {
    "username": "v-token-abc123-8f3a-2026-10-20",
    "password": "A1b2C3d4E5f6G7h8",
    "lease_duration": 3600,
    "lease_id": "database/creds/app/..."
  }
}
```

Step 5 — revoke

```
bao lease revoke database/creds/app/...
# or all of them:
bao lease revoke -prefix database/creds/
```

The revocation statement runs → the Postgres role is dropped → the credential dies immediately. **Not "in 1 hour" — now.**

Static roles (rotate an existing user's password)

```
bao write db/static-roles/app \
    db_name=postgres \
    username="legacy_app_user" \
    rotation_statements="ALTER USER \"{{name}}\" WITH PASSWORD '{{password}}';" \
    rotation_period="24h"
```

OpenBao rotates the password on a schedule and hands the current value to whoever asks. Good for migrating legacy systems off static passwords.

Revocation safety

⚠ `revocation_statements` **must** clean up properly, or you leak database roles. Every leaked role is a credential you forgot about. Monitor the role count:

```
SELECT count(*) FROM pg_roles WHERE rolname LIKE 'v-token-%';
```

If that number is much larger than your active lease count, revocation is failing.

Other database plugins

Plugin	Notes
`postgresql-database-plugin`	✓ Built-in
`mysql-database-plugin`	✓ Built-in. 2.6.0 added **multi-host failover** (`tcp(host1:3306,host2:3306)`)
`cassandra-database-plugin`	✓ Built-in
`influxdb-database-plugin`	✓ Built-in
`valkey-database-plugin`	✓ Built-in

Plugin	Notes
`mongodb-database-plugin`	✗ Removed (SSPL licence) — external plugin
`custom-database-plugin`	Your own, via the plugin SDK

※ **`valkey-database-plugin` exists** — that covers Redis-compatible servers (Valkey). See §4.9.

4.5 TOTP — step-up MFA inside OpenBao

```
bao secrets enable -path=totp totp

# Register a key (from a user's authenticator app enrolment URI)
bao write totp/keys/alice \
    url="otpauth://totp/OpenBao:alice@corp.com?secret=Y64VEVMBTSXCYIWRSHRNDZW62MPGVU2G&issuer
# Generate a code (OpenBao acts as the authenticator)
bao read totp/code/alice

# Validate a code the user typed in
bao write totp/verify/alice code=123456
```

[SEC] TOTP as a generator inside OpenBao means the **seed never leaves OpenBao** — a genuinely stronger design than a phone app, and fully audited. It also means OpenBao can be a **TOTP provider** (like Google Authenticator does for Google).

4.6 SSH — certificates instead of authorized_keys

```
bao secrets enable -path=ssh ssh

# CA key
bao write ssh/config/ca generate_signing_key=true

# Sign user keys with the CA
bao write ssh/sign/root \
    public_key=@~/.ssh/id_ed25519.pub \
    valid_principals="alice" \
    valid_after="1h" \
    valid_before="8h"

# Sign host keys
bao write ssh/sign/host \
    public_key=@/etc/ssh/ssh_host_ed25519_key.pub \
    valid_principals="myapp.internal" \
    valid_after="1h" \
    valid_before="720h"
```

Clients then trust the CA cert instead of a list of keys:

```
TrustedUserCAKeys /etc/ssh/trusted_ca.pub
```

※ One CA cert replaces hundreds of `authorized_keys` entries. Revoking a user is a CA revocation, not editing 500 servers.

4.7 Cubbyhole — per-token private storage

Every token gets its own `cubbyhole/` — a private namespace nothing else can read, and which is destroyed when the token dies.

```
bao write cubbyhole/my-step secret=intermediate-value
bao read cubbyhole/my-step
```

Use it for multi-step flows where a value must pass from one operation to another without being visible in a shared path. Batch tokens don't have a cubbyhole.

4.8 Response Wrapping — hand a secret to someone else, not to yourself

Response wrapping encrypts a response into a **single-use token** that must be explicitly unwrapped.

```
# Wrap (any read command supports -wrap-ttl)
bao kv get -mount=secret -wrap-ttl=60s myapp
```

```
{
  "wrap_info": {
    "token": "hvs.CAESIJ...",
    "accessor": "6f1c...",
    "ttl": 60,
    "creation_time": "...",
    "creation_path": "secret/myapp"
  }
}
```

```
# Unwrap — single use, and it deletes itself
bao unwrap hvs.CAESIJ...
```

Why it matters:

1. The secret never appears in a log, a shell history, or a terminal scrollback.
2. The token is **single-use** — replaying it fails.
3. It's **time-boxed**.
4. You can **hand it to a broker** (a message queue, a ticket, a person) who then unwraps it and gets the secret — without you revealing it.

```
# Server-side: wrap a read
WRAPPED=$(bao read -field=wrap_info.token -wrap-ttl=60s secret/myapp)
# ... pass $WRAPPED through a queue ...
# Consumer side:
bao unwrap "$WRAPPED"
```

Wrapping TTL control in policies

```
path "secret/data/very-sensitive" {
  capabilities = ["read"]
  min_wrapping_ttl = "30s"
  max_wrapping_ttl = "120s"
}
```

Prevents someone wrapping it for 30 days.

Cubbyhole vs wrapping vs control groups

Mechanism	Who can unwrap	When it dies	Use for
Cubbyhole	Only the token's own identity	Token dies	Private scratch space
Response wrapping	Anyone with the wrap token	TTL, or on unwrap	Passing a secret through
Control groups (2.7.0)	Only after a second person approves	TTL	Guarding access behind t

---

4.9 Redis / Valkey — what actually exists

> **Straight answer: there is no `redis-database-plugin` in OpenBao 2.7.1.**

What does exist:

Option	Description	Trad
`valkey-database-plugin`	✓ Built-in. Valkey is the Linux Foundation's Redis fork.	Worl
Static KV v2	Store the Redis password in KV.	Simp
Redis ACL user rotation via `transit`/scripts	You'd script the rotation yourself.	You
The `redis` auth method	✗ Not an OpenBao thing.	n/a

The pragmatic recommendation for your stack

Use KV v2 with a short TTL and a rotation process for Redis, and treat the Redis password as a lower-value secret (Redis is usually network-isolated and bound to a private network anyway).

```
bao kv put -mount=secret myapp/redis \
    host=redis \
    port=6379 \
    password='...redis-password...' \
    tls=true

bao policy write redis-reader - <<'EOF'
path "secret/data/myapp/redis" {
  capabilities = ["read"]
}
EOF
```

Better: issue Redis **ACL users** with per-service permissions and rotate them on a schedule via a small script that calls `ACL SETUSER` using the OpenBao-stored admin password. That gets you the dynamic credential property even without a dedicated plugin.

```
# Example rotation script (run from a controlled environment)
PASSWORD=$(bao kv get -mount=secret -field=password myapp/redis/admin)
redis-cli -a "$PASSWORD" ACL SETUSER "app-$SERVICE" on '>~app:*' '+@read' '+@write' '+ping' '
redis-cli -a "$PASSWORD" ACL SAVE
```

> [SEC] **Never grant `+@all` to a service ACL user.** Scope the command categories and key patterns.

Postgres, by contrast, does have a first-class dynamic credential engine — use it there (§4.4). That asymmetry is real: Postgres dynamic creds are a solved problem in OpenBao; Redis dynamic creds are not, out of the box.

4.10 Engine tuning and lifecycle

```
bao secrets list -detailed
bao secrets tune -default-lease-ttl=1h -max-lease-ttl=24h db
bao secrets tune -listing-visibility=false secret     # hide from `bao secrets list`
bao secrets disable db        # revokes all leases + deletes the data
bao secrets move db db-old    # ⚠ revokes all leases (leases are path-bound)
```

> ⚠ **`bao secrets move` revokes every lease** under the old path, because leases are bound to the path they were created at. Same for `disable`.

4.11 Level 4 checkpoint

1. Why does KV v2 need `data/` AND `metadata/` paths in your policy?
2. What does `cas` protect you from, and what happens if you lose the race?
3. Why is `transit/export` the most dangerous path in OpenBao?
4. What is the difference between a KV v2 "delete" and a "destroy"?
5. Draw the three-tier PKI model. Which tier holds the private key that signs leaves? Which one must be offline?
6. What privilege does OpenBao's Postgres admin account need, and what should you never give it?
7. How do you revoke a dynamic database credential immediately?
8. What is response wrapping good for, and what makes it safe?
9. Is there a Redis dynamic-credentials engine in OpenBao 2.7.1? What do you do instead?
10. Why should `allow_any_name` never be enabled on a PKI role?

Next → **Level 5: SECURITY-HARDENING.md** — the production security baseline that everything else assumes.

4.12 Scratchpad

```
Mounts:        secret/(kv-v2) transit/ pki_root/ pki_int/ db/ totp/ ssh/
KV v2 paths:  <m>/data/<k> <m>/metadata/<k> <m>/delete/<k> <m>/undelete/<k> <m>/destroy/<k>
Transit:     transit/keys/<name> transit/encrypt/<name> transit/datakey/plaintext/<name>
PKI:           pki/root/generate/internal  pki/intermediate/generate/internal
           pki/root/sign-intermediate  pki/intermediate/set-signed  pki/issue/<role>
Database:         db/config/<name>   db/roles/<name>   db/creds/<role>   db/static-
roles/<name>
Wrapping:      -wrap-ttl=<dur>   → bao unwrap <token>
```

Level 5 — Production Security Standards

> **Goal:** this is the chapter that separates "a secrets manager running in production" from "a secrets manager that will get you fired". Every item here is a **top-level production grade security standard**, expressed as a decision with a reason and a command.

5.0 The baseline, at a glance

#	Control	Why	Where
5.1	TLS everywhere	Credentials in plaintext on the wire	§5.1
5.2	No/encrypted swap, no core dumps	Master key must not hit disk	§5.2
5.3	Auto-unseal design	Removes humans from the boot path	§5.3
5.4	Root token hygiene	Root can read everything	§5.4
5.5	Audit strategy	You must prove what happened	§5.5
5.6	Network exposure	Attack surface	§5.6
5.7	Quotas + lockout	Blast radius of a runaway app	§5.7
5.8	Policy hygiene	Least privilege	§5.8
5.9	Secret lifecycle	Rotation, not eternal secrets	§5.9
5.10	Supply chain	Plugins are code you run	§5.10
5.11	Patch cadence	Real CVEs ship often	§5.11
5.12	The checklist	Print it	§5.12

5.1 TLS everywhere — no exceptions

Why

Every request carries `X-Vault-Token`. Over plaintext HTTP, that token is readable by **anything on the network path**: a compromised switch, a rogue container on the same bridge network, a passive tap. TLS is not optional.

Server config

```
listener "tcp" {
  address         = "0.0.0.0:8200"
  cluster_address = "0.0.0.0:8201"

  tls_cert_file = "/openbao/certs/openbao.crt"
  tls_key_file  = "/openbao/certs/openbao.key"

  tls_min_version = "tls12"
  # tls_max_version = "tls13"   # optional

  # 2.7.0: reload TLS material automatically when the files change
  # (no SIGHUP needed)
  tls_auto_reload = true
}
```

Option	Notes
`tls_disable`	**Never** in production. Only for a loopback d
`tls_min_version`	`"tls12"` minimum. `"tls13"` if all clients su
`tls_cert_file` / `tls_key_file`	PEM cert + key.
`tls_client_ca_file` + `tls_require_and_verify_client_cert`	mTLS for the cluster listener — strong.
`tls_auto_reload`	**2.7.0** — hot-reloads cert files on change. G
`tls_key_exchange_preferences`	**2.7.0** — enforce post-quantum key exchang

> ※ **2.7.0 pure-PQC TLS.** You can now require post-quantum key exchange and even serve ML-DSA certificates. For a ”harvest now, decrypt later” threat model, this matters. Note the `tls_key_exchange_preferences` option works on server, agent and proxy listeners.

Client config

```
export BAO_ADDR=https://openbao.corp.internal:8200
export BAO_CACERT=/etc/ssl/certs/corp-internal-ca.crt
```

Or per-request: `curl --cacert /path/ca.crt https://openbao:8200/v1/sys/health`

Python client

```
client = hvac.Client(
    url="https://openbao.corp.internal:8200",
    verify="/etc/ssl/certs/corp-internal-ca.crt",   # CA bundle, NOT verify=False
    timeout=30,
)
```

> [NO] **`verify=False` in production is a TLS-without-authentication failure mode.** It disables the check that the server is who it claims. In a lab, fine. In production, it means anyone who can redirect your traffic can read your tokens.

Separate the public API from the cluster API

```
listener "tcp" {
  address         = "0.0.0.0:8200"     # client API
  cluster_address = "0.0.0.0:8201"     # node-to-node cluster traffic

  tls_cert_file   = "/openbao/certs/openbao.crt"
  tls_key_file    = "/openbao/certs/openbao.key"
  tls_min_version = "tls12"
  tls_auto_reload  = true
}
```

> ⚠ **The cluster listener inherits its parent listener's TLS settings.** There is no way to give `cluster_address` a different TLS configuration inside the same `listener` block, and declaring a second `listener "tcp"` block bound to the same cluster port collides on the bind.
>
> **Practical consequence:** you cannot have plain TLS on 8200 and mTLS on 8201 from a single listener block. Two ways to protect the cluster port:
>
> 1. **Network isolation (recommended):** firewall 8201 so only the other OpenBao nodes can reach it. Combined with `cluster_addr` pointing at a private interface, this is what actually stops a rogue container from joining your Raft cluster.
> 2. **mTLS everywhere on that listener:** set `tls_require_and_verify_client_cert = true` — but that then applies to the API port on the same listener too, so every client needs a cert. If you want both, terminate mTLS for 8201 with a proxy in front of it.
>
> Note: `tls_disable_client_certs` and `tls_require_and_verify_client_cert` are **mutually exclusive** — don't set both.

5.2 Swap, core dumps, and memory protection

The problem

OpenBao holds the master key in memory. If it gets written to swap or a core dump, a disk image of that machine = your entire secret store.

⚠ OpenBao 2.x does NOT use mlock

`disable_mlock` was **removed in OpenBao 2.0.0** (the "mlock removal" RFC, GH-363). The 2.7.1 server **refuses to start** if the option is present:

```
OpenBao has dropped support for mlock. Please remove the line "disable_mlock" = false
from your config and disable or encrypt swap instead.
```

So there is no `mlock()` call, no `CAP_IPC_LOCK`, and no `memlock` ulimit to worry about. The hardening OpenBao asks for instead is **swap**.

The controls

Control	What it does	How
No swap for the OpenBao process	The master key can never be paged out	systemd `MemorySwapMax=0`,
Encrypted swap (alternative)	A swap dump is unreadable	LUKS / `zswap` / `dmsetup cryp`
`kernel.core_pattern`	No core dumps	`kernel.core_pattern = |/`
`NoNewPrivs`	Prevent privilege escalation	Docker `security_opt`
`read_only` rootfs	Prevent tampering	Docker `read_only: true`
`vm.swappiness=0`	Reduce swap pressure generally	sysctl

Linux (systemd)

The OpenBao systemd unit ships with swap already disabled for the process. Verify:

```
systemctl cat openbao | grep MemorySwapMax
# MemorySwapMax=0        ← this is what you want
```

If you are not using systemd, set the cgroup v2 value instead:

```
# cgroup v2
echo "memory.swap.max=0" | sudo tee /etc/systemd/system/openbao.service.d/swap.conf
```

Linux (host-wide, less precise)

```
# Disable swap entirely
sudo swapoff -a
sudo sed -i '/ swap / s/^/#/' /etc/fstab
```

> ⚠ The OpenBao docs note that disabling swap for the whole OS "is not recommended". Prefer the per-process approach.

Docker

The OpenBao docs are explicit: **"When running the Docker image, include the flag `--memory-swappiness=0`."**

```
services:
  openbao:
    image: openbao/openbao:2.7.1
    # Equivalent of --memory-swappiness=0
    memswap_limit: 0
    security_opt:
      - no-new-privileges:true
    read_only: true
    tmpfs:
      - /tmp
```

BSD / macOS / Windows

```
# BSD: confirm swap is encrypted
# (FreeBSD, NetBSD, OpenBSD have guides linked in the OpenBao docs)

# macOS: swap is always encrypted — nothing to do

# Windows (PowerShell) — you said Windows 11
fsutil behavior query encryptpagingfile
# 0 = DISABLED  →  enable it:
fsutil behavior set encryptpagingfile 1
# then reboot
```

Core dumps

```
echo "kernel.core_pattern=|/bin/false" | sudo tee /etc/sysctl.d/99-openbao.conf
sudo sysctl --system
```

Container privileges (2.7.0 change)

> ⚠ **2.7.0 container change:** the image now runs as the openbao user and **drops privileges automatically** before starting. If you rely on running as root, specify `user: root` explicitly. Also **all `VOLUME` instructions were removed** — you must declare your own volumes, and they must be writable by the openbao user (uid `100` / gid `1000` in the standard image).

5.3 Auto-unseal — designing the seal story

This is the single most consequential architectural decision you make.

Option A — Shamir seal (default, no external dependency)

`5 shares, threshold 3. 3 different humans must be present to boot the cluster.`

Pros	Cons
No external dependency at boot	Humans in the boot path
Nothing to keep alive besides OpenBao	Awkward for automated deployments
Strongest isolation from the platform	Slow disaster recovery (need 3 people)

The secret_id problem in CI/CD: if you must automate unsealing, you're storing unseal keys in a secret store — which is what you're trying to avoid. OpenBao's own docs flag this as "the discouraged pattern of automating Shamir's manual unseal" and point to **static key auto-unseal** instead.

Option B — Transit seal (OpenBao unseals itself using another OpenBao)

```
seal "transit" {
  address            = "https://bao-unseal.corp.internal:8200"
  disable_local_ca   = false
  ca_cert            = "/openbao/certs/unseal-ca.crt"
  tls_skip_verify    = false
  # The transit engine name that holds the key
```

```
  mount_path         = "transit"
  key_name           = "auto-unseal-key"
  # Recovery key configuration
  recovery_key_shares    = 5
  recovery_key_threshold = 3
}
```

The second cluster ("unseal cluster") holds the key that unseals the first. It's a **deliberate, documented trust boundary**: whoever controls the unseal cluster controls your main cluster.

Pros	Cons
Fully automated unseal	Extra cluster to run and secure
No human in the boot path	Two-cluster failure mode
Transit can be backed by a KMS/HSM	More moving parts

Option C — Cloud KMS seal (2.7.0: now a plugin)

> ⚠ **2.7.0 removed the built-in `awskms`, `gcpckms`, `azurekeyvault`, `alicloudkms`, `ocikms` and `pkcs11` seals from the binary.** They are now KMS plugins from `https://github.com/openbao/openbao-plugins`.

```
# 1. Register the plugin
plugin "kms" "awskms" {
  # pulled from OCI, or installed locally
  # image = "ghcr.io/openbao/openbao-plugin-kms-awskms:<version>"
  # or:
  command = "openbao-plugin-kms-awskms"
}

# 2. Use it as a seal
seal "awskms" {
  region     = "eu-west-1"
  mount_path = "awskms/"
  key_name   = "my-openbao-unseal-key"
  # ... provider-specific options
}
```

The AWS example in full (from the 2.6+ model):

```
plugin "kms" "awskms" {
  command = "openbao-plugin-kms-awskms"
}

seal "awskms" {
  region                    = "eu-west-1"
  mount_path                = "awskms/"
  key_name                  = "prod-openbao-unseal"
  credential_type           = "iam"
  iam_role_arn              = "arn:aws:iam::123456789012:role/openbao-unseal"
  iam_endpoint              = "https://iam.eu-west-1.amazonaws.com"
  sts_endpoint              = "https://sts.eu-west-1.amazonaws.com"
  sts_region                = "eu-west-1"
  rotation_period           = "720h"
  recovery_key_shares       = 5
  recovery_key_threshold    = 3
  recovery_key_pgp_keys     = "alice.asc,bob.asc,carol.asc,dave.asc,erin.asc"
```

```
  disable_automated_rotation  = false
  min_encryption_key_version  = 0
}
```

Option D — Static key seal (for fully automated environments)

```
seal "static" {
  secret_path = "/openbao/seals/unseal.key"   # 0600, on encrypted storage
  recovery_key_shares    = 5
  recovery_key_threshold = 3
}
```

> [RED] The static key is a **single file whose loss = total loss**, and whose theft = total compromise. The docs explicitly say to use this only when you have "an automated alternative to Shamir's stored in Kubernetes secrets, OpenStack Barbican, or similar static secret stores". Use it as a **last resort**, protect it with disk encryption + strict permissions, and understand you have re-created the problem you're solving.

Decision matrix

Scenario	Recommended seal
Small team, manual ops, air-gapped-ish	**Shamir** (5/3)
Production K8s / Docker with automation	**Transit seal** (2-cluster) or **cloud KMS plugin**
Cloud-native, IAM already nailed down	**Cloud KMS plugin** (AWS/GCP/Azure)
HSM / compliance requirement	**PKCS#11 KMS plugin** (from `openbao-plugins`)
Fully automated, no external deps	**Static** (with heavy caveats)

The non-negotiable rules

1. **Never lose the KMS key.** The docs are blunt: "If the seal mechanism or its keys are permanently deleted, then the OpenBao cluster cannot be recovered, **even from backups**."
2. **Guard the KMS key with IAM/SCP-style controls** so a compromised OpenBao node can't delete the key that unseals it.
3. **Always generate recovery keys** and store them offline.
4. **Rehearse** the recovery path. An unrehearsed break-glass is a failed break-glass.
5. **Monitor seal health.** 2.6.0 added a periodic seal health check:

```
seal "awskms" {
  # ...
  health_check_enabled        = true
  health_check_timeout        = "1m"
  health_check_interval       = "10m"
  health_check_interval_unhealthy = "1m"
}
```

> When the health check fails, OpenBao logs warnings — so you find out **before** you need to unseal.

Seal migration

Migrating between seal types requires **downtime** and both mechanisms available simultaneously. The procedure (from the docs):

1. Snapshot first.

2. Take a standby down, add the new `seal` block (or `disabled = "true"` on the old one), bring it back up, unseal with `-migrate`.
3. Repeat for each standby, one at a time (keeps quorum).
4. `bao operator step-down` the active node.
5. The new active node performs the migration — watch the logs.
6. Take down the old active node, update its config, bring it up.
7. Clean up the config files to have only the new seal block.

⚠ **Known limitation:** migrating **from AWSKMS to AWSKMS** is currently not supported. All other same-type migrations are.

5.4 Root token hygiene

> "In fact the OpenBao team recommends that root tokens are only used for just enough initial setup... and are revoked immediately after they are no longer needed."

The lifecycle of a root token

```
1.  Init produces a root token         ← use ONCE
2.  Use it to create: a real admin path (OIDC + break-glass policy)
3.  Revoke the root token immediately  ← DO THIS
4.  If you ever need root again:
    bao operator generate-root -init    ← requires a quorum of recovery keys
    ... use it, then revoke it again
```

```
# 3. Revoke it
bao token revoke <root-token>
bao token lookup | jq    # → confirms it's gone
```

Creating a proper admin path instead

```
# A break-glass role with real (but non-root) power
bao policy write breakglass-admin - <<'EOF'
path "secret/data/*" {
  capabilities = ["create", "read", "update", "patch", "delete", "list", "scan"]
}
path "secret/metadata/*" {
  capabilities = ["read", "list"]
}
path "sys/mounts"  { capabilities = ["read", "list"] }
path "sys/auth"    { capabilities = ["read", "list"] }
path "sys/audit"   { capabilities = ["read", "list"] }
path "sys/policies/acl" { capabilities = ["list"] }
path "auth/token/lookup-self" { capabilities = ["read"] }
path "auth/token/renew-self"  { capabilities = ["update"] }
EOF

# Bind it to an identity group reachable only via a protected path
bao write identity/group name=breakglass policies=breakglass-admin
```

⚠ Note what `breakglass-admin` **cannot** do: create tokens, write policies, mount engines, use `sys/raw`. It manages secrets, not the system. That's a deliberate design choice: break-glass should not be a path to full compromise.

Two-person rule with control groups (2.7.0)

Level 8 covers control groups in depth. The concept: even a break-glass secret read requires a second approver. This is the strongest anti-insider control OpenBao offers.

Emergency root generation

```
# Requires a quorum of recovery keys
bao operator generate-root -init
# → asks for recovery key 1 of 5, 2 of 5, 3 of 5
# → prints a new root token

# Use it, then revoke it
bao token revoke <new-root-token>
```

> ✓ Since 2.6.0 this uses the **authenticated** `/sys/generate-root-token` endpoints. The old unauthenticated `/sys/generate-root/attempt` is deprecated and disabled by default.

Never store root tokens

✗ Never	✓ Instead
`export BAO_TOKEN=hvs.root...` in a long-lived shell	`bao login` + token helper
Root token in `.env`	`bao login` in the deploy pipeline, or AppRole
Root token in CI secrets	OIDC/JWT auth for CI (Level 7)
Root token in a shared vault	A break-glass policy + `generate-root`

5.5 Audit strategy

⚠ In 2.7.1, audit devices are created from the CONFIG, not the API

`bao audit enable` now fails by default:

```
cannot enable audit device via API; use declarative, config-based
audit device management instead
```

This is a **deliberate hardening change**: an operator (or a compromised admin token) can no longer add or remove an audit device at runtime, so an attacker cannot quietly disable your audit trail. The config file is the source of truth, and the config is reviewed in git.

The declarative audit device (2.4.0+)

The `audit` stanza takes **two labels** — the device type and the mount path — and `options` is a **block** of **string** values:

```
audit "file" "file" {
  description = "Local audit log (declarative)."
  options {
    file_path = "/openbao/logs/audit.log"
    mode      = "0600"
    log_raw   = "false"
  }
}

audit "socket" "remote" {
```

```
  description = "Ship audit logs to the SIEM collector."
  options {
    address      = "tcp://audit-sink.internal:9023"
    socket_type  = "tcp"
    write_timeout = "2s"
  }
}
```

> ⚠ `log_raw` is a **string** (`"false"`), not a boolean. `options` is a block, not an `=` map. Getting either wrong makes the server refuse to start with a cryptic parse error.

Always validate before restarting:

```
bao operator validate-config -config=/openbao/config/openbao.hcl
```

Audit devices defined this way are enabled on the active node at startup and on SIGHUP. Removing the stanza removes the device — so **keep the configuration identical across all nodes**.

The escape hatch: `unsafe_allow_api_audit_creation`

If you genuinely need to add a device at runtime (an emergency second sink, a test), set this server option — it can be toggled with SIGHUP without a restart:

```
unsafe_allow_api_audit_creation = true
```

```
bao audit enable -path=file file file_path=/var/log/openbao/audit.log mode=0600
# ...then set the option back to false and SIGHUP
```

The name is a warning. **Enable it, do the thing, turn it back off.**

A sibling option, `allow_audit_log_prefixing`, allows devices that use the audit-log prefixing mode. Same discipline: temporary only.

Why at least two devices

OpenBao considers a request successful if it can log to **at least one** device. If all devices fail, **OpenBao refuses to serve requests**. That's fail-closed and correct — but it means a single failed device is a single point of failure, and a full disk on the audit sink takes the cluster down. Two devices = redundancy + tamper-evidence (compare HMACs between two logs to detect tampering).

The rules

Rule	Why
Define devices in the config	The API path is disabled by default in 2.7.1
`log_raw = "false"` **always**	Otherwise secrets land in the audit log in plaintext
`mode = "0600"`	The audit log is a security artefact
Same config on every node	Otherwise devices drift between nodes
Ship off-box	An attacker with root on the box can edit the local log
Ship to immutable storage	Same reason — use WORM/SIEM
Monitor disk space	A full audit disk = a down cluster
Alert on audit failure	Detect the fail-closed condition before users do
Never enable `log_raw` "temporarily"	There is no temporary

Log shipping

```
openbao ──▶ local file (0600)     [declared in config]
        └──▶ socket ──▶ rsyslog/fluentbit ──▶ SIEM (Splunk/Elastic/Sentinel)
```

```
                                                  └──▶ immutable/WORM bucket
```

What to alert on

```
- denied requests (403) spiking        → credential stuffing or probing
- reads of secret/data/* by a new IP   → possible exfiltration
- login failures by user               → brute force
- token creation rate spike            → token minting abuse
- audit device failure                 → fail-closed imminent
- seal health check failure            → auto-unseal about to break
```

5.6 Network exposure

The default posture should be: OpenBao is reachable from your application networks and from nowhere else.

Port	Purpose	Expose to
8200	Client API	Application networks only
8201	Cluster API (Raft)	Only other OpenBao nodes

Controls

1. **Network policies / security groups** — 8201 open only to other nodes.
2. **mTLS on the cluster listener** — `tls_require_and_verify_client_cert` (note: it inherits the parent listener's TLS config, so it applies to the API port of that listener too — otherwise firewall 8201 to node-to-node).
3. **No public ingress.** Not even for the UI. Put the UI behind SSO + VPN.
4. **A reverse proxy / load balancer** in front for TLS termination only if you also keep TLS to the node (don't downgrade to plaintext behind the LB).
5. **Firewall the unauthenticated endpoints** if your platform can't do path-based filtering — a proxy that only allows `/v1/sys/health` and `/v1/auth/*` from the internet.

   ⚠ `sys/health`, `sys/seal-status`, `sys/leader`, `sys/metrics` and `sys/pprof/*` may be reachable unauthenticated depending on listener config. `sys/metrics` leaks a lot of operational detail. `sys/pprof` is a profiling endpoint — **never** expose it publicly.

Disable pprof explicitly

```
# There is no explicit "disable"; it is off unless enabled.
# Just make sure the listener isn't exposed and consider:
# (no pprof_options stanza = disabled)
```

Actually — pprof in OpenBao is enabled via the `pprof` listener option in some versions. In 2.7.1, confirm with:

```
bao sys/config/state/sanitized | jq '.data.listeners'
```

and never put a public listener in front of it.

5.7 Quotas, rate limits and lockout

Rate limit quotas

```
# Global rate limit: 50 requests/second per client IP
bao write sys/quotas/rate-limit/global \
    rate=50 \
    path="" \
    block_interval=60

# Per-mount
bao write sys/quotas/rate-limit/kv \
    rate=200 \
    path="secret/" \
    block_interval=30

# Per-role login limit (most specific wins)
bao write sys/quotas/rate-limit/oidc-login \
    rate=5 \
    path="auth/oidc/login" \
    role="human-login"
```

※ **Precedence:** the most specific rule wins — path > mount > namespace > global; a rule with a `role` beats all of them. Quotas are **per-node, per client IP**, and are not replicated across the cluster.

Lease count quotas

```
# Limit the number of leases in a mount (prevents a crashloop from
# creating thousands of database users)
bao write sys/quotas/lease-count/db \
    max_leases=1000 \
    path="db/"
```

[ALERT] This is a real, observed failure mode: ”a deployment that requests credentials in a tight loop, a crashlooping pod, or a job that never revokes will pile up leases until revocation lags and the database fills with users.” Set lease-count quotas on your database mounts.

User lockout

Enabled by default:

Setting	Default
Lockout threshold	5 failed attempts
Lockout duration	15 minutes
Counter reset	15 minutes after last attempt

Configurable:

```
user_lockout "all" {
  lockout_threshold = 5
  lockout_duration  = "15m"
  lockout_counter_reset = "15m"
}
```

```
user_lockout "userpass" {
  disable_lockout = false
}
```

Or per-mount:

```
bao write auth/userpass/tune disable_lockout=false
```

> ⚠ **Only `userpass`, `ldap` and `approle` support lockout.** Not OIDC/JWT (the IdP handles that) and not kubernetes.

Lease TTL discipline

```
# Never let a mount hand out long-lived dynamic creds
bao secrets tune -default-lease-ttl=1h -max-lease-ttl=4h db
```

5.8 Policy hygiene

The five rules

1. **Deny by default.** Start from nothing, add what you need.
2. **Never `path "*" { capabilities = ["*"] }`.** If you must have a broad grant, enumerate capabilities and carve out deny paths.
3. **Never `sudo` on `sys/raw`.** That is root.
4. **Scope by path AND by capability.** `read` on `secret/data/app` ≠ `list` on `secret/metadata/app`.
5. **Separate humans from machines.** Different auth methods, different policies, different groups.

The policy review checklist (repeat from Level 3)

```
□ Narrowest possible path?
□ Minimum capabilities?
□ Does it need list/scan (leaks data shape)?
□ Does it need sudo (almost never)?
□ Shared write path? (someone can overwrite you)
□ deny carve-outs for sensitive sub-paths?
□ Survives "app fully compromised"?
□ Scoped to one namespace?
```

Namespace isolation as a security boundary

```
bao namespace create team-a
bao namespace create team-b
```

Each namespace has its own mounts, policies, auth methods, tokens. Root namespace admins are **not** automatically admins inside a namespace. Use namespaces to make ”team A cannot read team B’s secrets” a structural property rather than a policy promise.

> [SEC] Cross-namespace identity references are off by default. `unsafe_cross_namespace_identity` should stay **off** — the docs say it ”allows for unconfined permission sharing”.

5.9 Secret lifecycle — rotation without downtime

The rotation matrix

Secret type	How it rotates	Automation
Dynamic DB creds	Lease expiry + revocation	Built in ✓
PKI leaf certs	TTL expiry + client renewal	Built in ✓
KV v2 static values	You write a new version	Script / CI
Transit keys	`transit/keys/<n>/rotate`	Script / CI
Cloud IAM creds	Cloud plugin lease	Plugin ✓
Passwords in an external IdP	IdP's job	Keycloak ✓

Automated KV rotation with a controlled window

```
#!/usr/bin/env bash
# rotate-secret.sh — rotates a KV v2 value and keeps the old one valid briefly
set -euo pipefail

MOUNT="${1:?mount}"; KEY="${2:?key}"; NEW_VALUE="${3:?new value}"
TTL="${4:-24h}"

# 1. Write the new value
bao kv put -mount="$MOUNT" "$KEY" password="$NEW_VALUE"

# 2. Keep the previous version readable for the overlap window
#    (KV v2 keeps N versions by default — verify max_versions is high enough)

# 3. Audit
echo "rotated $MOUNT/$KEY at $(date -Is)" >&2
```

Wire it into GitHub Actions with an OIDC login (Level 7) — **never** with a stored root token.

Rotation policy for a real org

```
LLM provider API key   → every 90 days, or immediately on suspicion
Database password      → dynamic creds, no static password at all
TLS leaf certs         → every 24h (automatic)
Transit keys           → every 6 months, or per NIST 800-38D guidance
Service-to-service     → dynamic creds
Human passwords        → Keycloak's job
```

5.10 Supply chain — plugins are code you execute

> **2.7.0 moved a LOT of functionality into plugins.** That makes plugin management a first-class supply-chain concern.

What moved to plugins in 2.7.0

Moved	Where it lives now
`awskms`, `gcpckms`, `azurekeyvault`, `alicloudkms`, `ocikms`, `pkcs11` seals	`openbao/openbao-plugins`
`kerberos` auth	`openbao/openbao-plugins`
`ldap` auth	`openbao/openbao-plugins`

Moved	Where it lives now
radius auth	openbao/openbao-plugins
ldap secrets	openbao/openbao-plugins

Declarative plugin distribution (2.5.0+, hardened in 2.7.0)

```
plugin_directory = "/openbao/plugins"

plugin_auto_download   = true
plugin_auto_register   = true    # ← default true since 2.7.0
plugin_download_behavior = "fail"  # fail loudly rather than silently skip

plugin "kms" "pkcs11" {
  # Pin by digest — 2.7.0 feature
  image = "ghcr.io/openbao/openbao-plugin-kms-pkcs11@sha256:<digest>"
  # 2.7.0: binary_name is auto-detected from ENTRYPOINT/CMD if unset
  # binary_name = "openbao-plugin-kms-pkcs11"
}

plugin "secrets" "aws" {
  image = "ghcr.io/openbao/openbao-plugin-secrets-aws:2.7.1"
}
```

[SEC] **Supply-chain rules:**

1. **Pin by digest**, never by tag. Tags move.
2. `plugin_auto_download = true` means OpenBao **pulls and executes code at boot**. Only do that from a registry you control, over TLS, with digests pinned. If you don't want auto-download, set it to `false` and install plugins as a build step.
3. `plugin_download_behavior = "fail"` — don't let a failed download silently degrade to "no plugin".
4. 2.5.5 added `plugin_download_max_size` (default 512 MB) to limit the size of downloaded plugin binaries. Keep it.
5. `plugin_auto_register = true` (2.7.0 default) means a downloaded plugin is registered automatically. Understand that before enabling it.
6. 2.7.0 fixed a security issue where the plugin command name was not resolved relative to `plugin_directory` (GHSA-j6wc-jpvg-xfxq) and another restricting catalog writes to the root namespace (GHSA-cg72-x35g-xfp8). **Run 2.7.1.**

Manual plugin management

```
bao plugin init <sha256> <command> [args...]   # register a local binary
bao plugin list
bao plugin info <name>
bao plugin register <sha256> <command>
bao plugin reload <name>                        # 2.7.0: KMS plugin configs reload on SIGHUP
bao plugin prune                                # 2.7.0: remove unused OCI downloads
bao plugin deregister <name>
```

※ **2.7.0 improvement:** KMS plugin configuration stanzas now reload on SIGHUP and respawn plugin clients — so you can upgrade a KMS plugin **without restarting the whole cluster**.

5.11 Patch cadence

OpenBao ships security fixes frequently. Look at 2.7.1 alone:

GHSA	What it fixed
GHSA-3237-j65r-m5rp	`tls_acme_eab_mac_key` disclosure via `sys/config/state/sanitized`
GHSA-q74w-hv5x-6hxf	Unauthenticated root-gen/rekey endpoints returned a response even when audit loggi
GHSA-7m59-mp95-w6ph	**AppRole secret_id usable after expiry** (before `tidy` ran)
GHSA-5v22-543h-wg96	AppRole form-data login JSON-decode path
GHSA-gf3j-hm38-jhh5	**Kubernetes SA JWT not validated on renewal** — a revoked token could keep rene

That is **five security fixes in a patch release**. The Kubernetes one is the scariest: a revoked service account token could keep renewing.

The policy

```
  Security patch releases (2.7.1, 2.6.3, 2.5.5)   → within 7 days
  Minor releases (2.7.0, 2.6.0)                    → within 30 days, after testing
 Major/minor upgrades                          → planned window, never "just deploy it"
```

Automate it:

```yaml
# .github/workflows/openbao-patch-watch.yml
name: OpenBao patch watch
on:
  schedule:
    - cron: "0 9 * * 1"     # Mondays 09:00 UTC
  workflow_dispatch:

jobs:
  check:
    runs-on: ubuntu-latest
    steps:
      - uses: actions/checkout@v4
      - name: Check latest release
        id: rel
        run: |
          LATEST=$(curl -s https://api.github.com/repos/openbao/openbao/releases/latest \
                   | jq -r .tag_name)
          echo "latest=$LATEST" >> $GITHUB_OUTPUT
      - name: Compare with deployed version
        run: |
          if [ "${{ steps.rel.outputs.latest }}" != "v2.7.1" ]; then
            echo "::warning::OpenBao ${{ steps.rel.outputs.latest }} is available (deployed:
          fi
```

> [NOTE] Also: **2.7.0 removed the `file` storage backend** and **deprecated it for `operator migrate`** (removal in 2.8.0). If you use pebbledb, note the deprecation is scoped to `bao operator migrate`.

5.12 The complete hardening checklist

Print this. Tick it. Do it in this order.

Before first production traffic

```
□ TLS enabled on every listener, tls_min_version >= tls12
□ BAO_CACERT configured on every client; no verify=False anywhere
□ Swap disabled (or encrypted) for the OpenBao process; core dumps disabled
□ Audit device(s) enabled — at least two, log_raw=false, mode=0600
□ Audit log shipped off-box to immutable storage
□ Root token revoked
□ Break-glass admin path created and TESTED
□ Recovery keys generated, stored offline, and the procedure rehearsed
□ Seal health check enabled and alerting
□ All policies reviewed: no "*" capabilities, no sudo, no sys/raw
□ Unused auth methods disabled
□ Unused secret engines disabled
□ Quotas configured (rate limit + lease count on dynamic mounts)
□ Namespaces created for team isolation (if multi-team)
□ Network policies: 8201 only between nodes; no public ingress
□ Cluster port protected (mTLS on the listener, or firewall to node-to-node)
□ Snapshot taken and a RESTORE tested in a lab
□ Monitoring + alerting live (health, audit failures, seal health, lease counts)
```

Ongoing

```
□ Patch cadence enforced
□ Secret rotation schedule defined and automated
□ Access reviews (quarterly): tokens, policies, entities, groups
□ Lease/credential leak check: DB role count vs active lease count
□ Restore drill performed at least twice a year
□ Unseal-key/unseal-cluster access reviewed
□ Plugin versions pinned and tracked
```

5.13 Level 5 checkpoint

1. Why is `verify=False` in a Python client a security failure, not a convenience?
2. Why does OpenBao 2.x have no `disable_mlock` option, and what does it ask you to do instead?
3. Which auto-unseal mechanisms are **no longer built in** to 2.7.1?
4. What happens to your cluster if the KMS key that unseals it is deleted?
5. What happens to OpenBao if all audit devices fail? Is that good or bad?
6. Why is `log_raw` never acceptable in production?
7. What's the difference between a rate-limit quota and a lease-count quota, and which one stops the "crashloop creates 10,000 DB users" problem?
8. Why should `unsafe_cross_namespace_identity` stay off?
9. Name three things you should pin about a plugin.
10. Which GHSA from 2.7.1 would have let a revoked Kubernetes token keep working?

Next → Level 6: HA-PRODUCTION.md — the reference production architecture: 5-node Raft, PostgreSQL backend, snapshots, upgrades, observability, disaster recovery.

5.14 Scratchpad

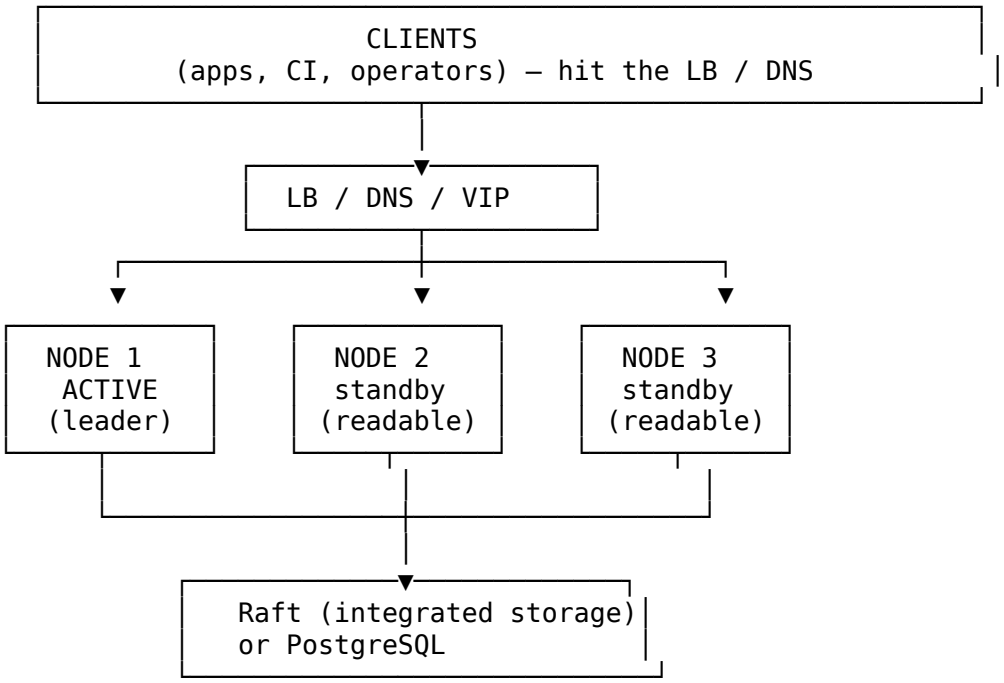
```
TLS:                tls_min_version="tls12"  tls_auto_reload=true  (2.7.0)
Swap:               no disable_mlock in 2.x! disable/encrypt swap (mem_swap.max=0)
Seal (2.7.0):       plugin "kms" "name" {...} + seal "name" {...}
Audit:              ≥2 devices, log_raw=false, mode=0600, ship off-box
```

```
Root token:          init → use → revoke → generate-root when needed
Quotas:              sys/quotas/rate-limit, sys/quotas/lease-count
Lockout:             userpass/ldap/approle only; 5 tries / 15 min default
Plugins:             pin by sha256 digest, plugin_download_behavior="fail"
Patches:             2.7.1 fixes: GHSA-3237, q74w, 7m59, 5v22, gf3j
```

Level 6 — High Availability & Production Architecture

Goal: you can design, deploy, upgrade, back up, restore and monitor a production OpenBao cluster — and you know the failure modes before they happen.

6.1 The HA mental model



The rules of HA in OpenBao

Rule	Detail
One active node	Only the leader performs writes. All writes go through Raft consensus on
Standbys serve reads	Since **2.5.0** (Raft) and **2.7.0** (PostgreSQL), standbys answer storage-rea
Reads are eventually consistent	A write on the leader may not be visible on a standby for a moment. 2.7
Unsealed standbys only	A sealed node cannot be a standby.
Quorum is king	Lose 2 of 3 voters and the cluster is down.
Sealing is per-node	With Shamir, each node needs its own threshold of shares.
Failover is automatic	If the active node dies, a standby is elected. Typically seconds.

Sizing

Cluster size	Fault tolerance	Use when
1 node	0 (single point of failure)	Dev, or a non-critical internal tool
3 nodes	1 node	**The minimum for production.**
5 nodes	2 nodes	Serious production, larger blast radius tolerance
7 nodes	3 nodes	Rarely worth it — more IO, slower consensus

※ **Start with 3.** 5 nodes gives you one more failure tolerance but costs more IO and slower Raft commits. If you can only afford one production-grade cluster, 3 well-run nodes beat 5 poorly-tuned ones.

Voter vs non-voter

Raft nodes are either **voters** (part of quorum) or **non-voters** (standby, replicate but don't vote). You can add extra non-voters for read capacity without affecting quorum.

```
bao operator raft autopilot state
```

```
Healthy:                       true
Failure Tolerance:             1
Leader:                        raft1
Voters:
   raft1
   raft2
   raft3
Non-Voters:
   raft4
```

6.2 The reference architecture: 5-node Raft in Docker Compose

This is a complete, runnable production-shaped cluster.

Directory

```
openbao-ha/
├── docker-compose.yml
├── config/
│   ├── node1.hcl ... node5.hcl
├── certs/
├── init.sh
└── scripts/
```

`config/node1.hcl` (nodes 2–5 differ only in `node_id` and `api_addr`)

```
ui           = true
cluster_name = "openbao-prod"

api_addr     = "https://openbao-1:8200"
cluster_addr = "https://openbao-1:8201"

storage "raft" {
  path    = "/openbao/data"
  node_id = "openbao-1"
```

```
  # Production tuning
  performance_multiplier = 1     # highest performance; needs good network
  snapshot_threshold     = 8192
  snapshot_interval      = 120
  trailing_logs          = 10000

  # Auto-join: lets nodes find the leader without manual `raft join`
  retry_join {
    retry_join_initial_interval = "1s"
    retry_join_max_interval     = "30s"
    retry_join_max_attempts     = -1
  }
}

listener "tcp" {
  address         = "0.0.0.0:8200"
  cluster_address = "0.0.0.0:8201"
  tls_cert_file   = "/openbao/certs/openbao.crt"
  tls_key_file    = "/openbao/certs/openbao.key"
  tls_min_version = "tls12"
  # The cluster listener inherits this listener's TLS config, so enforce
  # node-to-node access to 8201 at the NETWORK layer (firewall), not with
  # client certs here — client certs would also apply to the API port.
  tls_disable_client_certs = false
}

# ⚠ No `disable_mlock` in OpenBao 2.x (removed in 2.0.0). Harden swap
#    instead — see Level 5, §5.2.

telemetry {
  prometheus_retention_time = "30s"
  disable_hostname = false
  usage_gauge_period = "1m"
}

default_lease_ttl = "24h"
max_lease_ttl     = "168h"
```

> ※ **2.6.0 added `auto_join` via DNS SRV records** (and 2.7.0 fixed `auto_join` breaking on providers that include ports in discovered addresses, GH-4113). With `retry_join` stanzas you can pre-declare all peers and let them join automatically.

docker-compose.yml

```
x-openbao-base: &openbao-base
  image: openbao/openbao:2.7.1
  command: server -config=/openbao/config/openbao.hcl
  volumes:
    - ./certs:/openbao/certs:ro
    - ./logs:/openbao/logs
  memswap_limit: 0
  security_opt:
    - no-new-privileges:true
  healthcheck:
    test: ["CMD-SHELL", "BAO_CACERT=/openbao/certs/ca.crt bao status >/dev/null 2>&1"]
```

```
    interval: 10s
    timeout: 5s
    retries: 12
  restart: unless-stopped
  networks: [openbao]

services:
  openbao-1:
    <<: *openbao-base
    container_name: openbao-1
    volumes:
      - ./certs:/openbao/certs:ro
      - ./logs:/openbao/logs
      - openbao-data-1:/openbao/data
    environment:
      BAO_RAFT_NODE_ID: openbao-1
      BAO_ADDR: https://openbao-1:8200
    depends_on: [openbao-2, openbao-3, openbao-4, openbao-5]

  openbao-2:
    <<: *openbao-base
    container_name: openbao-2
    volumes:
      - ./certs:/openbao/certs:ro
      - ./logs:/openbao/logs
      - openbao-data-2:/openbao/data
    environment:
      BAO_RAFT_NODE_ID: openbao-2
      BAO_ADDR: https://openbao-2:8200

  openbao-3:
    <<: *openbao-base
    container_name: openbao-3
    volumes:
      - ./certs:/openbao/certs:ro
      - ./logs:/openbao/logs
      - openbao-data-3:/openbao/data
    environment:
      BAO_RAFT_NODE_ID: openbao-3
      BAO_ADDR: https://openbao-3:8200

  openbao-4:
    <<: *openbao-base
    container_name: openbao-4
    volumes:
      - ./certs:/openbao/certs:ro
      - ./logs:/openbao/logs
      - openbao-data-4:/openbao/data
    environment:
      BAO_RAFT_NODE_ID: openbao-4
      BAO_ADDR: https://openbao-4:8200

  openbao-5:
    <<: *openbao-base
    container_name: openbao-5
    volumes:
```

```
      - ./certs:/openbao/certs:ro
      - ./logs:/openbao/logs
      - openbao-data-5:/openbao/data
    environment:
      BAO_RAFT_NODE_ID: openbao-5
      BAO_ADDR: https://openbao-5:8200

networks:
  openbao:
    driver: bridge

volumes:
  openbao-data-1:
  openbao-data-2:
  openbao-data-3:
  openbao-data-4:
  openbao-data-5:
```

Bootstrap sequence

```
# 1. Start the cluster (all nodes come up sealed & uninitialized)
docker compose up -d

# 2. Init ONE node — the storage backend is shared, so one init is enough
export BAO_ADDR=https://openbao-1:8200
export BAO_CACERT=./certs/ca.crt

bao operator init -key-shares=5 -key-threshold=3

# 3. Unseal node 1
bao operator unseal '<key1>'; bao operator unseal '<key2>'; bao operator unseal '<key3>'

# 4. Join the other nodes to the Raft cluster
for n in 2 3 4 5; do
  BAO_ADDR=https://openbao-$n:8200 bao operator raft join https://openbao-1:8200
done

# 5. Unseal each remaining node (each needs its own threshold!)
for n in 2 3 4 5; do
  for k in 1 2 3; do
    BAO_ADDR=https://openbao-$n:8200 bao operator unseal '<key>'
  done
done

# 6. Verify
bao operator raft list-peers
bao operator raft autopilot state
bao status
```

> ⚠ **Raft membership is security-critical.** Anyone who can reach the cluster port and has a valid TLS client cert could try to join. That's why `tls_require_and_verify_client_cert = true` on the cluster listener matters — and why `cluster_addr` must be firewalled to node-to-node only.

Verify

```
bao operator raft list-peers
```

```
Node              ID                Address         State     Voter
openbao-1         openbao-1         openbao-1:8201  leader    true
openbao-2         openbao-2         openbao-2:8201  follower  true
openbao-3         openbao-3         openbao-3:8201  follower  true
```

```
bao status
```

```
{
  "sealed": false,
  "initialized": true,
  "storage_type": "raft",
  "cluster_name": "openbao-prod",
  "cluster_id": "..."
}
```

Removing a node

```
bao operator raft remove-peer openbao-5
```

Then stop the container and delete its data volume. **Always remove the peer before destroying the node**, otherwise Raft keeps trying to contact a dead member and wastes time on elections.

6.3 The PostgreSQL storage backend (the alternative)

When to choose it

Choose Raft when	Choose PostgreSQL when
You want a self-contained system	You already have a battle-tested Postgres with HA + backups
You want snapshot/restore built in	You want all your data in one backup/DR story
Fewer operational dependencies	You have Postgres expertise and monitoring
You don’t want a distributed database	You want SQL-level inspection of the data

Config

```
storage "postgresql" {
  connection_url = "postgres://openbao:CHANGE_ME@postgres-primary:5432/openbao?sslmode=verify

  # HA mode
  ha_enabled = "true"
  ha_table   = "openbao_ha_locks"

  # 2.7.0: read scalability — REQUIRES physical replication
  # (cluster_addr must be reachable from standbys to the primary)

  table                = "openbao_kv_store"
  skip_create_table    = "false"
  max_parallel         = "128"
  max_idle_connections = 10
  max_connect_retries  = 5
```

```
}

cluster_addr = "https://openbao-1:8201"
api_addr     = "https://openbao-1:8200"
```

Critical 2.7.0 caveat: physical replication only

> ”Only works with PostgreSQL physical replication; will not work with logical replication.”

That means:

- ✓ Streaming replication (native Postgres)
- ✓ Patroni / HAProxy / pgbouncer in front of a streaming-replica pair
- ✗ Logical replication (pglogical, postgres_fdw, logical decoding)
- ✗ Some managed ”read replica” services that use logical replication

Verify before you design around it:

```
SELECT * FROM pg_stat_replication;
-- `replication_state = 'streaming'` on the replica = physical. Good.
```

TLS to Postgres

```
connection_url = "postgres://openbao:CHANGE_ME@postgres:5432/openbao?sslmode=verify-full&sslr
```

> ⚠ The plugin **attempts SSL by default**. If your Postgres doesn’t have SSL enabled, you must explicitly set `sslmode=disable` — or better, turn on SSL and use `verify-full`.

Manual table creation (for a least-privilege DB user)

```
CREATE TABLE openbao_kv_store (
  parent_path TEXT COLLATE "C" NOT NULL,
  key         TEXT COLLATE "C" NOT NULL,
  hashed_path TEXT COLLATE "C" NOT NULL,
  hashed_key  TEXT COLLATE "C" NOT NULL,
  value       BYTEA NOT NULL,
  created_at  TIMESTAMPTZ NOT NULL,
  PRIMARY KEY (parent_path, key)
);
CREATE INDEX openbao_kv_store_hashed_path ON openbao_kv_store (hashed_path);
CREATE INDEX openbao_kv_store_hashed_key  ON openbao_kv_store (hashed_key);

CREATE TABLE openbao_ha_locks (
  id           INTEGER NOT NULL PRIMARY KEY,
  network_addr TEXT,
  ha_enabled   BOOLEAN
);
```

Then set `skip_create_table = "true"` so the OpenBao DB user never needs DDL rights.

6.4 Consistency — the 2.7.0 headers

The problem

With standby reads enabled (default since 2.5.0), you can write to node A and read from node B and **not** see your write. That's eventual consistency.

The 2.7.0 solution

```
# 1. Write — the response includes X-Vault-Index
curl -sD- -X POST \
  -H "X-Vault-Token: $TOKEN" \
  -d '{"data":{"value":"x"}}' \
  https://openbao:8200/v1/secret/data/myapp | grep -i x-vault-index
# X-Vault-Index: 3f9a2c...

# 2. Read, demanding consistency
curl -s -X GET \
  -H "X-Vault-Token: $TOKEN" \
  -H "X-Vault-Index: 3f9a2c..." \
  -H "X-Vault-Inconsistent: forward-active-node" \
  https://openbao:8200/v1/secret/data/myapp
```

`X-Vault-Inconsistent` values:

Value	Behaviour
`fail`	Respond **429 Too Many Requests** with `Retry-After`
`forward-active-node`	Forward the request to the active node
`await-state`	Hold the request locally until it catches up (optionally falling back to the above)

Listener defaults (2.7.0)

```
listener "tcp" {
  # ...
  consistency_fallback_behavior      = "forward-active-node"
  consistency_missing_header_forward = false
  consistency_max_index_wait         = "30s"
}
```

> ※ **When do you actually need this?** Read-after-write matters when a pipeline writes a secret and the app immediately reads it. For a write-once-at-deploy, read-many-later pattern, you don't need it. Use `await-state` sparingly — it holds connections and adds latency.

Disabling standby reads

```
disable_standby_reads = true
```

Every request then goes to the active node → **strong consistency, no read scaling**. The right choice when correctness trumps throughput (rare), and the **safe first setting** while you're still learning.

6.5 Snapshots, backup and restore

Raft snapshots

```
# Save
bao operator raft snapshot save /backup/openbao-$(date +%F-%H%M).snap

# Inspect
bao operator raft snapshot inspect /backup/openbao-2026-10-20-1200.snap

# Restore (destroys current data!)
bao operator raft snapshot restore /backup/openbao-2026-10-20-1200.snap
```

> ⚠ **Restore replaces the entire cluster state.** The cluster must be stopped/managed carefully, and this is a **destructive** operation. Practise it in a lab first.

The restore runbook (the one you'll actually need)

```
# ═══ DISASTER RECOVERY: full cluster restore from snapshot ═══

# 1. STOP EVERYTHING. All nodes. No exceptions.
docker compose down

# 2. Preserve the evidence (if this is an incident)
cp -a openbao-data-1/ /incident-evidence/node1-$(date +%s)/

# 3. Wipe all data volumes
docker volume rm openbao-ha_openbao-data-{1,2,3,4,5}

# 4. Start ONE node only
docker compose up -d openbao-1

# 5. Restore the snapshot into that node
export BAO_ADDR=https://openbao-1:8200
export BAO_CACERT=./certs/ca.crt
bao operator raft snapshot restore /backup/openbao-2026-10-20-1200.snap

# 6. Unseal it
bao operator unseal '<key1>'; bao operator unseal '<key2>'; bao operator unseal '<key3>'

# 7. Bring up the rest and re-join them
docker compose up -d
for n in 2 3 4 5; do
  BAO_ADDR=https://openbao-$n:8200 bao operator raft join https://openbao-1:8200
done
for n in 2 3 4 5; do
  for k in 1 2 3; do BAO_ADDR=https://openbao-$n:8200 bao operator unseal '<key>'; done
done

# 8. VERIFY
bao operator raft list-peers
bao operator raft autopilot state
bao status
bao secrets list
bao policy list
```

[RED] **The unseal keys are the same across the cluster.** If you lost the unseal shares, a snapshot does **not** help — you need the recovery keys (auto-unseal) or the original unseal shares. And if you lost the KMS key that unseals the cluster, **the snapshot is useless**: the data is encrypted with a key that only the KMS-held root key can derive. This is the single most important sentence in this chapter.

Backup strategy

```
Every hour:   Raft snapshot → /backup (local, then off-box)
Daily:         Snapshot → object storage (encrypted, versioned, immutable)
Quarterly:     FULL RESTORE DRILL into a scratch cluster  ← the important one
```

```
#!/usr/bin/env bash
# snapshot-backup.sh — run from cron
set -euo pipefail

STAMP=$(date -u +%Y%m%d-%H%M%S)
FILE="/backup/openbao-${STAMP}.snap"

export BAO_ADDR=https://openbao-1:8200
export BAO_CACERT=/openbao/certs/ca.crt

bao operator raft snapshot save "$FILE"
sha256sum "$FILE" > "$FILE.sha256"

# Ship off-box
aws s3 cp "$FILE" "s3://openbao-backups/raft/" --sse aws:kms \
  --sse-kms-key-id "$BACKUP_KMS_KEY" || \
rclone copy "$FILE" "remote:openbao-backups/raft/"

# Retention: keep 24 hourly, 14 daily, 12 monthly
find /backup -name 'openbao-*.snap' -mtime +14 -delete

echo "snapshot ok: $FILE"
```

※ **A backup you have never restored is a hope, not a backup.** The quarterly drill is the single highest-value item on this page.

Postgres backend backup

If you're on PostgreSQL, you back up Postgres — but you also need the **seal material** and the **unseal/recovery keys** to make the data usable.

```
pg_dump -Fc -d openbao > openbao-$(date +%F).dump
```

Plus: snapshots of the same logical state, because `pg_dump` is point-in-time-consistent but you still need a matching seal.

6.6 Upgrades

The rolling upgrade procedure

```
# 1. Check the current state
bao status
bao operator raft autopilot state
```

```
bao operator raft snapshot save /backup/pre-upgrade-$(date +%F).snap

# 2. Read the release notes for EVERY version you're skipping.
#    2.5 → 2.6 → 2.7 each have changes.
#    https://openbao.org/community/release-notes/

# 3. Upgrade the STANDBY nodes first, one at a time.
#    (Never upgrade the active node first.)

# 4. For each standby: stop → change image tag → start → unseal → verify
docker compose stop openbao-2
# edit docker-compose.yml: openbao/openbao:2.7.0
docker compose up -d openbao-2
# unseal it, verify it's a healthy follower:
bao operator raft autopilot state

# 5. Step down the active node so the last-upgraded node becomes leader
bao operator step-down

# 6. Now upgrade the (new) active node
docker compose stop openbao-1
docker compose up -d openbao-1
# unseal

# 7. Verify the whole cluster
bao status
bao operator raft autopilot state
bao secrets list
bao policy list
```

Cross-version compatibility

> ⚠ 2.7.0 fixed a bug in **cross-version request forwarding during rolling upgrades to v2.6.x** (GH-3900). This is exactly the kind of thing that breaks a rolling upgrade.

Rules:

1. **Read every intermediate release note.** 2.5→2.6→2.7 is not one upgrade.
2. **Upgrade one minor version at a time.** Don't jump 2.4 → 2.7.
3. **Always snapshot first.**
4. **One node at a time.** Keep quorum.
5. **Unseal every node after restart.**
6. **Don't upgrade the leader first.**
7. **Have the previous image tag ready** for a fast rollback.

Notable upgrade gotchas in 2.5 → 2.7

Change	What you must do
`file` storage backend removed (2.7.0)	Migrate to `raft` or `postgresql` **before** upgrading
pkcs11/cloud seals removed from binary (2.7.0)	Install the KMS plugins **before** upgrading, or you're
Kerberos/LDAP/RADIUS auth removed (2.7.0)	Install the plugins **before** upgrading
`openbao-hsm` image discontinued (2.7.0)	Switch to the standard image + `kms-pkcs11` plugin
`sys/generate-root/attempt` deprecated (2.6.0)	`bao operator generate-root` already uses the ne
`stored_shares` removed from `sys/init` (2.6.0)	Stop sending it
`build_date` → `commit_date` (2.6.0)	Update any scripts reading that field

Change	What you must do
`sys/health` lost `performance_standby/last_wal` (2.6.0)	Update monitoring checks
Container image labels changed (2.6.0)	Use OpenContainers labels, not the old ones
Container runs as openbao user (2.6.0)	Fix volume ownership

※ **Plugin-first upgrade rule:** if your config references a plugin that is no longer built in, install and register the plugin **before** you upgrade the binary. Otherwise the new node comes up and can't unseal.

6.7 Observability

Metrics

```
telemetry {
  prometheus_retention_time = "30s"
  disable_hostname  = false
  usage_gauge_period = "1m"
}
```

```
curl -s --cacert certs/ca.crt https://openbao:8200/v1/sys/metrics | grep -E '^(vault|openbao)
```

Key metrics to alert on:

Metric	Alert when
`vault.core.unsealed`	`== 0` on any node → **critical**
`vault.core.active`	`== 0` cluster-wide → no leader
`vault.raft.leader.lastContact`	rising → leader problems
`vault.expire.lease`	lease expiry rate spikes
`vault.core.fsm.pending`	rising → storage lag
`vault.raft.commit.lag`	rising → replication lag
`vault.token.count`	rising fast → token leak
`vault.secret.kv.*` (per mount)	unexpected growth
`vault.quota.*`	rate limit hits (an app is misbehaving)

[NOTE] 2.7.0 added a dedicated metrics loop that reports `vault.core.unsealed=0` when OpenBao is sealed, so you get an accurate signal instead of a stale one.

Health endpoint

```
curl -s --cacert certs/ca.crt https://openbao:8200/v1/sys/health | jq
```

```
{
  "initialized": true,
  "sealed": false,
  "standby": false,
  "replication_performance_mode": "disabled",
  "replication_dr_mode": "disabled",
  "server_time_utc": 1792483433,
  "version": "2.7.1",
  "cluster_name": "vault-cluster-b075f70e",
  "cluster_id": "13c7cf87-0830-03ae-5aa0-5393b55b80bf"
}
```

> ⚠ **Do not script against `performance_standby`, `last_wal` or `enterprise`** — those fields were **removed in 2.6.0**. The 2.7.1 response has `replication_performance_mode` and `replication_dr_mode` instead, and there is no `enterprise` field at all (OpenBao has no enterprise tier).

HTTP status codes: `200` = initialized, unsealed, active. `429` = standby. `472` = DR secondary. `473` = performance standby. `501` = not initialized. `503` = sealed.

Audit log monitoring

```
# Denied requests in the last hour
jq 'select(.error != "" and (.time > (now-3600)|todate))' audit.log | \
  jq -r '.request.path' | sort | uniq -c | sort -rn

# Distinct source IPs reading secrets
jq -r 'select(.request.path | startswith("secret/data/")) | .request.remote_address' audit.lo

# Token creation rate
jq -r 'select(.request.operation=="update" and (.request.path|startswith("auth/token/create")
```

Logs

```
docker compose logs -f openbao-1
docker compose logs -f openbao-1 | grep -i "seal\|unseal\|election\|raft"
```

Set `BAO_LOG_LEVEL=info` (default). Use debug temporarily, `trace` only for a specific incident (it prints request/response pairs — **including secrets**).

> ⚠ **`TRACE` logs contain sensitive information.** The self-initialization docs say so explicitly. Never leave trace on.

6.8 Disaster recovery scenarios

Scenario	Impact	Recovery
One node dies	None (if ≥3 voters)	Automatic failover. Restart the node, re-unseal, re-jo
Active node dies	Brief write pause	Automatic election (seconds).
Two of three voters lost	**Cluster down**	Restore from snapshot.
Disk corruption on one node	None	Replace the node's data volume, re-join.
All nodes lost	**Total outage**	Restore snapshot + unseal keys.
KMS key deleted	**Unrecoverable**	Even backups are useless. Prevent with IAM controls
Unseal shares lost	**Unrecoverable** (Shamir)	Recovery keys (if auto-unseal) or restore + new init
Audit disk full	**Cluster stops serving**	Free space immediately; add capacity.
Bad policy pushed	Some access broken	`bao policy write` the previous version (policies are
Malicious token minted	Blast radius	Revoke the token/ancestor, `lease revoke -prefix`,

The "bad deploy" runbook

```
# 1. What changed?
bao audit list -detailed
git log --oneline -10        # your config repo

# 2. Roll back the policy
```

```
bao policy read app-readonly > /tmp/old-policy.hcl
git revert <commit> && bao policy write app-readonly - < /tmp/old-policy.hcl

# 3. Revoke the offending token
bao token revoke -accessor <accessor>
bao lease revoke -prefix secret/

# 4. Verify
bao token capabilities <token> secret/data/myapp
```

> ※ **Policies are versioned.** `bao policy read` shows a `version` field, and each write increments it. Keep your policies in **git** and treat the repo as the source of truth — then rollback is a `git revert`.

6.9 Infrastructure as Code

Declarative self-initialization (2.4.0+)

This closes the dangerous window between ”initialized” and ”configured” where a root token exists and **nothing is audited**.

```
# The audit device is declared with the `audit` stanza (see below) —
# in 2.7.1 you CANNOT create it via the API.
audit "file" "file" {
  description = "Local audit log (declarative)."
  options {
    file_path = "/openbao/logs/audit.log"
    mode      = "0600"
    log_raw   = "false"
  }
}

# Self-initialization stanzas (2.4.0+) run on first start.
initialize "auth" {
  request "enable-oidc" {
    operation = "update"
    path      = "sys/auth/oidc"
    data = { type = "oidc", description = "Keycloak OIDC" }
  }

  request "enable-approle" {
    operation = "update"
    path      = "sys/auth/approle"
    data = { type = "approle", description = "Machine auth" }
  }
}

initialize "secrets" {
  request "enable-kv" {
    operation = "update"
    path      = "sys/mounts/secret"
    data = { type = "kv", options = { version = "2" } }
  }
}
```

⚠ **In 2.7.1, do NOT create the audit device through self-initialization either** — the underlying API call is disabled by default, so the request would fail at runtime. Use the declarative `audit "file" "file" { ... }` stanza (shown above) for the audit device, and reserve `initialize` for auth methods, secret engines and policies.

[NOTE] 2.6.0 added **CEL support**, **`text/template` support**, a **`when`** conditional, custom **headers**, and self-init stanzas in **dev mode**. 2.6.0 also made self-initialization **error on unknown keys** instead of silently ignoring typos.

⚠ **Self-initialization requires an auto-unseal mechanism** — it does not generate recovery keys, and the root token is **revoked after use** (it's never returned to you). Use the authenticated `/sys/rotate/recovery` endpoints to generate recovery keys afterwards.

※ **Sensible split (from the OpenBao team's own advice):** keep self-initialization minimal — enable audit, enable one human OIDC path, one machine path, create one admin policy. Everything else belongs in OpenTofu/Terraform or a reviewed Git repo.

The OpenTofu/Terraform approach (the right way at scale)

```
# main.tf
resource "openbao_mount" "kv" {
  path = "secret/"
  type = "kv"
  options = { version = "2" }
}

resource "openbao_policy" "app_readonly" {
  name   = "app-readonly"
  policy = <<-EOT
path "secret/data/myapp" {
  capabilities = ["read"]
}
path "secret/metadata/myapp" {
  capabilities = ["read", "list"]
}
EOT
}

resource "openbao_approle_role" "myapp" {
  role_id        = "myapp"
  token_policies = ["app-readonly"]
  token_ttl      = 1200
  token_max_ttl  = 3600
}
```

Run it from CI with a **short-lived, least-privilege** token, not a root token. Configure the provider with `BAO_TOKEN` from a **GitHub Actions OIDC** login (Level 7).

6.10 Level 6 checkpoint

1. How many nodes do you need to tolerate 1 failure? 2 failures?
2. What happens to a write followed immediately by a read on a different node with standby reads enabled? How do you fix it in 2.7.0?
3. Why must `cluster_addr` be set when using the Raft backend?
4. What is the correct order for a full cluster restore from a snapshot?
5. Why is a snapshot useless if the KMS key is gone?

6. In a rolling upgrade, why do you upgrade standbys before the active node?
7. Which 2.7.0 removals would leave you unable to unseal if you didn't prepare?
8. What happens to the cluster if the audit log disk fills up?
9. Why does Postgres HA read-scaling require physical replication?
10. What does `bao operator raft remove-peer` prevent you from having to do by hand?

Next → **Level 7: INTEGRATION-STACK.md** — wiring OpenBao into your actual stack: Keycloak, FastAPI, LangChain, PostgreSQL, Redis, GitHub Actions.

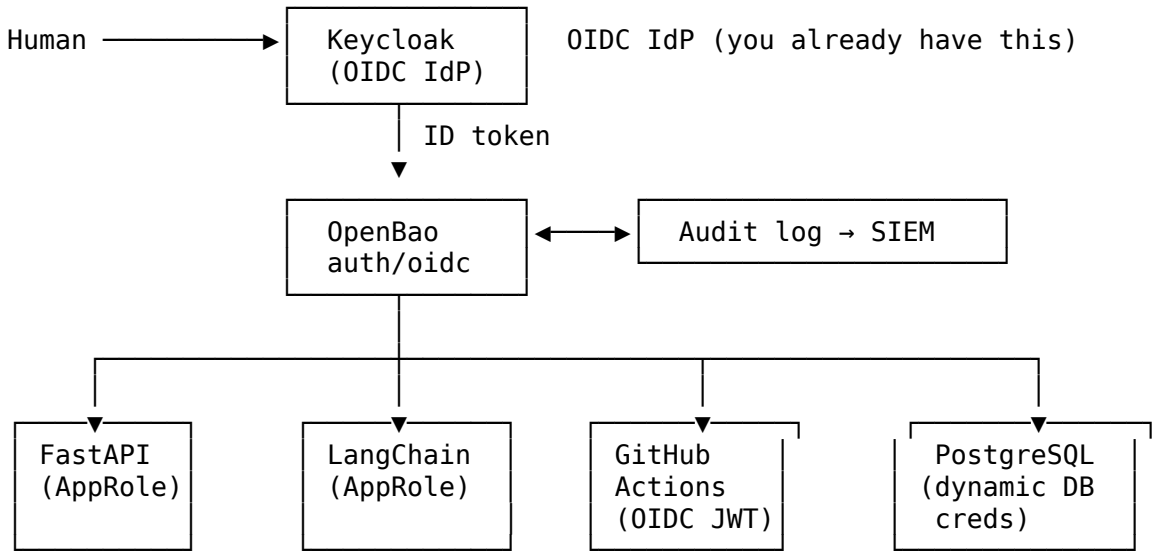
6.11 Scratchpad

```
Raft:       storage "raft" { path, node_id, performance_multiplier=1 }
Join:       bao operator raft join https://openbao-1:8200
Peers:      bao operator raft list-peers
Autopilot:  bao operator raft autopilot state
Snapshot:   bao operator raft snapshot save|inspect|restore <file>
Step down:  bao operator step-down
Members:    bao operator members
Consistency:  X-Vault-Index  +  X-Vault-Inconsistent:  forward-active-node|await-
state|fail
Postgres:   storage "postgresql" { connection_url, ha_enabled, ha_table }
```

Level 7 — Integration With Your Stack

> **Goal:** OpenBao is now part of your system, not a thing next to it. Keycloak for humans, AppRole/JWT for machines, GitHub Actions for CI, FastAPI and LangChain consuming secrets safely, PostgreSQL with dynamic credentials.

7.0 The integration map



The one rule that makes this work: every consumer authenticates with its **own identity**, and gets a **scoped token**. No shared credentials, ever.

7.1 Keycloak → OpenBao (OIDC)

7.1.1 The Keycloak side

In Keycloak, create a **client** for OpenBao:

Setting	Value
Client type	`OpenID Connect`
Client ID	`openbao`
Access type	`confidential`
Standard flow	**enabled**
Direct access grants	disabled
Valid redirect URIs	`http://localhost:8250/oidc/callback` and `https://openbao.corp.internal:8200/`
Web origins	`https://openbao.corp.internal:8200`
Client secret	copy it (it's needed for `oidc/client_secret`)
Scope	add the `groups` client scope (see below)

[DEBUG] **Keycloak groups claim — the gotcha everyone hits.** Keycloak does **not** put groups in the ID token by default. You must add a **group membership mapper** to the client's scope:

1. Create a client scope (e.g. `openbao-groups`) with a **Hardcoded claim** mapper or, better, a **Group membership** mapper with:
   - "Full group path" → **off** (otherwise you get `/platform/sre`)
   - "Claim" → `groups`
2. Assign the scope to the client (Client → Client scopes → Add).

Without this, OpenBao sees no groups and every user lands in the default policy.

Also create a **group** in Keycloak, e.g. `openbao-sre`, and add a client scope mapper for it. And note Keycloak's **groups overage**: a user in a very large number of groups will have the claim omitted entirely rather than truncated. Design for that (use a dedicated mapper on a specific group, not "all groups").

7.1.2 The OpenBao side

```
# 1. Enable the OIDC auth method
bao auth enable -path=oidc oidc

# 2. Point it at Keycloak's discovery endpoint
bao write auth/oidc/config \
    oidc_discovery_url="http://keycloak:8080/realms/enterprise" \
    oidc_client_id="openbao" \
    oidc_client_secret="<from-keycloak>" \
    default_role="human"
```

[ALERT] **Two corrections that catch everyone:**

1. **The config path is `auth/<mount>/config`** — not `auth/oidc/oidc` (that path returns "unsupported path").
2. **`oidc_discovery_url` is the BASE URL.** OpenBao appends `/.well-known/openid-configuration` itself, and **rejects** the full URL: "'oidc_discovery_url' contains '.well-known' component".
3. The keys are **`oidc_client_id` / `oidc_client_secret`** — plain `client_id` / `client_secret` are silently ignored.

Check the result:

```
bao read -format=json auth/oidc/config | jq -r .data.status
# → "valid"   ← discovery fetched and verified
```

The `issuer` in Keycloak's discovery document must match the URL OpenBao uses to reach Keycloak. If OpenBao reaches it at `http://keycloak:8080` (Docker DNS) but the token says `https://keycloak.corp.internal`, validation fails. **Make them match**, and set Keycloak's frontend URL to match too.

7.1.3 The role — this is where identity mapping happens

```
bao write auth/oidc/role/human \
    user_claim="preferred_username" \
    oidc_scopes="openid profile email groups" \
    role_type="oidc" \
    allowed_redirect_uris="http://localhost:8250/oidc/callback,https://openbao.corp.internal:
    token_ttl="1h" \
    token_max_ttl="8h" \
    token_policies="human-default"
```

For group-based mapping, use a **JSON write** (map values can't be set via individual CLI flags):

```
bao write auth/oidc/role/human - <<'EOF'
{
  "user_claim": "preferred_username",
  "oidc_scopes": "openid profile email groups",
  "role_type": "oidc",
  "allowed_redirect_uris": "http://localhost:8250/oidc/callback",
  "token_ttl": "1h",
  "token_max_ttl": "8h",
  "bound_claims": {
    "groups": ["openbao-sre", "openbao-platform"]
  }
}
EOF
```

7.1.4 Test the login

```
bao login -method=oidc -path=oidc -role=human
# → opens your browser → Keycloak login → back to OpenBao
```

```
bao token lookup | jq '.data.policies, .data.entity_id'
```

7.1.5 Debugging OIDC (the docs' own advice)

> "Start simple. The only claim configuration a role requires is `user_claim`."

1. Start with just `user_claim` and `oidc_scopes`. Get it working.
2. Then add `bound_claims`.
3. **Check OpenBao's logs** — validation failures are logged there.
4. **Redirect URIs must match exactly**: http vs https, `127.0.0.1` vs `localhost`, port numbers, trailing slashes.
5. Decode the token yourself to see the claims:

   ```
   cat jwt.json | jq -r .access_token | cut -d. -f2 | base64 -d | jq
   ```

6. `log_oidc_info = true` on the role logs the received token — **debug only**, it writes claims to the server log verbatim.

⚠ **2.7.0 improvement:** `auth/oidc` now returns `oidc_discovery_url` validation errors (like a bad issuer) from the underlying library, instead of a generic failure. If your setup suddenly starts failing loudly, read the message.

7.1.6 Keycloak → OpenBao with the `jwt` auth method (alternative)

Use `jwt` instead of `oidc` when you want to validate tokens Keycloak issued without the browser redirect (e.g. service-to-service, or a token your frontend already holds).

```
bao auth enable -path=keycloak-jwt jwt

bao write keycloak-jwt/jwt/keycloak \
    jwks_url="http://keycloak:8080/realms/enterprise/protocol/openid-connect/certs" \
    bound_issuer="http://keycloak:8080/realms/enterprise" \
    bound_audiences="openbao" \
    user_claim="preferred_username" \
    roles_claim="/realm_access/roles" \
    token_policies="human-default" \
    token_ttl="1h" \
    token_max_ttl="8h"
```

Login with a token:

```
bao write auth/keycloak-jwt/login role=keycloak jwt="$KEYCLOAK_ACCESS_TOKEN"
```

※ **When to use `oidc` vs `jwt`:**

- `oidc` = interactive browser login. **This is what you want for humans.**
- `jwt` = you already have a signed JWT and want OpenBao to trust it.
- `oidc` supports PKCE and the device flow; `jwt` is simpler but there's no browser flow.

7.2 Machines: AppRole in Docker Compose

The pattern

```
  role_id     → baked into the image/config (NOT a secret)
  secret_id   → injected at deploy time by a trusted party (secret)
```

Option A — entrypoint script (simplest, most common)

`docker-compose.yml`:

```
services:
  app:
    build: ./app
    environment:
      BAO_ADDR: https://openbao:8200
      BAO_CACERT: /certs/ca.crt
      APPROLE_ROLE_ID: ${APPROLE_ROLE_ID}
      APPROLE_SECRET_ID: ${APPROLE_SECRET_ID}   # injected by CI, never committed
    volumes:
      - ./certs:/certs:ro
    depends_on:
      openbao:
        condition: service_healthy
```

`app/entrypoint.sh`:

```
#!/usr/bin/env bash
set -euo pipefail

echo "authenticating to OpenBao..."
export BAO_TOKEN=$(bao write -format=json auth/approle/login \
    role_id="${APPROLE_ROLE_ID}" \
    secret_id="${APPROLE_SECRET_ID}" | jq -r .auth.client_token)

# Optional: background token renewal
(
  while true; do
    sleep 240
    export BAO_TOKEN=$(bao write -format=json auth/token/renew-self \
        token="$BAO_TOKEN" increment="20m" | jq -r .auth.client_token) || {
      echo "renewal failed, re-authenticating" >&2
      export BAO_TOKEN=$(bao write -format=json auth/approle/login \
          role_id="${APPROLE_ROLE_ID}" \
          secret_id="${APPROLE_SECRET_ID}" | jq -r .auth.client_token)
    }
  done
) &

exec "$@"
```

> ⚠ **The `_FILE` convention is better.** Most apps (including many Docker images) support reading a secret from a file. Combine it with a Docker secret or a mounted file so the secret never appears in `docker inspect` output or the process environment of other processes:

```
environment:
  APPROLE_SECRET_ID_FILE: /run/secrets/approle_secret_id
secrets:
  - approle_secret_id
```

Option B — `bao agent` sidecar (the professional answer)

`bao agent` handles authentication, renewal, and secret rendering for you. The app never sees a token at all.

`agent.hcl`:

```
pid_file = "/tmp/bao-agent.pid"

vault {
  address      = "https://openbao:8200"
  ca_cert      = "/openbao/certs/ca.crt"
  retry {
    num_retries = 5
  }
}

auto_auth {
  method "approle" {
    mount_path = "auth/approle"
    config = {
      role_id   = "..."
      secret_id = "..."
    }
```

```
  }

  # Write the token to a file the app can read (mode 0640, owned by the app user)
  sink "file" {
    config = {
      path = "/secrets/openbao-token"
      mode = 0640
    }
  }
}

template {
  source            = "/secrets/openbao-token"
  destination       = "/secrets/openbao-token"
  error_on_missing_key = true
  wait { min_duration = "5s" }
}
```

docker-compose.yml:

```
services:
  bao-agent:
    image: openbao/openbao:2.7.1
    container_name: bao-agent
    command: agent -config=/agent/config/agent.hcl
    volumes:
      - ./config:/agent/config:ro
      - agent-secrets:/secrets
    depends_on:
      openbao:
        condition: service_healthy
    restart: unless-stopped

  app:
    build: ./app
    volumes:
      - agent-secrets:/secrets:ro
    environment:
      BAO_ADDR: https://openbao:8200
      BAO_CACERT: /certs/ca.crt
      BAO_TOKEN_FILE: /secrets/openbao-token
    depends_on: [bao-agent]

volumes:
  agent-secrets:
```

※ **`bao agent` templates secrets to files** — including arbitrary transformations. Your app reads `/secrets/db_password` and never handles a token. This is the **best pattern for containers**: it means a compromised app process can't escalate to "read everything in OpenBao", and the agent handles re-auth transparently.

Option C — `bao proxy` (for apps that speak the OpenBao API)

`bao proxy` sits in front of OpenBao, caches responses, and handles authentication. Your app points at `http://127.0.0.1:8100` instead of OpenBao.

```
pid_file = "/tmp/bao-proxy.pid"

vault {
  address = "https://openbao:8200"
}

api_addr = "127.0.0.1:8100"

auto_auth {
  method "approle" {
    mount_path = "auth/approle"
    config = { role_id = "...", secret_id = "..." }
  }
}

cache {
  use_static_token = true
  persist = "memory"
}

listener "tcp" {
  address = "127.0.0.1:8100"
  tls_disable = true      # it's a loopback sidecar listener
}
```

> ※ **Proxy is great for the ”many small reads” pattern** — it caches KV reads so you’re not hammering OpenBao. But **cached secrets have a TTL you must choose**, and a proxy that caches a dynamic credential is dangerous (the lease keeps ticking while the cache serves it). Cache static KV, not dynamic creds.

7.3 FastAPI — the production integration

The complete, correct pattern

```
"""
app/secrets.py — the only place in your codebase that talks to OpenBao.

Design rules encoded here:
  1. One long-lived OpenBao client, created at startup.
  2. Auth via AppRole; the token is renewed in a background task.
  3. Secrets are cached with a TTL and refreshed in the background.
  4. NOTHING is logged. No secret is ever in an exception message.
  5. Fail closed: if OpenBao is unreachable at boot, the app does not start.
  6. Dynamic credentials are fetched on demand and renewed, never cached long.
"""
from __future__ import annotations

import asyncio
import contextlib
import logging
import os
import time
from typing import Any
```

```
import hvac

log = logging.getLogger(__name__)

BAO_ADDR = os.environ.get("BAO_ADDR", "https://openbao:8200")
BAO_CACERT = os.environ.get("BAO_CACERT")
ROLE_ID = os.environ.get("APPROLE_ROLE_ID")
SECRET_ID = os.environ.get("APPROLE_SECRET_ID")
SECRET_PATH = os.environ.get("SECRET_PATH", "myapp")
MOUNT = os.environ.get("SECRET_MOUNT", "secret")


class SecretManager:
    """Owns the OpenBao client, the token, and the secret cache."""

    def __init__(self) -> None:
        self.client = hvac.Client(url=BAO_ADDR, verify=BAO_CACERT, timeout=30)
        self._cache: dict[str, tuple[float, dict[str, Any]]] = {}
        self._ttl = float(os.environ.get("SECRET_CACHE_TTL", "300"))

    # ── auth ──────────────────────────────────────────────────────────
    def login(self) -> None:
        resp = self.client.auth.approle.login(role_id=ROLE_ID, secret_id=SECRET_ID)
        self.client.token = resp["auth"]["client_token"]
        log.info("authenticated to OpenBao")

    def _renew(self) -> None:
        try:
            resp = self.client.auth.token.renew_self(
                token=self.client.token, increment="20m"
            )
            self.client.token = resp["auth"]["client_token"]
        except Exception:
            log.warning("token renewal failed; re-authenticating")
            self.login()

    async def keepalive(self) -> None:
        """Background task: renew the token and refresh the cache."""
        while True:
            await asyncio.sleep(240)
            try:
                await asyncio.to_thread(self._renew)
            except Exception as exc:
                # Never log the exception's payload; it can contain a token.
                log.error("openbao keepalive error: %s", type(exc).__name__)

    # ── secret access ─────────────────────────────────────────────────
    def _read(self, path: str) -> dict[str, Any]:
        resp = self.client.secrets.kv.v2.read_secret_version(
            path=path, mount_point=MOUNT
        )
        return resp["data"]["data"]

    def get(self, path: str) -> dict[str, Any]:
        """Read a KV v2 secret with a short in-process cache."""
        now = time.monotonic()
```

```
        hit = self._cache.get(path)
        if hit and hit[0] > now:
            return hit[1]
        data = self._read(path)
        self._cache[path] = (now + self._ttl, data)
        return data

    def get_db_credentials(self) -> dict[str, str]:
        """Fetch DYNAMIC database credentials.

        These are deliberately NOT cached beyond their lease: the whole point
        is that the credential dies. We track the lease so we can renew it.
        """
        resp = self.client.secrets.database.read_creds(
            name=os.environ.get("DB_ROLE", "app"), mount_point="db"
        )
        return {
            "username": resp["data"]["username"],
            "password": resp["data"]["password"],
            "lease_id": resp["data"]["lease_id"],
            "lease_duration": resp["data"]["lease_duration"],
        }

    def close(self) -> None:
        with contextlib.suppress(Exception):
            self.client.revoke(self.client.token)


secrets = SecretManager()
```

app/main.py:

```
from contextlib import asynccontextmanager

from fastapi import FastAPI

from .secrets import secrets


@asynccontextmanager
async def lifespan(app: FastAPI):
    # Fail closed: if we cannot authenticate, do not serve traffic.
    try:
        secrets.login()
    except Exception as exc:
        raise RuntimeError(
            f"could not authenticate to OpenBao: {type(exc).__name__}"
        ) from exc

    task = asyncio.create_task(secrets.keepalive())
    try:
        yield
    finally:
        task.cancel()
        secrets.close()
```

```
app = FastAPI(title="AI Harness", lifespan=lifespan)


@app.get("/health")
def health() -> dict[str, str]:
    return {"status": "ok"}


@app.get("/config-check")
def config_check() -> dict[str, object]:
    """Prove the app can read its config — without leaking it."""
    s = secrets.get("myapp")
    return {
        "db_host": s.get("db_host"),
        "llm_key_present": bool(s.get("llm_api_key")),
        "redis_host": s.get("redis_host"),
    }
```

Testing OpenBao-dependent code

Use a **fake** in tests. Never point your test suite at production OpenBao.

```
# tests/conftest.py
import pytest
from app import secrets as secrets_module


@pytest.fixture
def fake_secrets(monkeypatch):
    monkeypatch.setattr(
        secrets_module.secrets,
        "get",
        lambda path: {"db_host": "localhost", "llm_api_key": "sk-test"},
    )
    monkeypatch.setattr(
        secrets_module.secrets,
        "get_db_credentials",
        lambda: {"username": "u", "password": "p"},
    )
```

7.4 LangChain — sourcing API keys safely

The reality

LangChain reads configuration from **environment variables** (`OPENAI_API_KEY`, `ANTHROPIC_API_KEY`, `LANGCHAIN_API_KEY`, `TAVILY_API_KEY`, …). It does not know about OpenBao. So the integration is:

`OpenBao KV v2 → (at process start) → os.environ → LangChain`

The pattern

```
# app/bootstrap.py
"""Populate os.environ from OpenBao, before anything imports LangChain.
```

```
Order matters: this MUST run before `from langchain...` imports in your app,
because many LangChain integrations read env vars at import time.
"""
import os

from .secrets import secrets


def load_environment() -> None:
    cfg = secrets.get("myapp/llm")

    mapping = {
        "OPENAI_API_KEY": cfg.get("openai_api_key"),
        "ANTHROPIC_API_KEY": cfg.get("anthropic_api_key"),
        "LANGCHAIN_API_KEY": cfg.get("langsmith_api_key"),
        "LANGCHAIN_TRACING_V2": "true" if cfg.get("langsmith_enabled") else "false",
        "LANGCHAIN_PROJECT": cfg.get("langsmith_project", "ai-harness"),
        "TAVILY_API_KEY": cfg.get("tavily_api_key"),
    }
    for k, v in mapping.items():
        if v:
            os.environ[k] = v
```

```
# app/main.py
import uvicorn

from .bootstrap import load_environment

load_environment()          # ← BEFORE any langchain import

from .api import app         # noqa: E402  (deliberate late import)

if __name__ == "__main__":
    uvicorn.run(app, host="0.0.0.0", port=8000)
```

> ⚠ **The import-order trap is real.** If any module does `from langchain_openai import ChatOpenAI` at module scope, and that module is imported before `load_environment()`, the key may be None or cached incorrectly. Use `load_environment()` in the container's entrypoint script instead — that's even more robust.

The container entrypoint (most robust)

```
#!/usr/bin/env bash
set -euo pipefail

# 1. Authenticate
export BAO_TOKEN=$(bao write -format=json auth/approle/login \
    role_id="$APPROLE_ROLE_ID" secret_id="$APPROLE_SECRET_ID" | jq -r .auth.client_token)

# 2. Export secrets as env vars for LangChain
export OPENAI_API_KEY=$(bao kv get -mount=secret -field=openai_api_key myapp/llm)
export ANTHROPIC_API_KEY=$(bao kv get -mount=secret -field=anthropic_api_key myapp/llm)
export LANGCHAIN_API_KEY=$(bao kv get -mount=secret -field=langsmith_api_key myapp/llm)

# 3. Start
exec uvicorn app.main:app --host 0.0.0.0 --port 8000
```

A `SecretProvider` for LangChain callbacks

For things you do want to look up at runtime (RAG corpora, model configs):

```
from typing import Any
from langchain_core.callbacks import BaseCallbackHandler


class OpenBaoSecretHandler(BaseCallbackHandler):
    """Resolve secrets lazily, from OpenBao, inside a LangChain callback."""

    def __init__(self, path: str = "myapp/llm") -> None:
        self._path = path

    def get(self, key: str, default: Any = None) -> Any:
        return secrets.get(self._path).get(key, default)


handler = OpenBaoSecretHandler()
handler.get("openai_api_key")
```

What NOT to do

```
# ✗ Never
os.environ["OPENAI_API_KEY"] = "sk-hardcoded"
# ✗ Never
langchain_api_key=os.environ.get("OPENAI_API_KEY")  # in a module-level constant
# ✗ Never
client = ChatOpenAI(openai_api_key=secrets["openai_api_key"])  # in a module-level global
```

Module-level globals capture the value at import time, which makes rotation impossible and makes the secret immortal in the process.

7.5 PostgreSQL — dynamic credentials end to end

7.5.1 One-time Postgres setup

```
-- As a superuser, ONCE
CREATE ROLE openbao_admin WITH LOGIN PASSWORD 'strong-password' CREATEROLE;
GRANT CONNECT ON DATABASE appdb TO openbao_admin;
```

7.5.2 OpenBao config

```
bao secrets enable -path=db database

bao write db/config/postgres \
    plugin_name=postgresql-database-plugin \
    connection_url="postgresql://{{username}}:{{password}}@postgres:5432/appdb?sslmode=requir
    username="openbao_admin" \
    password="strong-password" \
    verify_connection=true \
```

```
    allowed_roles="app,readonly,migrate"

bao write db/roles/app \
    db_name=postgres \
    creation_statements="CREATE ROLE \"{{name}}\" WITH LOGIN PASSWORD '{{password}}' VALID UN
    revocation_statements="DROP ROLE IF EXISTS \"{{name}}\";" \
    default_ttl="1h" \
    max_ttl="4h" \
    renew_statements="ALTER ROLE \"{{name}}\" VALID UNTIL '{{expiration}}';" \
    username_template="v-token-{{identity.entity.id}}-{{random 6}}-{{timestamp}}"

bao write db/roles/migrate \
    db_name=postgres \
    creation_statements="CREATE ROLE \"{{name}}\" WITH LOGIN PASSWORD '{{password}}' VALID UN
    revocation_statements="DROP ROLE IF EXISTS \"{{name}}\";" \
    default_ttl="15m" \
    max_ttl="30m"
```

7.5.3 Policy — the app can only read its own creds

```
bao policy write app-db - <<'EOF'
path "database/creds/app" {
  capabilities = ["read"]
}
EOF
```

7.5.4 Use it from FastAPI

```
import psycopg2
from psycopg2.pool import ThreadedConnectionPool


class RotatingPool:
    """A connection pool that recycles when the dynamic credential expires."""

    def __init__(self) -> None:
        self._pool: ThreadedConnectionPool | None = None
        self._expires_at = 0.0

    def get_pool(self) -> ThreadedConnectionPool:
        now = time.monotonic()
        if self._pool is None or now >= self._expires_at:
            creds = secrets.get_db_credentials()
            self._pool = ThreadedConnectionPool(
                minconn=1,
                maxconn=10,
                host=os.environ.get("DB_HOST", "postgres"),
                port=int(os.environ.get("DB_PORT", "5432")),
                dbname="appdb",
                user=creds["username"],
                password=creds["password"],
            )
            # Recycle a bit before the lease actually dies
            self._expires_at = now + max(30, creds["lease_duration"] * 0.7)
```

```
        return self._pool
```

> ※ **The subtle part:** the Postgres role has `VALID UNTIL` = lease expiry. Your connection pool might hold connections created before expiry; those stay alive (Postgres doesn't kill established sessions), but **new** connections will fail after expiry. So the pool must be recycled on a schedule tied to the lease. The 0.7 factor above is deliberate.

7.5.5 Verify revocation works

```
-- In Postgres
SELECT rolname, rolvaliduntil FROM pg_roles WHERE rolname LIKE 'v-token-%';
```

Revoke from OpenBao and watch the role disappear:

```
bao lease revoke database/creds/app/...
```

7.6 Redis — the honest version

As established in §4.9: **there is no Redis dynamic-credentials plugin in OpenBao 2.7.1.** Here is what to actually do.

Approach A — static credential in KV v2 (recommended for most cases)

```
bao kv put -mount=secret myapp/redis \
    host=redis \
    port=6379 \
    username=app \
    password='redis-app-password' \
    tls=true \
    db=0
```

```
bao policy write redis-reader - <<'EOF'
path "secret/data/myapp/redis" {
  capabilities = ["read"]
}
EOF
```

Approach B — Redis ACL users with scripted rotation (better)

Redis 6+ has **ACL users**. You can create one per service with narrow permissions and rotate the password on a schedule.

```
#!/usr/bin/env bash
# rotate-redis-acl.sh — run from a controlled environment (CI or a cron sidecar)
set -euo pipefail

SERVICE="${1:?service name}"
KEYS="${2:?key pattern, e.g. app:*}"

# Admin password comes from OpenBao
ADMIN_PW=$(bao kv get -mount=secret -field=password myapp/redis/admin)
NEW_PW=$(openssl rand -base64 32 | tr -d '/+=' | head -c 32)

redis-cli -u "redis://:${ADMIN_PW}@redis:6379" --no-auth-warning ACL SETUSER "$SERVICE" \
```

```
  on ">$KEYS" "+@read" "+@write" "+ping" "+get" "+set" ">${NEW_PW}" "+@all" -@all

# Store the new password for the service to pick up
bao kv put -mount=secret "myapp/redis/${SERVICE}" \
    host=redis port=6379 username="$SERVICE" password="$NEW_PW" tls=true

# Reload the ACL file so it survives a Redis restart
redis-cli -u "redis://:${ADMIN_PW}@redis:6379" --no-auth-warning ACL SAVE
```

> ⚠ **Be careful with the ACL command above** — the ordering of `+@all -@all` matters. The safe sequence is: reset the user, then grant only what you need. Test it in a lab.

Approach C — the Valkey plugin (if you can switch)

OpenBao ships `valkey-database-plugin`. If you can run **Valkey** (the Linux Foundation's Redis fork, drop-in compatible), you get real dynamic credentials:

```
bao write db/config/valkey \
    plugin_name=valkey-database-plugin \
    connection_url="redis://{{username}}:{{password}}@valkey:6379" \
    username="openbao_admin" \
    password="..." \
    allowed_roles="app"
```

Recommendation for your stack

```
Postgres  → real dynamic credentials (postgresql-database-plugin)     ✓
Redis     → static KV v2 + ACL-user rotation script                   ⚠
```

Redis is usually network-isolated and lower-value than a database credential. Don't over-engineer it. But **do** give each service its own Redis ACL user with scoped key patterns, so a compromised service can't read another's keys.

7.7 GitHub Actions → OpenBao (OIDC, no stored secrets)

This is the single biggest win available to your CI: **zero long-lived secrets in GitHub**.

7.7.1 The flow

```
  GitHub Actions (OIDC identity token)
        │  JWT signed by GitHub, aud="openbao"
        ▼
  OpenBao `jwt` auth method
        │  validates signature via GitHub's JWKS
        │  checks bound_audiences + bound_claims
        ▼
  OpenBao token with CI policies
```

7.7.2 Configure the OpenBao side

```
bao auth enable -path=github-actions jwt

bao write auth/github-actions/config - <<'EOF'
{
```

```
  "oidc_discovery_url": "https://token.actions.githubusercontent.com",
  "bound_issuer": "https://token.actions.githubusercontent.com",
  "default_role": "github"
}
EOF

bao write auth/github-actions/role/github - <<'EOF'
{
  "role_type": "jwt",
  "user_claim": "actor",
  "bound_audiences": ["openbao"],
  "bound_claims": {
    "repository": "your-org/your-repo",
    "ref": "refs/heads/main"
  },
  "token_policies": ["ci-deploy"],
  "token_ttl": "15m",
  "token_max_ttl": "30m"
}
EOF
```

[ALERT] **Three corrections:**

1. The mount config is at **`auth/<mount>/config`**; the role is at **`auth/<mount>/role/<name>`**. There is no `auth/<mount>/jwt/<name>`.
2. `oidc_discovery_url` is the **base URL** — no `.well-known` suffix (OpenBao appends it). The literal `/.well-known/jwks` form only applies to `jwks_url`.
3. `bound_claims` is a **map**, so it needs a JSON write (or `-` heredoc) — you cannot set it with repeated `key=value` flags.

The `oidc_discovery_url` form pulls the JWKS automatically. You can also set `jwks_url="https://token.actions.githubusercontent.com/.well-known/jwks"` explicitly (they are mutually exclusive).

Tightening `bound_claims`:

```
# Restrict to a specific repo AND branch AND workflow
bao write auth/github-actions/role/github - <<'EOF'
{
  "role_type": "jwt",
  "user_claim": "actor",
  "bound_audiences": ["openbao"],
  "bound_claims": {
    "repository": "your-org/your-repo",
    "ref": "refs/heads/main",
    "workflow": "deploy.yml",
    "repository_owner": "your-org"
  }
}
EOF
```

※ `bound_claims` is a **map**, so it needs a JSON write (or a `-` heredoc). You cannot set it with repeated `key=value` flags.

[SEC] **`bound_audiences` must match the `aud` you request from GitHub.** If they don't, the token is rejected. This is the #1 cause of "401 unauthorized" in this setup.

7.7.3 The CI policy

```
bao policy write ci-deploy - <<'EOF'
# CI can read the config it needs to deploy
path "secret/data/myapp" {
  capabilities = ["read"]
}
path "secret/data/myapp/config" {
  capabilities = ["read"]
}
# CI can write the versioned app config it owns
path "secret/data/myapp/deploy" {
  capabilities = ["create", "read", "update", "patch"]
}
path "secret/metadata/myapp/deploy" {
  capabilities = ["read", "list"]
}
# CI needs to see what it's deploying
path "sys/health" {
  capabilities = ["read"]
}
EOF
```

> ⚠ **Do NOT give CI `list` on `secret/data/*` or `secret/metadata/*`.** The whole point of CI having a token is that it can read specific things. `list` on a KV mount reveals every secret path in it.

7.7.4 The GitHub Actions workflow

```
# .github/workflows/deploy.yml
name: Deploy

on:
  push:
    branches: [main]
  workflow_dispatch:

permissions:
  contents: read
  id-token: write        # ← REQUIRED to get an OIDC token

jobs:
  deploy:
    runs-on: ubuntu-latest
    environment: production   # ← set "Require approval" in GitHub settings

    steps:
      - name: Get OpenBao token from OIDC
        id: bao
        env:
          BAO_ADDR: https://openbao.corp.internal:8200
          BAO_CACERT: ${{ secrets.BAO_CACERT }}
        run: |
          # Request the GitHub OIDC token
          OIDC_TOKEN=$(curl -sS -H "Authorization: bearer $ACTIONS_ID_TOKEN_REQUEST_URL" \
                            "$ACTIONS_ID_TOKEN_REQUEST_URL&audience=openbao" | jq -r .value)
```

```
      # Exchange it for an OpenBao token
      BAO_TOKEN=$(curl -sS --cacert "$BAO_CACERT" \
        -H "X-Vault-Request: true" \
        -H "Content-Type: application/json" \
        -d "{\"role\":\"github\",\"jwt\":\"$OIDC_TOKEN\"}" \
        "$BAO_ADDR/v1/auth/github-actions/login" | jq -r .auth.client_token)

      echo "::add-mask::$BAO_TOKEN"
      echo "token=$BAO_TOKEN" >> "$GITHUB_OUTPUT"

  - name: Read app config from OpenBao
    env:
      BAO_TOKEN: ${{ steps.bao.outputs.token }}
    run: |
      # Write to a file, not an env var in the job log
      curl -sS --cacert "$BAO_CACERT" \
        -H "X-Vault-Token: $BAO_TOKEN" \
        "$BAO_ADDR/v1/secret/data/myapp" > app-config.json
      echo "Config fetched (values not printed)"

  - name: Deploy
    run: ./deploy.sh
```

[KEY] **The aud must match bound_audiences.** In the workflow above, audience=openbao; in OpenBao, bound_audiences="openbao". Change one, change both.

※ **environment: production** with "required reviewers" gives you a human approval gate on the deploy — a poor-man's but effective control group, and it also means the OIDC token's environment claim is set.

7.7.3b Using bao CLI in CI

```
  - name: Install bao CLI
    run: |
      wget -qO- https://github.com/openbao/openbao/releases/download/v2.7.1/openbao_2.7.1
        | funzip > /usr/local/bin/bao
      chmod +x /usr/local/bin/bao

  - name: Fetch secrets
    env:
      BAO_ADDR: https://openbao.corp.internal:8200
      BAO_CACERT: ${{ secrets.BAO_CACERT }}
      OIDC_TOKEN: ${{ steps.oidc.outputs.token }}
    run: |
      export BAO_TOKEN=$(curl -sS --cacert "$BAO_CACERT" \
        -H "Content-Type: application/json" \
        -d "{\"role\":\"github\",\"jwt\":\"$OIDC_TOKEN\"}" \
        "$BAO_ADDR/v1/auth/github-actions/login" | jq -r .auth.client_token)
      echo "::add-mask::$BAO_TOKEN"
      bao kv get -mount=secret -field=api_key myapp/deploy > /tmp/deploy-key
```

※ BAO_CACERT is a GitHub secret, but it's a **public CA certificate** — not a credential. It's fine (and correct) to store it as a secret so you can rotate it, but it provides no security. The only thing protecting your secrets here is the OIDC token's short lifetime and its bound claims.

7.7.4 The alternative — OpenBao as the OIDC provider

You can flip it: make OpenBao the IdP and have GitHub Actions log in to it.

```
bao write sys/oidc/providers/default \
    client_type="public" \
    allowed_redirect_uris="https://github.com/org/repo" \
    allowed_scopes="openid profile email" \
    default_scopes="openid profile email" \
    issuer="https://openbao.corp.internal:8200"
```

Then create a client for GitHub and configure the **OIDC** auth method in OpenBao to trust OpenBao's own provider.

> ⚠ **2.6.0 added a Client Credentials flow to the OIDC Provider** — so OpenBao can now issue tokens for machine-to-machine flows directly. And 2.7.0 removed `prompt=none` redirection support in the OIDC provider (GHSA-2cjw-94fw-wqjx) — a security fix. Don't build flows that rely on it.

Which direction should you use?

	GitHub → OpenBao (`jwt` auth)	OpenBao → GitHub (OIDC provider)
Simplicity	✓ Simple, well-documented	More moving parts
GitHub native	✓ Uses the built-in `id-token: write`	Needs custom setup
Revocation	GitHub's OIDC tokens are short-lived	OpenBao's session model
Use when	CI needs OpenBao secrets	You want one IdP for everything

Recommendation: GitHub → OpenBao. It's simpler, and GitHub's OIDC tokens are already short-lived and audience-scoped.

7.8 The complete stack in one Compose file

```
services:
  # ———————— OpenBao ————————
  openbao:
    image: openbao/openbao:2.7.1
    container_name: openbao
    command: server -config=/openbao/config/openbao.hcl
    ports: ["127.0.0.1:8200:8200"]
    volumes:
      - ./config:/openbao/config:ro
      - ./certs:/openbao/certs:ro
      - openbao-data:/openbao/data
      - ./logs:/openbao/logs
    memswap_limit: 0
    security_opt: ["no-new-privileges:true"]
    healthcheck:
      test: ["CMD-SHELL", "BAO_CACERT=/openbao/certs/ca.crt bao status >/dev/null 2>&1"]
      interval: 10s
      timeout: 5s
      retries: 12
    networks: [backend]
    restart: unless-stopped
```

```yaml
# ──────────── Keycloak ────────────
keycloak:
  image: quay.io/keycloak/keycloak:26.0
  container_name: keycloak
  command: ["start-dev", "--import-realm"]
  environment:
    KC_BOOTSTRAP_ADMIN_USERNAME: ${KC_ADMIN:-admin}
    KC_BOOTSTRAP_ADMIN_PASSWORD: ${KC_ADMIN_PASSWORD:?set KC_ADMIN_PASSWORD}
    KC_HEALTH_ENABLED: "true"
  volumes:
    - ./keycloak/realm.json:/opt/keycloak/data/import/realm.json:ro
  ports: ["127.0.0.1:8080:8080"]
  healthcheck:
    test: ["CMD-SHELL", "exec 3<>/dev/tcp/127.0.0.1/8080 && echo -e 'GET /health/ready HTTP
    interval: 10s
    timeout: 5s
    retries: 20
  networks: [backend]

# ──────────── Postgres ────────────
postgres:
  image: postgres:16-alpine
  container_name: postgres
  environment:
    POSTGRES_USER: postgres
    POSTGRES_PASSWORD: ${POSTGRES_PASSWORD:?set POSTGRES_PASSWORD}
    POSTGRES_DB: appdb
  volumes:
    - pgdata:/var/lib/postgresql/data
  healthcheck:
    test: ["CMD-SHELL", "pg_isready -U postgres"]
    interval: 5s
    timeout: 3s
    retries: 10
  networks: [backend]

# ──────────── Redis ────────────
redis:
  image: redis:7-alpine
  container_name: redis
  command: ["redis-server", "--requirepass", "${REDIS_PASSWORD:?set REDIS_PASSWORD}", "--ap
  volumes: [redisdata:/data]
  healthcheck:
    test: ["CMD", "redis-cli", "-a", "${REDIS_PASSWORD}", "ping"]
    interval: 5s
    timeout: 3s
    retries: 10
  networks: [backend]

# ──────────── The app ────────────
app:
  build: ./app
  container_name: harness-app
  environment:
    BAO_ADDR: https://openbao:8200
    BAO_CACERT: /certs/ca.crt
```

```
      APPROLE_ROLE_ID: ${APPROLE_ROLE_ID}
      APPROLE_SECRET_ID: ${APPROLE_SECRET_ID}
      SECRET_PATH: myapp
      DB_HOST: postgres
      REDIS_HOST: redis
    volumes:
      - ./certs:/certs:ro
    depends_on:
      openbao:   { condition: service_healthy }
      postgres:  { condition: service_healthy }
      redis:     { condition: service_healthy }
      keycloak:  { condition: service_healthy }
    networks: [backend]

networks:
  backend:
    driver: bridge

volumes:
  openbao-data:
  pgdata:
  redisdata:
```

The bootstrap script (runs once after the stack is up)

```
#!/usr/bin/env bash
# bootstrap.sh — idempotent initial configuration
set -euo pipefail

export BAO_ADDR=https://127.0.0.1:8200
export BAO_CACERT=./certs/ca.crt

# ── audit first ──
# NOTE: 2.7.1 — audit devices are declared in the server config:
#   audit "file" "file" {
#     description = "Local audit log (declarative)."
#     options { file_path = "/openbao/logs/audit.log" mode = "0600" log_raw = "false" }
#   }
bao audit list -detailed

# ── secret engines ──
bao secrets enable -path=secret kv-v2 2>/dev/null || true
bao secrets enable -path=db database 2>/dev/null || true
bao secrets enable -path=transit transit 2>/dev/null || true

# ── auth methods ──
bao auth enable -path=oidc oidc 2>/dev/null || true
bao auth enable -path=approle approle 2>/dev/null || true

# ── policies ──
bao policy write app-readonly - <<'EOF'
path "secret/data/myapp"        { capabilities = ["read"] }
path "secret/metadata/myapp"    { capabilities = ["read", "list"] }
path "database/creds/app"       { capabilities = ["read"] }
path "transit/encrypt/myapp"     { capabilities = ["update"] }
```

```
path "transit/decrypt/myapp"     { capabilities = ["update"] }
EOF

# ── approle for the app ──
bao write auth/approle/role/myapp \
    secret_id_ttl=10m secret_id_num_uses=1 \
    token_ttl=20m token_max_ttl=1h 2>/dev/null || true

# ── oidc for humans ──
#    NOTE: config path is auth/<mount>/config; oidc_discovery_url is the
#    BASE url (no .well-known); the client keys are oidc_client_id/_secret.
bao write auth/oidc/config \
    oidc_discovery_url="http://keycloak:8080/realms/enterprise" \
    oidc_client_id="openbao" \
    oidc_client_secret="$KEYCLOAK_CLIENT_SECRET" \
    default_role="human" 2>/dev/null || true

echo "bootstrap complete"
```

7.9 The anti-patterns

✗ Anti-pattern	Why it's bad	✓ Do this
One shared "app" credential for all services	Can't attribute, can't revoke one	One role per service
`path "secret/*" { capabilities = ["*"] }`	Any app can read any secret	Per-service paths
Storing secrets in `.env` in git	Permanent exposure	OpenBao + AppRole
Root token in CI	Full compromise on CI breach	GitHub OIDC
`verify=False` in the TLS client	MITM	`BAO_CACERT`
Long TTL dynamic credentials (24h+)	Defeats the whole point	1h default, 4h max
Logging the whole config at startup	Leaks into log aggregators	Log keys present, not valu
One long-lived OpenBao token for the app	Can't rotate without downtime	Short TTL + renewal loop
`bao login` in a shared CI runner	Token cached on disk	Per-job OIDC login
Catching and printing OpenBao exceptions	Exceptions can contain tokens	Log the exception type on

---

7.10 Level 7 checkpoint

1. What must match between Keycloak's issuer and OpenBao's `oidc_discovery_url`?
2. Why does Keycloak need a groups mapper configured on the client?
3. `role_id` vs `secret_id` in AppRole: which goes in the image, which goes in the deploy pipeline? Why?
4. Why is `bao agent` better than an entrypoint script for containers?
5. Why must `load_environment()` run before LangChain imports?
6. What is the danger of caching a dynamic credential in `bao proxy`?
7. Your Postgres pool holds connections past the lease expiry. What breaks and how do you fix it?
8. Is there a Redis dynamic-credentials plugin in 2.7.1? What do you do?
9. In the GitHub Actions flow, what must match between `audience=` and OpenBao's `bound_audiences`?
10. What should a CI policy not be able to do, and why?

Next → **Level 8: EXPERT.md** — namespaces, the plugin system, workflows, External Keys, post-quantum crypto, CEL roles and control groups.

7.11 Scratchpad

```
Keycloak discovery:  http://keycloak:8080/realms/<realm>              (BASE url!)
Keycloak  JWKS:                http://keycloak:8080/realms/<realm>/protocol/openid-
connect/certs
OpenBao OIDC config: bao write auth/oidc/config oidc_discovery_url=... oidc_client_id=... oidc_
OIDC role:        bao write auth/oidc/role/human user_claim=preferred_username groups_claim=gro
GitHub Actions:    bao write auth/github-actions/config oidc_discovery_url=https://token.actio
              bao write auth/github-actions/role/github bound_audiences=["openbao"] bound_clai
DB creds:             bao read db/creds/app
Transit:              bao write transit/encrypt/myapp plaintext=<base64>
LangChain env:        OPENAI_API_KEY / ANTHROPIC_API_KEY / LANGCHAIN_API_KEY
```

Level 8 — Expert: The Advanced Surface

> **Goal:** the features that separate a competent OpenBao operator from an expert. Namespaces at scale, the plugin system, workflows, External Keys, post-quantum crypto, CEL validation programs, control groups, and the password-policy subsystem.

8.1 Namespaces at scale

What a namespace really is

A namespace is a **mini-OpenBao** living inside one instance:

```
root/                          ← the cluster's control plane
├── auth/oidc/                 ← IdP config
├── identity/                  ← entities, groups
├── secret/                    ← shared platform secrets
├── policy/                    ← shared policies
│
├── ns-platform/               ← a tenant
│   ├── secret/                ← its own KV
│   ├── auth/approle/          ← its own auth methods
│   ├── identity/              ← its own entities
│   └── policy/                ← its own policies
│
└── ns-payments/               ← another tenant
    ├── secret/
    └── ...
```

Each namespace has its **own login paths**. A user logging into `ns-platform/auth/oidc` is entirely separate from `ns-payments/auth/oidc`.

Creating and working with namespaces

```
# Create
bao namespace create platform-team
bao namespace list

# Work inside one
export BAO_NAMESPACE=platform-team
bao secrets enable kv-v2
bao policy write my-policy - <<'EOF'
```

```
path "secret/data/*" { capabilities = ["read"] }
EOF

# Or per-command
bao secrets list -namespace=platform-team
bao -namespace=platform-team kv get secret/myapp
```

The critical difference from Vault: namespaces are free

In HashiCorp Vault, namespaces are an **Enterprise** feature with client-count metering. In OpenBao, they're **free** — and that removes the usual objection to multi-tenancy (the "client count" fear doesn't apply).

The delegation model

```
# Create a namespace as the platform team
bao namespace create team-a

# Give them an admin policy *inside* their namespace
bao policy write ns-admin - <<'EOF'
path "sys/mounts/*"    { capabilities = ["create", "read", "update", "delete", "list"] }
path "sys/auth/*"      { capabilities = ["create", "read", "update", "delete", "list"] }
path "sys/policies/acl/*" { capabilities = ["create", "read", "update", "delete", "list"] }
path "identity/*"      { capabilities = ["create", "read", "update", "delete", "list"] }
path "secret/*"        { capabilities = ["create", "read", "update", "delete", "list", "scan"
EOF

# Assign it to a group in the parent namespace
bao write identity/group name=team-a-admins policies=ns-admin
```

Now the team admins can manage their own namespace without being root-namespace admins.

Child namespaces and inheritance

```
ns-platform/
└── ns-platform/subsidiary/
```

A child namespace **inherits** policies and identities from its parent. You can also let a parent "assert" policies on identities in a child.

Namespace sealing (2.6.0)

```
# Create a namespace with its own Shamir seal
bao namespace create -seal-shares=5 -seal-threshold=3 team-sensitive
```

This gives the namespace **its own barrier keyring**. A tenant can then seal their own namespace, revoking the operator's access to their data **without touching any other tenant**. Seal status is synchronised globally, so it's visible from any node.

```
bao namespace seal-status team-sensitive
bao namespace seal team-sensitive
```

> ※ This is a genuinely differentiated feature vs Vault Community, and it's the kind of thing that matters if you're offering OpenBao as an internal service to other teams.

Cross-namespace identity — leave it off

```
unsafe_cross_namespace_identity = false     # ← the default, keep it
```

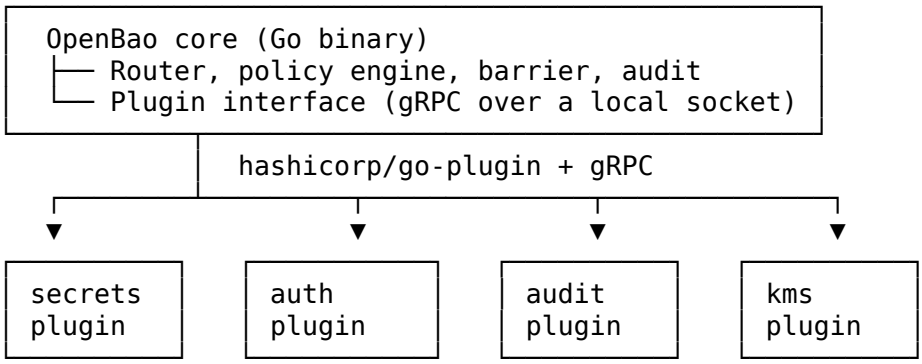
The docs are explicit: it "gives compatibility with Vault Enterprise's cross-namespace group membership and **should not be enabled**" because "there are no mitigating controls on what identities or groups another namespace can reference and allows for unconfined permission sharing."

Namespace quotas

```
bao namespace quota \
    rate_limit=200 \
    path="" \
    namespace="team-a"
```

8.2 The plugin system

Architecture



Plugin types: `secrets`, `auth`, `audit`, `kms`, `database`.

What became a plugin (and why it matters to you)

Plugin	Was	Now	Why
`kms-pkcs11`	Built-in pkcs11 seal	Plugin (2.7.0)	HSM
`kms-aws`	Built-in awskms seal	Plugin (2.7.0)	Same
`kms-gcp`, `kms-azure`, `kms-alicloud`, `kms-oci`	Built-in	Plugin (2.7.0)	Same
`kms-ovhcloud`, `kms-tcloudpublic`, `kms-securosys`	Built-in	Plugin (2.7.0)	Same
`auth-ldap`, `auth-kerberos`, `auth-radius`	Built-in	Plugin (2.7.0)	Lega
`auth-github`, `auth-aws`	—	Plugin	Cloud
`secrets-ldap`	Built-in	Plugin (2.7.0)	Lega
`secrets-aws`, `secrets-gcp`, `secrets-consul`, `secrets-nomad`	—	Plugin	Cloud
`database-mongodb`	Built-in in Vault (SSPL)	Plugin	Mong
`secrets-oauthapp`	—	Separate repo	OAut

The verified plugin registry (as of October 2026)

First-party plugins are published as **GitHub releases** in `https://github.com/openbao/openbao-plugins` and as **OCI images** on `ghcr.io/openbao/`. The image name is `ghcr.io/openbao/openbao-plugin-<name>:<tag>`, and the tag matches the release tag. Verified live:

OCI image	Available tags
ghcr.io/openbao/openbao-plugin-kms-pkcs11	v0.1.0, v0.2.0, **v0.2.1**
ghcr.io/openbao/openbao-plugin-kms-aws	v0.1.0
ghcr.io/openbao/openbao-plugin-kms-azure	v0.1.0
ghcr.io/openbao/openbao-plugin-secrets-aws	v0.0.1 … **v0.3.1**
ghcr.io/openbao/openbao-plugin-secrets-gcp	… v0.22.0, v0.23.0, **v0.23.1**
ghcr.io/openbao/openbao-plugin-auth-ldap	v0.1.0
ghcr.io/openbao/openbao-plugin-auth-github	v0.0.1

※ There are also two separate plugin repos outside `openbao-plugins`: `openbao/openbao-plugin-auth-jwt` and `openbao/openbao-plugin-secrets-oauthapp`.

Pin by digest, not by tag:

```
# Get the digest for a tag
TOKEN=$(curl -s "https://ghcr.io/token?scope=repository:openbao/openbao-plugin-kms-pkcs11:pul
curl -s -H "Authorization: Bearer $TOKEN" \
  -H "Accept: application/vnd.oci.image.index.v1+json" \
  "https://ghcr.io/v2/openbao/openbao-plugin-kms-pkcs11/manifests/v0.2.1" \
  | jq -r '.manifests[0].digest'
```

[SEC] **The upgrade trap:** if your config references a plugin that is no longer built in, **install and register the plugin BEFORE upgrading the binary**. Otherwise the new node starts and cannot unseal.

Installing plugins

Option 1 — declarative, from an OCI registry (2.5.0+):

```
plugin_directory = "/openbao/plugins"

plugin_auto_download     = true
plugin_auto_register     = true
plugin_download_behavior = "fail"

plugin "kms" "pkcs11" {
  # Pin by digest (2.7.0) — never by a moving tag
  image = "ghcr.io/openbao/openbao-plugin-kms-pkcs11@sha256:abc123..."
}

plugin "secrets" "aws" {
  image = "ghcr.io/openbao/openbao-plugin-secrets-aws:v0.3.1"
  # binary_name auto-detected from ENTRYPOINT/CMD in 2.7.0
}
```

Option 2 — manual:

```
bao plugin init <sha256> <command> [args...]
bao plugin list
bao plugin info <name>
bao plugin reload <name>
bao plugin prune          # 2.7.0: remove unused OCI downloads
bao plugin deregister <name>
```

2.7.0's plugin security fixes — read these

GHSA	Fix
GHSA-j6wc-jpvg-xfxq	Plugin command name is now resolved **relative to `plugin_directory`** before executi
GHSA-cg72-x35g-xfp8	Writes to `sys/plugins/catalog/*` are restricted to the **root namespace**
—	`plugin_auto_register` now defaults to `true`
—	`sha256sum` no longer required for manually installed plugin binaries
—	OCI plugins can be pinned **by manifest digest**

Using a plugin

```
# Secrets plugin
bao secrets enable -path=aws aws
bao write aws/config/root access_key=... secret_key=...
bao write aws/roles/my-role credential_type=iam_user

# Auth plugin
bao auth enable -path=aws aws
bao write auth/aws/role/my-role credential_type=iam_user

# KMS plugin as a seal
seal "pkcs11" {
  # ... PKCS#11 specific options
}
```

Plugin upgrade without downtime (2.7.0)

> ”core/plugins: Reload KMS plugin configuration stanzas on SIGHUP and respawn plugin clients as needed, enabling plugin upgrades at runtime.”

```
# Replace the plugin binary
cp new-openbao-plugin-kms-pkcs11 /openbao/plugins/

# Reload
kill -HUP $(pgrep bao)
# or
bao plugin reload pkcs11
```

8.3 Workflows (2.6.0)

What they are

Workflows let operators define a **named, multi-request sequence** that users execute in one call. They provide ”a simplified or managed interface over OpenBao’s standard API” and ”a basic (workflow-defined) form of cross-plugin communication”.

Think: a **macro** for API calls.

```
# Manage (privileged)
bao list sys/workflows/manage
bao read sys/workflows/manage/create-namespace
```

Where they’re defined

Workflows are configured **in the server config file** (like self-initialization), not via the API. The API only manages and executes them.

```
# (server config)
workflow "create-namespace" {
  description = "Create a namespace and its initial policy"

  step "create" {
    operation = "update"
    path      = "sys/namespaces/team-{{identity.entity.name}}"
    data = {
      description = "team namespace"
    }
  }

  step "policy" {
    operation = "update"
    path      = "sys/policies/acl/team-default"
    data = {
      policy = <<-EOT
path "secret/data/*" { capabilities = ["read"] }
EOT
    }
  }
}
```

Executing

```
bao write sys/workflows/execute/create-namespace
```

Or via the API: `POST /v1/sys/workflows/execute/:path`

The `allow_unauthenticated_workflows` flag

```
allow_unauthenticated_workflows = true
```

This lets the workflow be **dispatched** without authentication — but the requests inside it still require authentication, which can be supplied as a request parameter. Use this for a public "request a certificate" style flow.

> ⚠ **Manage workflows is a privileged operation.** The docs are clear: "a workflow cannot do anything more than the token it is executed as can do… but workflows can result in high server load depending on the endpoints called." Keep workflow definitions in git and review them like code.

When to use workflows

Use case	Workflow?
"Create a namespace + a policy + a mount"	✓ Perfect
"Issue a cert + write the private key to KV"	✓ Great
"Rotate a secret across 3 systems"	✓ Useful
A single API call	✗ Just call the API

8.4 External Keys (2.7.0) — keys in an HSM, not in OpenBao

The problem it solves

In a normal PKI setup, the **intermediate CA's private key lives inside OpenBao**. It's encrypted at rest, but it is in OpenBao. If OpenBao is compromised, the CA key is compromised — and a compromised intermediate CA means you must distrust every cert it ever issued.

External Keys lets the PKI and Transit engines use a key held in an **external KMS or HSM**. The private key never enters OpenBao at all.

> [NOTE] External Keys are OpenBao's equivalent of Vault Enterprise's Managed Keys, re-designed for flexibility. **Not API-compatible** with Managed Keys.

Supported engines

Engine	What it externalises
PKI	Certificate signing via root and intermediate paths, using external private keys
Transit	Sign, verify, encrypt, decrypt with external key material

The API

`/sys/external-keys/...`

```
# List configured external keys
bao list sys/external-keys

# Map a mount to an external key
bao write sys/external-keys/pki_int \
    keys="my-hsm-key" \
    backend="kms" \
    # ... backend-specific configuration
```

Then grant access to the mounts that use them (external keys are **per namespace** — a secrets engine in a namespace can only use external keys configured in that same namespace).

Backends

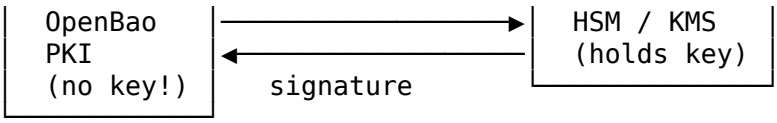
Backend	Source
Transit	Built-in. Similar in concept to the built-in Transit seal — a transit engine in the same or another clu
PKCS#11	`kms-pkcs11` plugin from `openbao-plugins`

```
# PKCS#11-backed external key
plugin "kms" "pkcs11" {
  command = "openbao-plugin-kms-pkcs11"
}

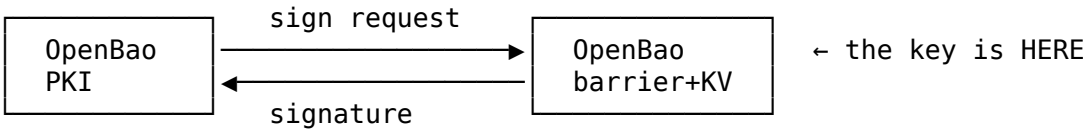
seal "pkcs11" {
  # ...
}
```

The architecture

```
    ┌──────────────┐    sign request    ┌──────────────────┐
```



Compare with a normal (non-external) PKI:



When to use it

Situation	External Keys?
Compliance requires the CA key in an HSM	✓ **Yes**
You want a stolen OpenBao to not mean a stolen CA	✓ **Yes**
You're on a cloud KMS and want one key story	✓ Yes
You're a small team, cost matters	✗ Overkill — the built-in PKI is fine
You want Transit to encrypt with an HSM key	✓ Yes

8.5 Post-quantum cryptography (2.7.0)

OpenBao 2.7.0 is one of the first secrets managers to ship **pure post-quantum ** protections at the TLS layer.

ML-DSA (NIST FIPS 204) in PKI and Transit

```
# PKI role using ML-DSA
bao write pki_int/roles/pqc-app \
    key_type="mldsa-44" \
    signature_bits=0 \
    max_ttl="24h"

# Transit ML-DSA keys
bao write transit/keys/pqc-key \
    type="mldsa-44"          # or mldsa-65, mldsa-87

bao write transit/sign/pqc-key \
    input=$(echo -n "data" | base64)

bao write transit/verify/pqc-key \
    input=$(echo -n "data" | base64) \
    signature="vault:v1:..."
```

Parameter	Meaning
`mldsa-44`	NIST security category 2
`mldsa-65`	NIST security category 3
`mldsa-87`	NIST security category 5

⚠ **The OCSP caveat, straight from the release notes:** "Go's OCSP implementation does not support ML-DSA so will be unusable with ML-DSA typed issuers." If your revocation strategy depends on OCSP, ML-DSA roles won't work with it.

Pure-PQC TLS (2.7.0)

```
listener "tcp" {
  address        = "0.0.0.0:8200"
  tls_cert_file  = "/openbao/certs/openbao.crt"
  tls_key_file   = "/openbao/certs/openbao.key"

  # Force post-quantum key exchange
  tls_key_exchange_preferences = ["X25519MLKEM768", "SecP256r1MLKEM768"]
}
```

Algorithm	Notes
X25519MLKEM768	X25519 + ML-KEM-768. The modern default choice.
SecP256r1MLKEM768	NIST P-256 + ML-KEM-768
SecP384r1MLKEM1024	NIST P-384 + ML-KEM-1024. Strongest.
MLKEM1024	Pure ML-KEM (no classical component)

You can also serve **ML-DSA certificates** from the listener, and OpenBao's **outbound** TLS (to a transit seal cluster, to Postgres, to an OIDC provider) can now connect to servers presenting these algorithms.

Why this matters

The "**harvest now, decrypt later**" threat: an adversary records encrypted traffic today and decrypts it in 10 years when quantum computers can break classical RSA/ECDHE. For a secrets manager whose traffic carries tokens and secrets, that's a real concern for long-lived data.

Practical advice: you don't need PQC everywhere today. But if your organisation has a PQC migration plan, OpenBao 2.7 gives you the server side of it, which is the hard part.

8.6 CEL validation programs

What CEL is

Common Expression Language — a small, fast, sandboxed expression language. OpenBao uses it in **PKI** and **JWT/OIDC** for declarative validation rules and policies.

The structure

```
CelProgram
├── Expression    ← the main expression: accept or reject
└── CelVariable[] ← named sub-expressions, evaluated in order
```

The main expression

It must return the engine's success object, or a `string`/`bool` error:

```
require_cn ? 'request should have a common_name' : (validate_ttl ? 'request has invalid TTL'
```

Use a **ternary** so a value is always produced:

```
cond ? SuccessObject : error string/bool
```

The warning you must internalise

> ⚠ "CEL is an advanced topic for operators. All validation is taken on by the CEL program created by the operator; limited to no validation is performed by OpenBao above and beyond what the CEL program contains. **It is up to the operator to craft CEL programs which adequately perform validation.**"

CEL programs replace OpenBao's built-in checks. If your program is wrong, OpenBao will do the wrong thing. Test extensively. Start by adding to the defaults, not replacing them.

Where it's used

PKI — a `validation` block on a role:

```
bao write pki_int/roles/strict \
    validation_cel_program='
      variable "require_cn" {
        expression = "has(request.common_name) && request.common_name != \"\""
      }
      variable "validate_ttl" {
        expression = "request.ttl <= 86400"
      }
      expression = "require_cn && validate_ttl ? ValidationOutput : \"Request Rejected: inval
    '
```

JWT/OIDC — a `validation_cel_program` on the role, controlling which claims are required and how they're interpreted.

Practical guidance

```
□ Start with the simplest possible program
□ Add one condition at a time
□ Test with both valid and invalid inputs
□ Keep the built-in checks AND your CEL program until you're confident
□ Review CEL programs like code (they're in git, they're reviewed)
```

8.7 Control groups (2.7.0) — the two-person rule

The concept

> "In some deployments, it may be desirable to require a 'second set of eyes' to approve access to a secret or to generate a certificate. Control groups implement second-party authorization by means of policy 'factors' applied to a path."

When a request hits a path with a control-group factor, OpenBao **does not execute it**. It returns a **wrapped response** containing a token and an accessor. A member of the approving group authorizes it. Then the requester unwraps it, and that is when the original request executes.

The workflow

```
1. Policy declares a control_group for a path
2. Requester calls the path
3. OpenBao returns a wrapped response (token + accessor), NOT the secret
4. Requester forwards the accessor to an approver
5. Approver reviews it, authorizes it
6. Requester unwraps → the original request executes → secret released
```

⚠ **OpenBao sends no notification to approvers.** You have to build the notification (a webhook, a queue, a Slack bot) or nobody will ever approve anything.

The policy

```
path "pki/issue/*" {
  capabilities = ["read", "update"]

  control_group = {
    factor "pki-approvers" {
      controlled_capabilities = ["update"]

      identity {
        # Who can approve
        group_names = ["pki-approvers", "security-team"]
        approvals   = 1        # how many must approve

        # How long an authorization stays valid
        ttl = "5d"

        # Allow self-authorization?
        allow_self_authorization = false
      }
    }
  }
}
```

The operations

Via `sys/wrapping` and `sys/control-group`:

Operation	Path	What it does
Authorize	`sys/control-group/authorize`	Approve the request using the accessor
Review	`sys/control-group/request`	View the original request (entity, path, payload) using the acce
Unwrap	`sys/wrapping/unwrap`	Execute the original request (only works once approved)

```
# 1. Requester
bao write pki/issue/myapp common_name="prod.example.com"
# → { "wrap_info": { "token": "hvs...", "accessor": "6f1c..." } }

# 2. Requester sends the accessor to an approver (out of band)

# 3. Approver reviews
bao write sys/control-group/request accessor=6f1c...

# 4. Approver authorizes
bao write sys/control-group/authorize accessor=6f1c...

# 5. Requester unwraps → the cert is issued
bao unwrap hvs...
```

Why this is the strongest anti-insider control in OpenBao

Even a **fully compromised OpenBao admin token** can't unilaterally read a protected path. They need a second identity with a separate group membership. That's a genuine separation of duties, enforced cryptographically by the approval flow.

This is the answer to "what if an admin account is popped?"

8.8 Password policies (advanced)

Password policies control **how OpenBao generates passwords** for dynamic and static users in certain engines.

```
bao write sys/policies/password/my-policy \
    length=24 \
    lower=true \
    upper=true \
    digit=true \
    special="!@#$%^&*" \
    policy_name="my-policy"
```

Rule	Meaning
`length`	Minimum length
`lower` / `upper` / `digit`	Require each class
`special`	The special-character set
`excludes`	Characters to exclude (ambiguous ones like `0`/`0`)

> ⚠ **The docs mark this as advanced and dangerous:** "This generates credentials for external systems (databases, LDAP, etc.) and should be used with caution." A password policy that's too strict produces credentials that break target systems' password rules.

Reference it from a database role:

```
bao write db/roles/app \
    db_name=postgres \
    creation_statements="..." \
    password_policy="my-policy"
```

> ※ **Password policies are unrelated to OpenBao's own `default` policy** — they just share a similar name. Don't confuse them.

8.9 Other expert features worth knowing

The `sudo` capability (and why it's scary)

```
path "sys/raw" { capabilities = ["sudo", "read"] }
```

`sudo` on `sys/raw` lets a token read and **write raw encrypted storage** — effectively root. If `raw_storage_endpoint = true` is set in the server config, this path exists.

```
# Default: the endpoint doesn't exist
raw_storage_endpoint = false
```

The OpenAPI spec endpoint

```
bao read sys/internal/specs/openapi > openapi.json
```

Generate client code, or hand it to your team so they can build against the real API.

The resultant ACL endpoint

```
bao read sys/internal/ui/resultant-acl
```

Shows the **effective** policy for a token — invaluable for debugging "why can they do that?".

`sys/monitor` — streaming request logs

```
bao monitor -log-level=info
```

Streams every request as it happens. Great for live debugging, dangerous in production (it can log secrets if `log_raw` is on).

`sys/leases/lookup` and lease metadata

```
bao write sys/leases/lookup lease_id=database/creds/app/...
```

Since 2.6.0, lease lookup responses include `path`, `namespace_path` and `revoke_error` — much easier to debug.

Server-side consistent tokens (SSCTs)

2.7.0 improved SSCTs to support storage-indicated index values (previously always `0`). SSCTs are encrypted tokens that carry their own authorisation — useful for delegated access without a server-side token record.

```
bao write sys/consistent-tokens/<name> \
    type=service \
    expiration_ttl=1h \
    orphan=true \
    path="secret/data/myapp"
```

`sys/tools/*` — small utilities

```
bao write sys/tools/hash sha256        # hash input
bao write sys/tools/random bytes=32     # crypto-random bytes
bao write sys/tools/lookup path=secret/data/myapp
```

`sys/config/state/sanitized` — what is the server actually running?

```
bao read sys/config/state/sanitized | jq
```

> ⚠ 2.7.1 fixed **GHSA-3237-j65r-m5rp** — a disclosure of `tls_acme_eab_mac_key` through this endpoint. Another reason to run 2.7.1 rather than 2.7.0.

`sys/host-info` and `sys/ha-status`

```
bao read sys/host-info | jq
bao read sys/ha-status | jq
```

8.10 Kubernetes (brief, but you should know it exists)

```
helm repo add openbao https://openbao.github.io/openbao-helm
helm repo update
helm install openbao openbao/openbao \
  --namespace openbao --create-namespace \
  --set 'server.auditStorage.enabled=true' \
  --set 'server.dataStorage.size=10Gi' \
  --set 'injector.enabled=true' \
  --set 'global.image.tag=2.7.1'
```

The Helm chart is at `openbao/openbao-helm` (chart version `0.30.x` as of October 2026).

Key components:

- **Server StatefulSet** with a `Raft` or `PostgreSQL` storage backend
- **Injector** — mutates pods to inject a `bao agent` sidecar that authenticates and renders secrets
- **Audit** — a sidecar that ships the audit log off-box

The injector pattern is the Kubernetes equivalent of the `bao agent` sidecar pattern from §7.2 — same idea, automated.

> ⚠ For Kubernetes, **auto-unseal is effectively mandatory** — you cannot have 3 humans unseal pods. Use the transit seal or a cloud KMS plugin.

8.11 Level 8 checkpoint

1. Why is namespace sealing (2.6.0) genuinely useful for a multi-tenant internal service?
2. Why must `unsafe_cross_namespace_identity` stay off?
3. What must you do before upgrading to 2.7.0 if you use the `pkcs11` seal?
4. Why pin OCI plugins by digest rather than tag?
5. What is a workflow, and when is it better than just calling the API?
6. What problem do External Keys solve that the built-in PKI doesn't?
7. What is the OCSP limitation of ML-DSA issuers?
8. What does a CEL program replace, and what is the risk?
9. Walk through the control-group flow from request to secret.
10. What breaks if you enable `sudo` on `sys/raw`?

Next → **Level 9: REFERENCE.md** — the cheatsheet and runbook.

8.12 Scratchpad

```
Namespaces:      bao namespace create <name>   BAO_NAMESPACE=<name>
Namespace seal:  bao namespace seal <name>     (2.6.0, own barrier keyring)
Plugins:         plugin "kms" "<name>" { image = "...@sha256:..." }
                 plugin_auto_download=true  plugin_auto_register=true
                 plugin_download_behavior="fail"
Workflows:       sys/workflows/manage  +  sys/workflows/execute/<name>
```

```
External keys:   sys/external-keys/<mount>  (PKI + Transit, per-namespace)
PQC:             key_type="mldsa-44|65|87"   tls_key_exchange_preferences
CEL:             validation_cel_program (PKI roles, JWT/OIDC roles)
Control groups:  control_group { factor "x" { identity { group_names, approvals } } }
                 sys/control-group/{authorize,request}  +  sys/wrapping/unwrap
Password policy: sys/policies/password/<name>
SSCT:            sys/consistent-tokens/<name>
```

Level 9 — Reference & Cheatsheet

> Print this. Bookmark it. Come back to it.

9.1 Every important bao command

Global flags and environment

```
bao [command] [options] [path] [args]
```

Flag	Purpose	Env var
`-address`	Server address	`BAO_ADDR`
`-token`	Token	`BAO_TOKEN` (legacy: `VAU`
`-namespace`	Namespace	`BAO_NAMESPACE`
`-format`	`table` | `json` | `yaml`	`BAO_FORMAT`
`-field`	Extract one field	—
`-output-curl-string`	Print the equivalent curl	—
`-output-policy`	Print the required policy	—
`-detailed`	More detail (where supported)	—
`-non-interactive`	Never prompt	—
`-no-store`	Don't write the token to disk	—
`-tls-skip-verify`	⚠ dev only	—
`-ca-cert` / `-ca-path` / `-client-cert` / `-client-key`	TLS material	`BAO_CACERT`

> The `BAO_` prefix is preferred, but the CLI **falls back to `VAULT_`** if present — for compatibility with existing Vault tooling.

Status & info

```
bao status                      # seal state, version, cluster info
bao status -format=json
bao status -field=sealed        # 2.7.0
bao version
bao version-history
```

Operator

```
bao operator init [-key-shares=N] [-key-threshold=T] [-pgp-keys=...]
bao operator init -status
bao operator unseal <key>
bao operator seal
bao operator step-down
```

```
bao operator generate-root [-init] [-format=json]
bao operator rekey -init
bao operator rotate-keys
bao operator key-status
bao operator members
bao operator diagnose
bao operator migrate ...
bao operator validate-config <file>
```

Raft

```
bao operator raft join https://openbao-1:8200
bao operator raft list-peers
bao operator raft remove-peer <node_id>
bao operator raft snapshot save <file>
bao operator raft snapshot inspect <file>
bao operator raft snapshot restore <file>
bao operator raft autopilot state
bao operator raft autopilot get-config
bao operator raft autopilot set-config ...
```

Secrets engines

```
# Built-in types verified on 2.7.1:
#   kv, kv-v2, transit, pki, database, totp, ssh
#   (cubbyhole/ and identity/ are auto-mounted and cannot be enabled)
# Plugin-only (from openbao/openbao-plugins):
#   ldap, aws, gcp, consul, nomad, mongodb
bao secrets enable -path=<path> <type>
bao secrets disable <path>
bao secrets move <old> <new>
bao secrets tune -default-lease-ttl=1h -max-lease-ttl=4h <path>
bao secrets list
bao secrets list -detailed
bao secrets list -format=json
```

KV

```
# Modern syntax: -mount=<mount> <key>
# (the path-like `<mount>/<key>` form is DEPRECATED in 2.7.x)
bao kv put -mount=secret myapp k=v
bao kv get -mount=secret myapp
bao kv get -mount=secret -field=mykey myapp
bao kv get -mount=secret -version=2 myapp
bao kv metadata get -mount=secret myapp
bao kv delete -mount=secret myapp
bao kv undelete -mount=secret myapp
bao kv destroy -mount=secret myapp
bao kv metadata delete -mount=secret myapp
bao kv list -mount=secret
bao kv scan -mount=secret
```

Auth methods

```
# Built-in types verified on 2.7.1:
#   oidc, jwt, approle, userpass, kubernetes, cert
#   (the "token" method is always mounted and cannot be enabled/disabled)
# Plugin-only (from openbao/openbao-plugins):
#   ldap, kerberos, radius, github, aws
bao auth enable -path=<path> <type>
bao auth disable <path>
bao auth list
bao auth accessor list
bao auth tune <path> ...
bao login -method=<type> [args]
# There is NO `bao logout` command in OpenBao 2.7.1. To "log out", delete
# the token file that the default token helper wrote:
rm -f ~/.vault-token          # or $BAO_TOKEN_PATH, if you set it
```

Policies

```
bao policy write <name> -        # from stdin
bao policy read <name>
bao policy list
bao policy delete <name>
# `policy fmt` takes a LOCAL FILE PATH and rewrites it in place.
# (No policy name, no -check flag.)
bao policy fmt ./policies/app-readonly.hcl
```

Tokens

```
bao token create [-policy=...] [-ttl=] [-renewable] [-orphan] [-batch] [-period=] ...
bao token lookup
bao token lookup -accessor <accessor>
bao token renew
bao token renew -increment=2h
bao token revoke <token>
bao token revoke -accessor <accessor>
bao token revoke -orphan <accessor>
bao token revoke -accessor <a> -revoke-lease
bao token capabilities <token> <path>
# (no CLI command — API only)
curl -X POST -H "X-Vault-Token: $BAO_TOKEN" -d '{"path":"<path>"}' \
  "$BAO_ADDR/v1/sys/capabilities-self"
```

Leases

```
bao lease renew -increment=1h <lease-id>
bao lease revoke <lease-id>
bao lease revoke -prefix <path-prefix>
bao lease lookup <lease-id>
```

Identity

```
bao write identity/entity name="alice" metadata=...
bao write identity/entity/id/<id> groups=<group>
bao write identity/entity-alias name=... canonical_id=... mount_accessor=...
bao write identity/group name=<group> policies=<p1>,<p2>
bao read identity/entity/name/alice
bao list identity/entity/name
bao list identity/group/name
```

Audit

```
# 2.7.1: creation via the API is DISABLED by default.
# Use the declarative `audit "<type>" "<path>" { options { ... } }`
# stanza in the server config instead (see §5.5). The command below
# only works with unsafe_allow_api_audit_creation = true.
bao audit enable -path=<name> <type> [options]
bao audit disable <name>
bao audit list
bao audit list -detailed
```

Plugins

```
bao plugin init <sha256> <command>
bao plugin list
bao plugin info <name>
bao plugin register <sha256> <command>
bao plugin reload <name>
bao plugin prune
bao plugin deregister <name>
```

Namespaces

```
bao namespace create <name>
bao namespace list
bao namespace delete <name>
bao namespace seal <name>
bao namespace seal-status <name>
bao namespace patch <name>
```

Misc

```
bao path-help <path>          # discover the API of a path
bao list <path>               # LIST
bao read <path>               # GET
bao write <path> k=v          # PUT/POST
bao delete <path>             # DELETE
bao patch <path> k=v
bao print token
bao unwrap <wrapping-token>
bao monitor [-log-level=info]
bao debug <path>
bao namespace ...
```

9.2 The API path map

System

```
GET    /v1/sys/health
GET    /v1/sys/seal-status
POST   /v1/sys/init
GET    /v1/sys/init
GET    /v1/sys/leader
GET    /v1/sys/ha-status
GET    /v1/sys/host-info
GET    /v1/sys/metrics
GET    /v1/sys/monitor
GET    /v1/sys/config/state/sanitized
GET    /v1/sys/config/auditing
POST   /v1/sys/config/cors
GET    /v1/sys/config/ui
```

Mounts, auth, policies

```
GET    /v1/sys/mounts
POST   /v1/sys/mounts/<type>
DELETE /v1/sys/mounts/<path>
POST   /v1/sys/mounts/<path>/tune
GET    /v1/sys/auth
POST   /v1/sys/auth/<type>
DELETE /v1/sys/auth/<path>
POST   /v1/sys/auth/<path>/tune
GET    /v1/sys/policies/acl
GET    /v1/sys/policies/acl/<name>
POST   /v1/sys/policies/acl/<name>
DELETE /v1/sys/policies/acl/<name>
GET    /v1/sys/policies/egp        (see note in §3.3)
GET    /v1/sys/policies/rgp        (see note in §3.3)
```

Tokens

```
POST   /v1/auth/token/create
POST   /v1/auth/token/create-orphan
GET    /v1/auth/token/lookup-self
POST   /v1/auth/token/renew-self
POST   /v1/auth/token/revoke-self
LIST   /v1/auth/token/accessors
POST   /v1/auth/token/revoke
POST   /v1/auth/token/revoke-accessor
POST   /v1/auth/token/renew
POST   /v1/auth/token/lookup
GET    /v1/auth/token/accessors
```

Audit & quotas

```
GET    /v1/sys/audit
POST   /v1/sys/audit/<type>
```

```
DELETE /v1/sys/audit/<path>
GET    /v1/sys/quotas/config
POST   /v1/sys/quotas/rate-limit
POST   /v1/sys/quotas/lease-count
```

Advanced (2.6/2.7)

```
POST   /v1/sys/generate-root-token          # 2.6+ authenticated root gen
POST   /v1/sys/generate-root-token/update
GET    /v1/sys/generate-root-token/attempt  # 2.6+ authenticated status
POST   /v1/sys/generate-recovery-token
GET    /v1/sys/external-keys                 # 2.7
POST   /v1/sys/external-keys/<mount>
LIST   /v1/sys/workflows/manage              # 2.6
GET    /v1/sys/workflows/manage/<path>
POST   /v1/sys/workflows/execute/<path>
POST   /v1/sys/control-group/authorize       # 2.7
GET    /v1/sys/control-group/request         # 2.7
POST   /v1/sys/wrapping/unwrap
GET    /v1/sys/internal/specs/openapi
GET    /v1/sys/internal/ui/resultant-acl
```

Auth method logins

```
POST   /v1/auth/userpass/login/<username>
POST   /v1/auth/approle/login
POST   /v1/auth/oidc/login
POST   /v1/auth/jwt/login
GET    /v1/auth/oidc/auth_url
GET    /v1/auth/oidc/callback
POST   /v1/auth/kubernetes/login
POST   /v1/auth/cert/login
```

Secrets engines

```
GET    /v1/secret/data/<path>          # KV v2
POST   /v1/secret/data/<path>
GET    /v1/secret/metadata/<path>
POST   /v1/secret/delete/<path>
POST   /v1/secret/undelete/<path>
POST   /v1/secret/destroy/<path>

GET    /v1/transit/keys/<name>
POST   /v1/transit/keys/<name>
POST   /v1/transit/keys/<name>/rotate
POST   /v1/transit/encrypt/<name>
POST   /v1/transit/decrypt/<name>
POST   /v1/transit/sign/<name>
POST   /v1/transit/verify/<name>
POST   /v1/transit/datakey/plaintext/<name>
POST   /v1/transit/hmac/<algo>
POST   /v1/transit/hash
POST   /v1/transit/random/<bytes>

POST   /v1/pki/root/generate/internal
POST   /v1/pki/root/sign-intermediate
```

```
POST   /v1/pki/intermediate/generate/internal
POST   /v1/pki/intermediate/set-signed
POST   /v1/pki/issue/<role>
POST   /v1/pki/revoke
POST   /v1/pki/crl/rotate
GET    /v1/pki/ca
GET    /v1/pki/cert/ca

POST   /v1/database/config/<name>
POST   /v1/database/roles/<name>
GET    /v1/database/creds/<role>
POST   /v1/database/static-roles/<name>
POST   /v1/database/rotate-role/<name>
```

Headers

Header	Purpose
`X-Vault-Token`	The token
`X-Vault-Request: true`	Required for some endpoints (agent/proxy `quit`)
`X-Vault-Wrap-TTL`	Request response wrapping
`X-Vault-Index`	(2.7) consistency index from a write response
`X-Vault-Inconsistent`	(2.7) `fail` | `forward-active-node` | `await-state`
`X-Vault-Namespace`	Namespace header
`X-Vault-Forward`	Internal forwarding

9.3 Production-grade HCL configuration (annotated)

```
# ═══════════════════════════════════════════════════════════════
#  OpenBao 2.7.1 — production server configuration
# ═══════════════════════════════════════════════════════════════

ui           = true
cluster_name = "openbao-prod"

api_addr     = "https://openbao-1.corp.internal:8200"
cluster_addr = "https://openbao-1.corp.internal:8201"

# ── Storage ──────────────────────────────────────────────────────
storage "raft" {
  path    = "/openbao/data"
  node_id = "openbao-1"

  performance_multiplier = 1
  snapshot_threshold     = 8192
  snapshot_interval      = 120
  trailing_logs          = 10000
}

# ── Listeners ────────────────────────────────────────────────────
listener "tcp" {
  address         = "0.0.0.0:8200"
  cluster_address = "0.0.0.0:8201"
```

```
  tls_cert_file = "/openbao/certs/openbao.crt"
  tls_key_file  = "/openbao/certs/openbao.key"
  tls_min_version = "tls12"
  tls_auto_reload  = true            # 2.7.0

  # Unauthenticated legacy endpoints stay OFF
  disable_unauthed_rekey_endpoints = true   # default true

  # Request limits
  max_request_duration   = "60s"
  max_request_size       = 33554432
  max_request_json_memory = 34078720
  max_request_json_strings = 1000
  http_read_timeout      = "30s"
  http_write_timeout     = "0"
  http_idle_timeout      = "5m"

  # 2.7.0 consistency behaviour
  consistency_fallback_behavior        = "forward-active-node"
  consistency_missing_header_forward   = false
  consistency_max_index_wait           = "30s"
}

# ── Memory protection ──────────────────────────────────────────
# ⚠ There is NO `disable_mlock` option in OpenBao 2.x — it was removed in
#    2.0.0 and the server refuses to start if it is present. Harden swap
#    instead (systemd MemorySwapMax=0, or cgroup v2 memory.swap.max=0;
#    in Docker: memswap_limit: 0 / --memory-swappiness=0).

# ── Lease policy ───────────────────────────────────────────────
default_lease_ttl = "24h"
max_lease_ttl     = "168h"
default_max_request_duration = "90s"

# ── User lockout ───────────────────────────────────────────────
user_lockout "all" {
  lockout_threshold    = 5
  lockout_duration     = "15m"
  lockout_counter_reset = "15m"
}

# ── Telemetry ──────────────────────────────────────────────────
telemetry {
  prometheus_retention_time = "30s"
  disable_hostname   = false
  usage_gauge_period = "1m"
}

# ── Response headers (optional, for debugging) ─────────────────
# enable_response_header_hostname = true
# enable_response_header_raft_node_id = true

# ── Plugins (2.5.0+, hardened in 2.7.0) ────────────────────────
plugin_directory = "/openbao/plugins"
plugin_auto_download     = true
```

```
plugin_auto_register     = true
plugin_download_behavior = "fail"
plugin_download_max_size = "536870912"   # 512MB default (2.5.5)

# plugin "kms" "pkcs11" {
#   image = "ghcr.io/openbao/openbao-plugin-kms-pkcs11@sha256:<digest>"
# }

# ── Auto-unseal (2.6.0+ plugin model) ──────────────────────────
# seal "pkcs11" {
#   # ...
#   health_check_enabled           = true
#   health_check_timeout           = "1m"
#   health_check_interval          = "10m"
#   health_check_interval_unhealthy = "1m"
# }

# ── Declarative self-initialization (2.4.0+) ──────────────────
# initialize "audit" {
#   request "enable-audit-file" {
#     operation = "update"
#     path      = "sys/audit/file"
#     data = {
#       type    = "file"
#       options = { file_path = "/openbao/logs/audit.log", mode = "0600" }
#     }
#   }
# }

# ── Declarative audit device (2.4.0+) ──────────────────────────
# audit "file" {
#   type    = "file"
#   options = { file_path = "/openbao/logs/audit.log", mode = "0600", log_raw = false }
# }

# ── Keep these OFF ─────────────────────────────────────────────
raw_storage_endpoint  = false
introspection_endpoint = false
unsafe_cross_namespace_identity = false
# unsafe_relative_paths = false
```

9.4 The full production runbook

Bootstrapping a new cluster

```
# 1. Start
docker compose up -d

# 2. Init (once)
export BAO_ADDR=https://openbao-1:8200
export BAO_CACERT=./certs/ca.crt
bao operator init -key-shares=5 -key-threshold=3 \
  -pgp-keys="alice.asc,bob.asc,carol.asc" \
  -root-token-pgp-key="admin.asc"
```

```
# 3. Unseal node 1
bao operator unseal '<k1>'; bao operator unseal '<k2>'; bao operator unseal '<k3>'

# 4. Join the rest
for n in 2 3 4 5; do
  BAO_ADDR=https://openbao-$n:8200 bao operator raft join https://openbao-1:8200
done

# 5. Unseal the rest (each node, each threshold)
for n in 2 3 4 5; do
  for k in 1 2 3; do
    BAO_ADDR=https://openbao-$n:8200 bao operator unseal '<k>'
  done
done

# 6. Audit FIRST
bao audit enable -path=file file file_path=/openbao/logs/audit.log mode=0600
bao audit enable -path=socket socket address="tcp://audit-sink:9023"

# 7. Configure
bao secrets enable -path=secret kv-v2
bao secrets enable -path=db database
bao secrets enable -path=transit transit
bao auth enable -path=oidc oidc
bao auth enable -path=approle approle
bao auth enable -path=github-actions jwt

# 8. Policies, roles, groups (from your IaC repo)

# 9. Revoke the root token
bao token revoke <root-token>

# 10. Snapshot
bao operator raft snapshot save /backup/initial.snap

# 11. Verify
bao status
bao operator raft autopilot state
bao operator raft list-peers
bao secrets list
bao policy list
```

Daily checks

```
bao status
bao operator raft autopilot state
curl -s --cacert certs/ca.crt $BAO_ADDR/v1/sys/metrics | grep -E 'unsealed|active'
df -h /openbao/data /openbao/logs
tail -f /openbao/logs/audit.log | jq -c '{t:.time,op:.request.operation,path:.request.path,er
```

Weekly checks

```
# Lease/credential leak check
psql -c "SELECT count(*) FROM pg_roles WHERE rolname LIKE 'v-token-%';"
```

```
bao list sys/leases/lookup

# Access review
bao list sys/policies/acl
bao list identity/group/name
bao auth accessor list
```

Monthly checks

```
# Restore drill (in a scratch cluster)
bao operator raft snapshot save /tmp/drill.snap
# ... spin up a scratch cluster, restore, verify ...
```

9.5 Version migration table (Vault → OpenBao, and 2.x changes)

Vault 1.14 → OpenBao 2.x

Vault	OpenBao	Notes
`vault` CLI	`bao` CLI	Config/env still honour `VAULT_` as a fallba
`VAULT_ADDR`	`BAO_ADDR`	`VAULT_` still works as fallback
`VAULT_TOKEN`	`BAO_TOKEN`	Same
`VAULT_NAMESPACE`	`BAO_NAMESPACE`	Same
`VAULT_TOKEN_HELPER`	`BAO_TOKEN_HELPER`	Same
`~/.vault-token`	`~/.vault-token`	Override with `BAO_TOKEN_PATH` (2.6.0)
`vault status`	`bao status`	Same
`vault kv put`	`bao kv put`	Same
`vault operator raft snapshot`	`bao operator raft snapshot`	Same
`VAULT_LICENSE`	—	Not applicable (MPL-2.0)
Enterprise namespaces	`bao namespace`	**Free**
Sentinel roles	CEL roles	Not API-compatible

OpenBao version changes that break things

Version	Change
2.3.1	Namespaces GA
2.4.0	Declarative self-init + audit devices; `file` backend legacy upsert dropped
2.5.0	OCI plugin distribution; standby reads; OIDC client credentials
2.5.5	`plugin_download_max_size` (512MB)
2.6.0	Namespace sealing; KMS plugins; workflows; authenticated root gen; distroless image; `stored_share`
2.6.3	K8s SA JWT validated on renewal
2.7.0	**PQC**; External Keys; control groups; consistency headers; PebbleDB; Postgres read-scaling; **`file` bac**
2.7.1	5 security fixes (GHSA-3237, q74w, 7m59, 5v22, gf3j)

9.6 Troubleshooting

Symptom	Likely cause
`permission denied` on a KV read	Missing `metadata/` path in the policy
`permission denied` on `bao kv metadata get`	Missing `list` on the metadata path
`permission denied` on `bao secrets list`	Policy too narrow (needs `list` on `sys/mounts`)
OIDC login loops	Redirect URI mismatch (http/https, host, port, trailing slash)
OIDC: no groups in the token	Keycloak client scope mapper not configured
OIDC: "issuer" error	Discovery URL host ≠ `iss` claim
AppRole: "invalid secret id"	Already used (`num_uses=1`) or expired
AppRole: 403 on the secret	Role's token policies don't include the read policy
AppRole: token can't create children	`token_num_uses` set
Unseal: "unseal key not valid"	Wrong key / wrong node / key already rotated
Raft: node won't join	`cluster_addr` unreachable, or TLS mismatch
Raft: no leader	Lost quorum
Audit log not written	Path unwritable, or `log_raw` issues
Cluster stops serving	**All** audit devices failed (fail-closed)
`503 sealed`	Node sealed
DB role count growing	Revocation failing
`vault.core.unsealed=0`	Sealed node
Transit decrypt fails after rotation	`min_decryption_version` archived the version
PKI: "common name not allowed"	`allowed_domains` doesn't include it
PKI: chain incomplete	Only the root cert distributed
bao can't reach the server	TLS / CA mismatch

⚠ Scripting gotchas (found the hard way)

These bite every script that talks to OpenBao. All three were found by running the code, not by reading the docs.

```
$ bao status >/dev/null; echo $?
2
```

1. `bao status` exits non-zero when sealed or uninitialised So this idiom **breaks** under `set -euo pipefail`:

```
# ✗ The pipeline's exit status is non-zero, so `|| echo true` ALSO fires
#    and you get V="true
true" — which never equals "true".
V=$(bao status -format=json | jq -r .sealed || echo true)
```

```
# ✓ Capture the JSON first, then parse it
raw=$(bao status -format=json 2>/dev/null || true)
V=$(printf '%s' "$raw" | jq -r '.sealed' 2>/dev/null || true)
```

The same applies to `bao operator init -status` (exit 2 = not initialised).

```
# ✗ Returns "true" when .sealed is FALSE — i.e. when the server is UNSEALED
jq -r '.sealed // "true"'

# ✓ Correct
jq -r 'if has("sealed") then (.sealed | tostring) else "true" end'
```

2. jq's `//` operator treats `false` as empty `false // "x"` → `"x"`, because `//` means "if the left side is null **or false**". This one is nasty: a perfectly healthy server gets reported as "still SEALED".

3. Enabling `kv-v2` briefly makes the mount unavailable

```
* Upgrading from non-versioned to versioned data. This backend will be
  unavailable for a brief period and will resume service shortly.
```

A one-time storage-format upgrade. **Retry** rather than treating it as a failure:

```
for i in 1 2 3 4 5 6; do
  if bao kv put -mount=secret myapp k=v >/dev/null 2>&1; then break; fi
  sleep 2
done
```

4. `bao write -f` is not "print the raw value" `-f` means "raw format", which still prints a table for multi-field responses. For a single field, parse the JSON:

```
CT=$(bao write -format=json transit/encrypt/app-data \
      plaintext=$(echo -n "sensitive" | base64) | jq -r .data.ciphertext)
```

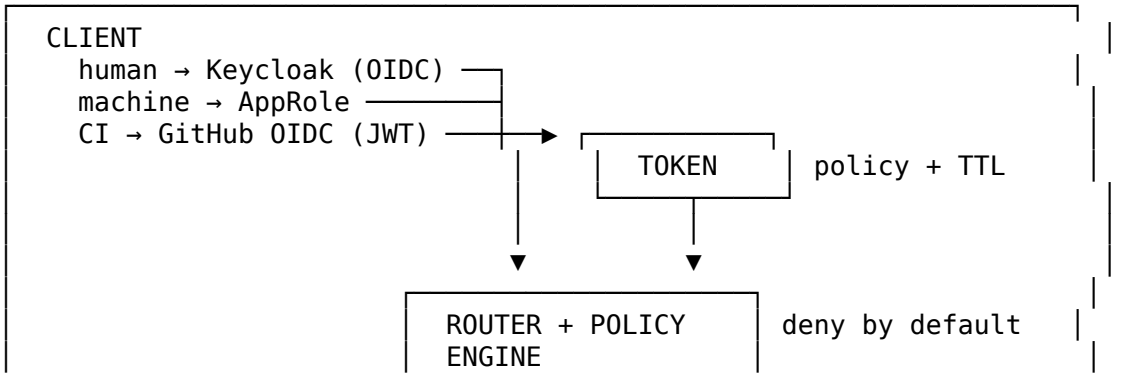
5. `bao read` fails on raw-PEM endpoints `/v1/pki_int/ca` and friends return **PEM**, not JSON. `bao read` expects JSON and will fail with "json: cannot unmarshal number into Go value of type api.Secret". Use `curl`:

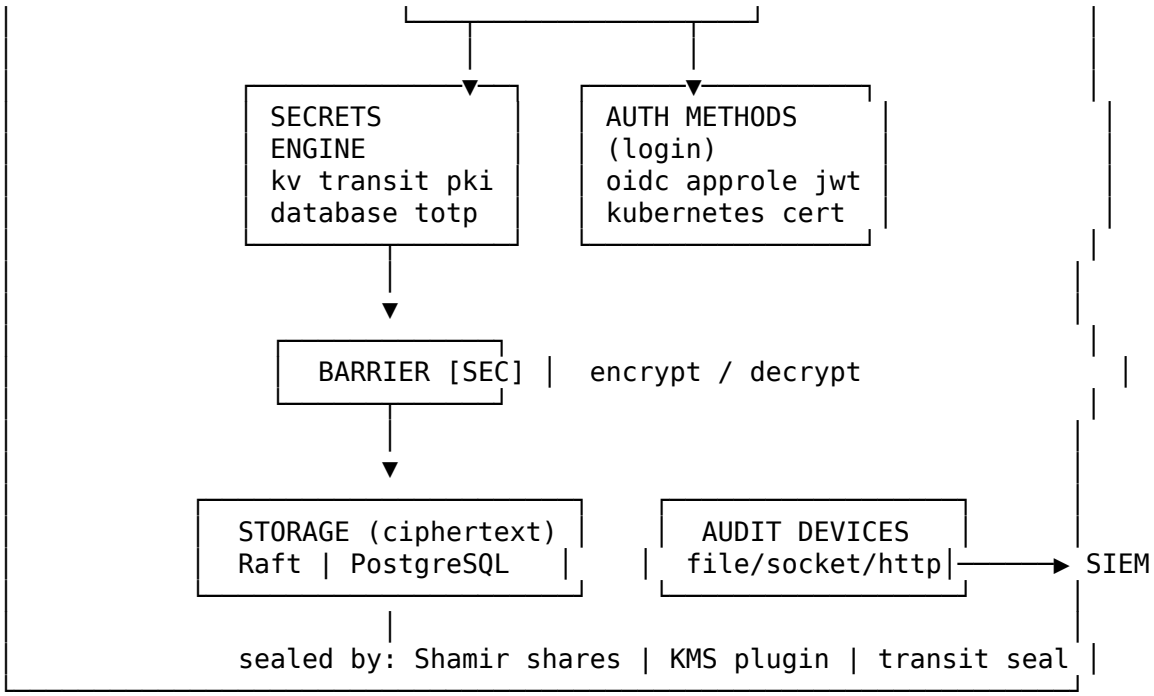
```
curl -s --cacert ca.crt "$BAO_ADDR/v1/pki_int/ca" > ca-chain.crt
```

Diagnostic commands

```
bao status
bao sys/config/state/sanitized | jq          # what is actually configured
bao sys/leader
bao sys/ha-status | jq
bao operator raft autopilot state
bao operator members
bao operator key-status
bao operator diagnose
bao sys/internal/ui/resultant-acl             # effective ACL for a token
curl -X POST -H "X-Vault-Token: $BAO_TOKEN" -d '{"path":"<path>"}' \
  "$BAO_ADDR/v1/sys/capabilities-self"
bao path-help <path>                          # discover an API
bao read sys/internal/specs/openapi > openapi.json
```

9.7 The one-page mental model





Ten truths to remember:

1. Everything is a **path**; every operation is a **capability** on it.
2. Policies are **deny by default**; grants **union**; deny always wins.
3. **Tokens** = identity + policies + TTL. Revoke the token, lose the access.
4. **Accessors** are safe to store; tokens are not.
5. **Leases** make secrets die. Dynamic creds without leases are a smell.
6. The **barrier** means storage holds only ciphertext.
7. The **seal** is the crown jewel. Losing the KMS key = losing everything, even backups.
8. **Audit covers everything** except a short list of unauthenticated paths.
9. If **all** audit devices fail, OpenBao stops serving. That's correct.
10. **Namespaces are free** in OpenBao. Use them for real isolation.

9.8 Where to go next

Topic	Where
Full docs	https://openbao.org/docs/
2.7 release notes	https://openbao.org/community/release-notes/2-7-0/
Source + changelog	https://github.com/openbao/openbao
Plugins	https://github.com/openbao/openbao-plugins
Helm chart	https://github.com/openbao/openbao-helm
API docs (live)	`bao read sys/internal/specs/openapi`
Glossary	https://openbao.org/docs/glossary/
Community	https://github.com/orgs/openbao/discussions
Upgrade guides	https://openbao.org/docs/guides/upgrade/
Migration from Vault	https://openbao.org/docs/guides/migration/

Course complete. You now know OpenBao from the barrier up: how it stores data, how it seals, how it authorizes, how it generates credentials, how it audits, how it clusters, how it fails, and how it plugs into Keycloak, FastAPI, LangChain, PostgreSQL, GitHub Actions and Docker Compose.

Go build something. Then break it in a lab and read the audit log.

Level 10 — Keycloak ⇄ OpenBao: The OIDC Deep Dive

> **Goal:** you can wire Keycloak to OpenBao so that who the user is in Keycloak determines what they can do in OpenBao — for humans, for service accounts, and for your own applications — and you can debug it when it breaks.

Everything in this chapter was executed against a live OpenBao 2.7.1 server with a mock OIDC provider. The commands, the API paths and the error messages are real output, not reconstructed.

10.1 The `oidc` and `jwt` auth methods are the SAME plugin

This is the single most confusing thing about OpenBao's OIDC support, and getting it wrong wastes hours.

`oidc` and `jwt` are two **mount types** of the **same plugin**. They expose **identical paths**:

```
^config$                ← the mount-level JWT/OIDC configuration
^role/(?P<name>…)$      ← roles
^login$                 ← authenticate
^oidc/auth_url$         ← start a browser login
^oidc/callback$         ← the provider redirects back here
^oidc/poll$             ← poll for the result (device / direct modes)
^cel/login$             ← authenticate against a CEL role
^cel/role/(?P<name>…)$  ← CEL roles
```

Verified with `bao path-help`:

```
$ bao path-help auth/oidc | grep -E '^\s+\^'
    ^cel/login$
    ^cel/role/(?P<name>\w(([\w-.]+)?\w)?)$
    ^cel/role/?$
    ^config$
    ^login$
    ^oidc/auth_url$
    ^oidc/callback$
    ^oidc/poll$
    ^role/(?P<name>\w(([\w-.]+)?\w)?)$
    ^role/?$
```

> [ALERT] **The path that trips everyone up: the config lives at `auth/<mount>/config`, NOT `auth/oidc/oidc`.**
>
> ```
> $ bao write auth/oidc/oidc oidc_discovery_url=…
>   * unsupported path
> $ bao write auth/oidc/config oidc_discovery_url=…
>   Success! Data written to: auth/oidc/config
> ```
>
> Likewise a role is at **`auth/<mount>/role/<name>`** — not `auth/oidc/oidc/role/<name>`.

Which mount type do I use?

	jwt mount	oidc mount
Browser login	✓	✓
Validate a JWT you already hold	✓	✓ (same code)
Default role_type	jwt	oidc
Typical use	CI, service tokens, API clients	Humans logging in with a browser

They are functionally identical. Pick oidc for human-facing mounts (so the UI offers a browser login) and jwt for machine-facing ones. Mount the same type twice at different paths if you like — the paths are independent.

10.2 The configuration keys (and the two that bite)

The mount config: auth/<mount>/config

Key	Notes
oidc_discovery_url	**The BASE URL.** OpenBao appends /.well-known/openid-co
jwks_url	A **literal** JWKS URL. Mutually exclusive with oidc_discovery_
jwt_validation_pubkeys	Comma-separated PEM public keys, validated locally. Mutually
oidc_client_id	The client ID at your provider. **Not** client_id.
oidc_client_secret	The client secret. **Not** client_secret. Write-only (reads retur
bound_issuer	Match against the iss claim. Defaults to the discovery docum
default_role	Role used when the login doesn't name one.
skip_jwks_validation	true = warn instead of failing if the provider is unreachable. D
oidc_response_type / oidc_response_types	e.g. code, id_token, code id_token
oidc_response_mode	e.g. query, fragment, form_post
namespace_in_state	true by default — puts the namespace into the OIDC state.

[ALERT] Gotcha #1 — oidc_discovery_url must NOT contain .well-known

```
$ bao write auth/oidc/config \
     oidc_discovery_url="http://keycloak:8080/realms/enterprise/.well-known/openid-
configuration"
  * 'oidc_discovery_url' contains '.well-known' component
```

Correct:

```
bao write auth/oidc/config \
  oidc_discovery_url="http://keycloak:8080/realms/enterprise" \
  oidc_client_id="openbao" \
  oidc_client_secret="$KEYCLOAK_CLIENT_SECRET" \
  default_role="human"
```

The API reference is explicit: "The base URL of the OIDC Discovery URL **without** .well-known/openid-configuration component."

> ※ This is the opposite of jwks_url, which **is** a literal URL including /.well-known/jwks. Two different conventions, one API. That is why everyone gets it wrong the first time.

[ALERT] Gotcha #2 — the client ID/secret keys are prefixed

```
$ bao write auth/oidc/config client_id="openbao" client_secret="s3cret"
  Success! …            # ← accepted, but silently IGNORED
```

```
$ bao read -format=json auth/oidc/config | jq .data.oidc_client_id
""                      # ← empty: the config never took
```

Correct:

```
bao write auth/oidc/config \
  oidc_client_id="openbao" \
  oidc_client_secret="$KEYCLOAK_CLIENT_SECRET"
```

```
$ bao read -format=json auth/oidc/config | jq '{oidc_client_id, status}'
{ "oidc_client_id": "openbao", "status": "valid" }
```

`status: "valid"` means OpenBao successfully fetched and validated the discovery document. **Always check this after configuring** — it is your fastest "is my IdP reachable?" signal.

Verify the whole config in one shot

```
bao read -format=json auth/oidc/config | jq '{
  discovery: .data.oidc_discovery_url,
  client_id: .data.oidc_client_id,
  issuer:    .data.bound_issuer,
  status:    .data.status
}'
```

```
{
  "discovery": "http://127.0.0.1:8098/realms/enterprise",
  "client_id": "openbao",
  "issuer": "",
  "status": "valid"
}
```

10.3 The role: where identity becomes policy

auth/<mount>/role/<name> holds the per-role rules.

```
bao write auth/oidc/role/human - <<'EOF'
{
  "role_type": "oidc",
  "user_claim": "preferred_username",
  "groups_claim": "groups",
  "oidc_scopes": ["openid", "profile", "email", "groups"],
  "allowed_redirect_uris": ["http://localhost:8250/oidc/callback"],
  "token_ttl": "1h",
  "token_max_ttl": "8h"
}
EOF
```

The identity parameters

Key	Meaning
`user_claim`	**Required.** The claim that uniquely identifies the user. Becomes the Identity alias
`user_claim_json_pointer`	`true` → treat `user_claim` as a JSON pointer
`groups_claim`	The claim holding a **list** of group names. Each becomes an **Identity group alias**
`claim_mappings`	Copy claims into token/alias metadata

Key	Meaning
`bound_audiences`	Accepted aud values. **Required for `jwt` roles**, optional for `oidc` roles.
`bound_subject`	Require a specific `sub`
`bound_claims`	Map of claim → required value(s)
`bound_claims_type`	`string` (default, exact match) or `glob` (* = any chars)

The three ways to reach a claim

```
{
  "division": "North America",              // top level
  "groups": { "primary": "Engineering" }    // nested
}
```

Notation	Points at
`division`	top-level key
`/groups/primary`	**JSON pointer** → nested value

This applies to `user_claim`, `groups_claim`, `bound_claims` and `claim_mappings`.

`groups_claim` — the cleanest path from Keycloak to OpenBao

This is the mechanism I most want you to understand, because it replaces ”mapping every user to policies” with ”mapping a group to policies”.

```
Keycloak group  "openbao-sre"
        │
        │  (groups claim in the ID token)
        ▼
OpenBao alias   group alias named "openbao-sre"
        │
        │  (linked to an OpenBao identity group)
        ▼
OpenBao group   "openbao-sre"  →  policies: [human-default, sre-write]
```

Step 1 — Keycloak must put groups in the token. Add a Group Membership protocol mapper to the openbao client (or a client scope assigned to it):

- ”Full group path” → **OFF** (otherwise you get `/openbao-sre` instead of `openbao-sre`)
- ”Claim” → `groups`
- ”Add to ID token” → ON, ”Add to access token” → ON

Step 2 — OpenBao role:

```
{ "groups_claim": "groups", "oidc_scopes": ["openid", "profile", "email", "groups"] }
```

Step 3 — create the OpenBao identity group and link the alias:

```
# The OpenBao group (carries the policies)
bao write identity/group name=openbao-sre \
    policies=human-default,sre-write

# The alias that connects the Keycloak group name to the OpenBao group
bao write identity/group-alias \
    name="openbao-sre" \
    canonical_id=$(bao read -field=id identity/group/name/openbao-sre) \
```

```
    mount_accessor=$(bao auth accessor list -format=json | \
      jq -r '.[] | select(.path=="auth/oidc/") | .accessor')
```

After the first successful login, OpenBao creates the alias automatically if you configure the group's alias with the right mount accessor.

> ※ **Why this beats `claim_mappings` + policy names:** with `claim_mappings` you copy the Keycloak group name into metadata and then have to write policies that check metadata. With `groups_claim`, OpenBao's **identity system** does the joining, and you manage authorisation in one place (`identity/group`). One place to review, one place to change.

`claim_mappings` — copying claims into metadata

```
{
  "email": "email",
  "/realm_access/roles": "kc_realm_roles"
}
```

- `"email"` → top-level claim `email`, metadata key `email`
- `"/realm_access/roles"` → JSON pointer to a nested claim, metadata key `kc_realm_roles`

The required/optional rule (this is the part people miss):

Key form	Behaviour
`email` (no leading `/`)	**Optional** — if the claim is absent, the mapping is skipped
`/realm_access/roles` (leading `/`)	**Required** — if the claim is absent, **authentication fails**

> ⚠ The metadata key **`role` is reserved** and cannot be used as a claim-mapping target.

Bound claims — restricting who may use a role

```
{
  "bound_claims": {
    "department": "Engineering",
    "email": ["fred@example.com", "julie@example.com"]
  }
}
```

A claim's expected value may be a **single string** or a **list** — the claim must match one of the items.

With globs:

```
{
  "bound_claims": { "email": "*@corp.internal" },
  "bound_claims_type": "glob"
}
```

> [SEC] The `bound_claims_type = "glob"` option means `*` matches any number of characters. It is convenient and it is also a footgun: `*@corp.internal` also matches `attacker@corp.internal`. Use narrow globs, or exact values where you can.

10.4 The browser login flow

CLI login

```
bao login -method=oidc -path=oidc -role=human
```

OpenBao prints (and tries to open) an authorization URL. You can also fetch it yourself — note it is a **POST**, not a GET:

```
bao write -format=json auth/oidc/oidc/auth_url \
    role=human \
    redirect_uri="http://localhost:8250/oidc/callback" \
  | jq -r .data.auth_url
```

Real output from the verified run:

```
http://127.0.0.1:8098/realms/enterprise/protocol/openid-connect/auth
  ?client_id=openbao
  &code_challenge=OBp1kZ-7dlPuUiTelIXQLi4WK3xHFDT-neJcJ4NQ2eI
  &code_challenge_method=S256
  &nonce=n_C6rddd7D8c2GX9aUyF4a
  &redirect_uri=http%3A%2F%2Flocalhost%3A8250%2Foidc%2Fcallback
  &response_type=code
  &scope=openid+profile+email+groups
  &state=...
```

Note **`code_challenge_method=S256`** — OpenBao uses **PKCE** automatically. That is a security feature: the CLI can never be tricked into redeeming a code it did not start.

Redirect URIs — exact match required

Flow	Redirect URI
CLI, `callback_mode=client` (default)	`http://localhost:8250/oidc/callback`
CLI, `callback_mode=direct`	`https://<host>:8200/v1/auth/<mount>/oidc/callback`
OpenBao UI	`https://<host>:8200/ui/vault/auth/<mount>/oidc/oidc/callbac`
`callback_mode=device`	**none** — the device flow has no callback

The same URIs must be registered at Keycloak, and they must match exactly: scheme, host, port, path, trailing slash.

`callback_mode`

Mode	How it works	Use whe
`client` (default)	OpenBao's CLI starts a tiny local listener on :8250 and receives the callback there	Local CL
`direct`	The provider redirects straight to the OpenBao server	Server-s
`device`	The user enters a code on another device; OpenBao polls	No brow

`direct` mode and the confirmation page

In `direct` mode OpenBao shows an **intermediate confirmation page** showing the requester's IP address, so the user can verify they initiated the login and aren't being phished by a crafted link (RFC 8628 §5.4).

> ※ Set `oidc_disable_confirmation = true` **only** if your provider always shows its own confirmation prompt. OpenBao's is on by default; leave it on.

`max_age` — force re-authentication

```
{ "max_age": "15m" }
```

Adds `max_age` to the authorization request, so the provider must have authenticated the user within that window. This is how you force a fresh password entry for sensitive operations.

10.5 The complete Keycloak → OpenBao setup

Keycloak side

Realm `enterprise`. **Client** `openbao`:

Setting	Value
Client type	OpenID Connect, **confidential**
Client ID	`openbao`
Client secret	(copy it — you need it for `oidc_client_secret`)
Standard flow	enabled
Direct access grants	**disabled**
Implicit flow	disabled
Valid redirect URIs	`http://localhost:8250/oidc/callback`, `https://openbao.corp.internal:8200/ui/`
Web origins	`https://openbao.corp.internal:8200`
PKCE method	S256

Protocol mappers on the client (or on a client scope assigned to it):

Mapper	Type	Setting
`groups`	Group Membership	Full group path **off**, claim name `groups`, add to ID + access token
`preferred_username`	User Property	attribute `username` → claim `preferred_username`
`email`	User Property	attribute `email` → claim `email`

OpenBao side

```
# 1. Enable the mount
bao auth enable -path=oidc oidc

# 2. Point it at Keycloak (BASE URL — no .well-known!)
bao write auth/oidc/config \
    oidc_discovery_url="http://keycloak:8080/realms/enterprise" \
    oidc_client_id="openbao" \
    oidc_client_secret="$KEYCLOAK_CLIENT_SECRET" \
    default_role="human"

# 3. Check status == "valid"
bao read -format=json auth/oidc/config | jq -r .data.status

# 4. The role
bao write auth/oidc/role/human - <<'EOF'
{
  "role_type": "oidc",
  "user_claim": "preferred_username",
```

```
  "groups_claim": "groups",
  "oidc_scopes": ["openid", "profile", "email", "groups"],
  "allowed_redirect_uris": [
    "http://localhost:8250/oidc/callback",
    "https://openbao.corp.internal:8200/ui/vault/auth/oidc/oidc/callback"
  ],
  "token_ttl": "1h",
  "token_max_ttl": "8h",
  "claim_mappings": {
    "email": "email"
  }
}
EOF

# 5. The OpenBao groups that carry the policies
bao write identity/group name=openbao-sre      policies=human-default
bao write identity/group name=openbao-platform policies=human-default,platform-write
```

Test

```
bao login -method=oidc -path=oidc -role=human
bao token lookup | jq '{policies: .data.policies, entity: .data.entity_id}'
```

10.6 Keycloak service accounts → OpenBao (machine auth)

Keycloak's **service accounts** (Client Credentials) are the natural machine identity in a Keycloak-centric org. You can exchange one for an OpenBao token using the `jwt` auth method.

Keycloak side

Enable **Service Accounts** on the client. Note the service account's `client_id` (usually `service-account-<clientId>`) — it becomes the `sub` claim.

OpenBao side

```
bao auth enable -path=keycloak-jwt jwt

bao write auth/keycloak-jwt/config \
    jwks_url="http://keycloak:8080/realms/enterprise/protocol/openid-connect/certs" \
    bound_issuer="http://keycloak:8080/realms/enterprise"

bao write auth/keycloak-jwt/role/svc-harness - <<'EOF'
{
  "role_type": "jwt",
  "user_claim": "preferred_username",
  "bound_subject": "service-account-harness-app",
  "bound_audiences": ["harness-api"],
  "token_policies": ["harness-service"],
  "token_ttl": "30m",
  "token_max_ttl": "1h"
}
EOF
```

Exchange

```
# 1. Get a token from Keycloak (Client Credentials grant)
KC_TOKEN=$(curl -s -X POST \
  "http://keycloak:8080/realms/enterprise/protocol/openid-connect/token" \
  -d "grant_type=client_credentials" \
  -d "client_id=harness-app" \
  -d "client_secret=$KC_CLIENT_SECRET" \
  -d "audience=harness-api" | jq -r .access_token)

# 2. Exchange it for an OpenBao token
bao write auth/keycloak-jwt/login role=svc-harness jwt="$KC_TOKEN"
```

> ※ **The `audience` parameter matters.** Keycloak only puts a given audience in the access token if you ask for it (`audience=`), unless you use an audience mapper. If `bound_audiences` doesn't match, the login fails with a confusing message — and it is the single most common cause of "it worked yesterday".

10.7 Keycloak → OpenBao for your own application (the reverse direction)

Your FastAPI app needs to know who the user is. Two options:

Option A — the app validates Keycloak's token itself

```
Browser → your app → (app redirects to Keycloak) → Keycloak → app
                                                    (app validates the ID token)
```

Your app is the OIDC **relying party**. It holds the Keycloak JWKS and verifies the token. OpenBao is not involved in the user path at all.

Use this for user-facing apps. It's the standard pattern and it keeps the request path fast.

```
# The app's own OIDC validation (not OpenBao's)
# - fetch Keycloak's JWKS
# - verify iss, aud, exp, signature
# - map the groups claim to your own roles
```

Option B — OpenBao is the OIDC provider to your app

OpenBao can issue OIDC tokens itself (`sys/oidc/providers/default`). Your app then treats OpenBao as its IdP.

```
bao write sys/oidc/providers/default \
    client_type="public" \
    allowed_redirect_uris="https://app.corp.internal/callback" \
    allowed_scopes="openid profile email" \
    default_scopes="openid profile email" \
    issuer="https://openbao.corp.internal:8200"
```

> ⚠ **2.7.0 removed `prompt=none` redirection support in the OIDC provider** (GHSA-2cjw-94fw-wqjx). If you had a flow that relied on silent re-authorization, it will now fail — by design.
>
> **2.6.0 added a Client Credentials flow to the OIDC provider**, so OpenBao can now issue tokens for machine-to-machine flows directly. **2.5.0** added the same to the provider. This is how your services get tokens from OpenBao as an IdP.

Which direction?

	App validates Keycloak	OpenBao as provider
Latency in the user path	Lower	Higher (extra hop)
Identity source	Keycloak	OpenBao's identity store
Central policy enforcement	In your app	**In OpenBao** ✓
Machine-to-machine	Your own plumbing	Built-in client_credentials ✓
Best for	User-facing apps	Apps that need central authorisation

 Recommended for your stack: Option A for your FastAPI app's user path (Keycloak is already your IdP), and **OpenBao as the provider** only if you want OpenBao to be the single authorisation point for service tokens.

10.8 `token_policies_template_claims` — one role, many policies

This is the most powerful and least-known feature in the OIDC/JWT role. It lets **the token's own claims decide which policies it gets**.

```
{
  "token_policies_template_claims": true,
  "token_policies": [
    "team-{{.team}}",
    "{{if eq \"production\" .environment}}prod-write{{end}}"
  ]
}
```

- Templates are `text/template` evaluated against **all claims** on the token.
- A template that evaluates to the **empty string is removed**.
- **All referenced claims must exist**, or the login fails — unless you guard it with `index`.

Conditional example from the docs:

`token_policies=["{{if eq \"prod\" .environment}}project_{{.project_id}}/env_{{.environment}}/b`

→ the policy is empty (and removed) when `environment` is `staging`.

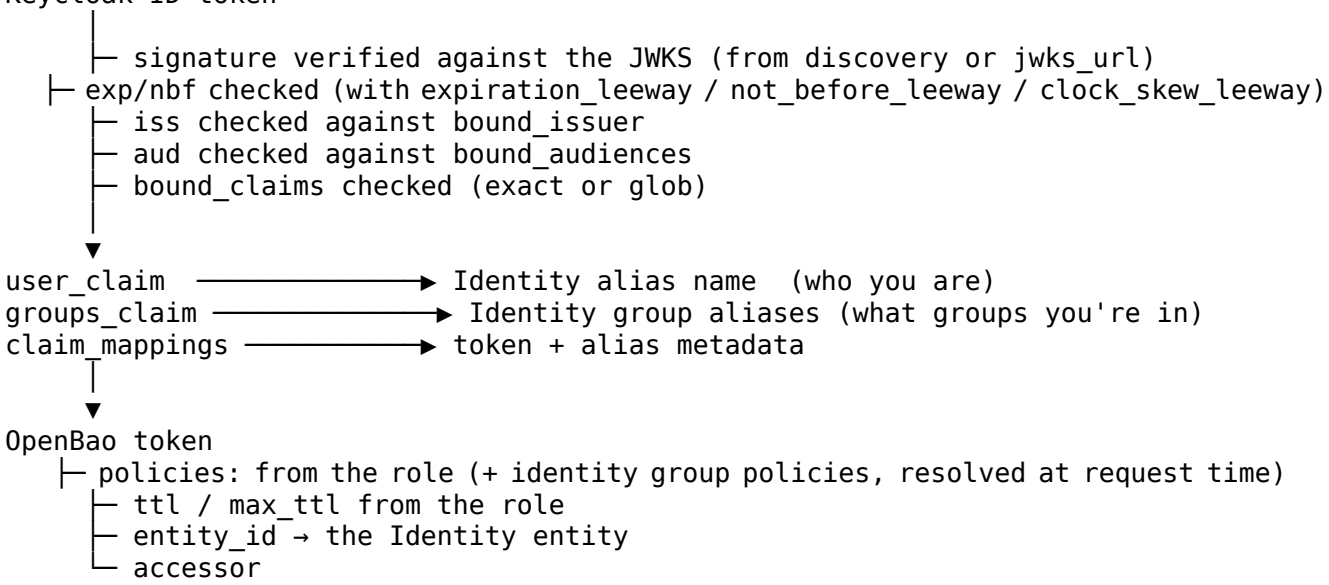
Verified working

```
$ bao write auth/github-actions/role/github - <<'EOF'
{
  "role_type": "jwt",
  "bound_audiences": ["openbao"],
  "bound_claims": {"repository": "your-org/your-repo"},
  "token_policies_template_claims": true,
 "token_policies": ["ci-{{.repository_owner}}", "{{if eq \"production\" .environment}}ci-
prod{{end}}"]
}
EOF
$ bao write auth/github-actions/login role=github jwt="$JWT"
token_policies       ["ci-prod" "ci-your-org" "default"]
```

The token got `ci-your-org` (from `repository_owner`) **and** `ci-prod` (because `environment == "production"`). One role, per-repo and per-environment authorisation, with no extra configuration.

※ **Guard optional claims.** If a claim might be absent, use `index`: `{{if index . "environment" "dev"}}…{{end}}` — otherwise a missing claim fails the whole login.

10.9 The claim → OpenBao token pipeline



Leeway parameters (relevant mostly for `jwt` roles):

Parameter	Default when `0`
`clock_skew_leeway`	60s
`expiration_leeway`	150s
`not_before_leeway`	150s

Set to `-1` to disable. These matter when your IdP and OpenBao clocks drift — and clocks **do** drift.

10.10 Troubleshooting matrix

Symptom	Cause
`unsupported path` on `auth/oidc/oidc`	Wrong config path
`'oidc_discovery_url' contains '.well-known' component`	You pasted the full discovery URL
`oidc_client_id` is empty after writing	You used `client_id`
`status` is not `"valid"`	Provider unreachable, or issuer mismatch
`OIDC login is not configured for this mount`	`oidc_client_id` empty
Login loops between Keycloak and OpenBao	Redirect URI mismatch
`no groups claim in token`	Keycloak mapper missing, or scope not reque
Groups show as `/openbao-sre`	”Full group path” is ON in the mapper
`permission denied` immediately after login	Role’s `token_policies` too narrow, or the ide
`invalid audience` / login refused	`bound_audiences` ≠ the token’s `aud`
Token works, then stops after 8h	Role `token_max_ttl`
”error verifying token signature”	JWKS unreachable, or the key has no matchin
Clock-skew rejections	IdP and OpenBao clocks differ

The three debugging tools

```
# 1. Is the mount configured and reachable?
bao read -format=json auth/oidc/config | jq '.data | {status, oidc_discovery_url, oidc_client

# 2. What does the browser flow actually request?
bao write -format=json auth/oidc/oidc/auth_url role=human \
    redirect_uri="http://localhost:8250/oidc/callback" | jq -r .data.auth_url

# 3. What can the issued token actually do?
bao token capabilities <token> secret/data/myapp
```

And for claim-level debugging (development only!):

```
{ "verbose_oidc_logging": true }
```

> [ALERT] **`verbose_oidc_logging` logs the received OIDC token and claims to the server log at debug level.** Claims data is logged verbatim and may contain sensitive information. **Never enable it in production.** Turn it on, debug, turn it off.

10.11 The Keycloak-side cheat sheet

I want…	Keycloak setting	OpenBao setting
Username in the token	`preferred_username` mapper	`user_claim=preferred_username`
Email in the token	`email` mapper	`claim_mappings={"email":"email"}`
Groups in the token	Group Membership mapper, full path OFF	`groups_claim=groups` + `oidc_scopes`
Realm roles in the token	Role scope mapper (realm roles)	`claim_mappings={"/realm_access/r`
Client roles in the token	Role scope mapper (client roles)	`claim_mappings={"/resource_acces`
Service account token	Service Accounts enabled	`bound_subject=service-account-<c`
Machine token	Client Credentials grant	`role_type=jwt` + `bound_audiences`
A specific audience	Audience mapper, or `audience=` param	`bound_audiences`
Token for one environment	Environment attribute	`bound_claims={"environment":"pro`

10.12 Checkpoint

1. What is the config path for an `oidc` mount, and what are the two `client_id`-shaped keys called?
2. Why must `oidc_discovery_url` not contain `.well-known`, but `jwks_url` may?
3. What does `status: "valid"` on the mount config tell you?
4. What is the difference between a `bound_claims` key with a leading `/` and one without, in `claim_mappings`?
5. How does `groups_claim` differ from `claim_mappings` for group-based authorisation?
6. What is `token_policies_template_claims` for, and what happens to a template that evaluates to an empty string?
7. Which OIDC redirect URI does the CLI use in `client` callback mode, and which does the UI use?
8. What does PKCE protect against here?
9. Why might a Keycloak access token not contain the `aud` your OpenBao role expects?

Next → **Level 11: GITHUB-ACTIONS-OIDC.md** — the CI pipeline, with zero stored secrets.

10.13 Scratchpad

```
Mount paths:    auth/<mount>/config | role/<name> | login | oidc/auth_url | oidc/callback
```

```
Config keys:      oidc_discovery_url (BASE url!)  oidc_client_id  oidc_client_secret
              jwks_url (literal url)  bound_issuer  default_role  skip_jwks_validation
Role keys:      user_claim  groups_claim  claim_mappings  bound_audiences  bound_subject
                  bound_claims  bound_claims_type  oidc_scopes  allowed_redirect_uris
                   callback_mode  max_age  token_policies_template_claims
Claim notation:  "email" = top level (optional)  "/realm_access/roles" = JSON pointer (required)
Redirect URIs:    CLI client → http://localhost:8250/oidc/callback
                  CLI direct → https://host:8200/v1/auth/<mount>/oidc/callback
                UI       → https://host:8200/ui/vault/auth/<mount>/oidc/oidc/callback
```

Level 11 — GitHub Actions ⇄ OpenBao: Secrets-Free CI

> **Goal:** your CI pipeline pulls secrets from OpenBao using a **short-lived OIDC identity** — with **zero long-lived credentials stored in GitHub**. Every command and every error message here was produced by a real OpenBao 2.7.1 server validating a real signed JWT.

11.1 The problem this solves

The default CI pattern is:

```yaml
# ✗ The old way
env:
  DB_PASSWORD: ${{ secrets.DB_PASSWORD }}
```

That secret sits in GitHub's vault, is visible to anyone with repo settings access, appears in the Actions UI, and cannot be rotated without touching every workflow. It is, functionally, a long-lived static credential — exactly the thing OpenBao exists to eliminate.

The OIDC pattern:

```yaml
# ✓ The new way
permissions:
  id-token: write      # ← GitHub mints a short-lived JWT proving "I am this workflow"
steps:
  - uses: actions/checkout@v4
  - run: exchange-the-jwt-for-an-openbao-token
```

GitHub issues a **signed JWT** (RS256, 5-minute lifetime, scoped by `audience`, `repository`, `ref`, `workflow`, `environment`). OpenBao validates it and issues its own token with **your** policies. Nothing is stored.

The properties you get for free

Property	Why it matters
Short-lived	The GitHub token lives ~5 minutes. The OpenBao token you get is as short-lived as you c
Scoped	`bound_audiences` + `bound_claims` mean a token minted for your repo on main cannot be
Attributable	The OpenBao audit log records exactly which workflow, which ref, which run touched whic
Revocable	Revoke the role, or tighten a `bound_claim` — every future run is affected immediately. No
No secret to leak	There is no `DB_PASSWORD` in GitHub to leak.

11.2 The claims GitHub puts in the token

```
{
  "iss": "https://token.actions.githubusercontent.com",
  "aud": "openbao",                        // ← you choose this
  "sub": "repo:your-org/your-repo:ref:refs/heads/main",
  "repository": "your-org/your-repo",
  "repository_owner": "your-org",
  "ref": "refs/heads/main",
  "workflow": "deploy.yml",
  "environment": "production",             // ← if you use GitHub Environments
  "actor": "octocat",
  "run_id": "1234567890",
  "run_number": "42",
  "repository_visibility": "private",
  "job_workflow_ref": "your-org/your-repo/.github/workflows/deploy.yml@refs/heads/main",
  "ref_type": "branch",
  "sha": "a1b2c3…",
  "runner_environment": "github-hosted"
}
```

The **sub** format encodes the context:

```
repo:{owner}/{repo}:ref:{ref}              ← a branch
repo:{owner}/{repo}:pull_request:{n}        ← a pull request
repo:{owner}/{repo}:environment:{env}       ← an environment
repo:{owner}/{repo}:tag:{tag}               ← a tag
```

11.3 OpenBao side — the complete configuration

Step 1 — the mount

```
bao auth enable -path=github-actions jwt
```

> ※ Use a **jwt** mount here, not oidc. You're validating a JWT GitHub already minted; there's no browser and no client secret. (Both mount types work identically — see §10.1 — but jwt is the honest name for it.)

Step 2 — the mount config

```
bao write auth/github-actions/config - <<'EOF'
{
  "oidc_discovery_url": "https://token.actions.githubusercontent.com",
  "bound_issuer": "https://token.actions.githubusercontent.com",
  "default_role": "github"
}
EOF
```

> [ALERT] **oidc_discovery_url is the BASE URL** — no .well-known suffix. OpenBao appends /.well-known/openid-configuration itself.

```
$ bao write auth/github-actions/config \
         oidc_discovery_url="https://token.actions.githubusercontent.com/.well-known/openid-configuration"
  * 'oidc_discovery_url' contains '.well-known' component
```

Verified live:

```
$ bao read -format=json auth/github-actions/config | jq -r .data.status
valid
```

The alternative (if you prefer to pin the key set explicitly):

```
bao write auth/github-actions/config - <<'EOF'
{
  "jwks_url": "https://token.actions.githubusercontent.com/.well-known/jwks",
  "bound_issuer": "https://token.actions.githubusercontent.com",
  "default_role": "github"
}
EOF
```

※ `jwks_url` **is** a literal URL — unlike `oidc_discovery_url`, it keeps the `/.well-known/jwks` path. And `oidc_discovery_url` and `jwks_url` are **mutually exclusive**: setting both fails.

Use `oidc_discovery_url` (recommended — GitHub rotates its keys and discovery picks up the new `jwks_uri` automatically). Use `jwks_url` only if you need to pin a specific JWKS endpoint.

Step 3 — the role

```
bao write auth/github-actions/role/github - <<'EOF'
{
  "role_type": "jwt",
  "user_claim": "actor",
  "bound_audiences": ["openbao"],
  "bound_claims": {
    "repository": "your-org/your-repo",
    "ref": "refs/heads/main"
  },
  "token_ttl": "15m",
  "token_max_ttl": "30m",
  "token_policies": ["ci-deploy"]
}
EOF
```

[ALERT] **The config path is `auth/<mount>/config` and the role path is `auth/<mount>/role/<name>`.** There is no `auth/<mount>/jwt/<name>` — that path returns `unsupported path`.

Step 4 — the policy

```
bao policy write ci-deploy - <<'EOF'
# CI can read the app's config
path "secret/data/myapp" {
  capabilities = ["read"]
}
path "secret/metadata/myapp" {
  capabilities = ["read", "list"]
}

# CI can update the deploy config it owns
path "secret/data/myapp/deploy" {
  capabilities = ["create", "read", "update", "patch"]
}
```

```
path "secret/metadata/myapp/deploy" {
  capabilities = ["read", "list"]
}

# Read-only health check
path "sys/health" {
  capabilities = ["read"]
}
EOF
```

> ⚠ **Never give CI `list` on `secret/data/*` or `secret/metadata/*`.** `list` reveals the shape of your secret store — every path in it. A compromised workflow that can list can enumerate everything, and only needs one more permission to read it.

Step 5 — verify the config

```
bao read -format=json auth/github-actions/config | jq '.data | {status, bound_issuer}'
```

```
{ "status": "valid", "bound_issuer": "https://token.actions.githubusercontent.com" }
```

11.4 The GitHub Actions workflow

```
name: Deploy

on:
  push:
    branches: [main]
  workflow_dispatch:

# ── The only permission that matters ──────────────────────
permissions:
  contents: read
  id-token: write        # ← REQUIRED. Without it, there is no OIDC token.

# ── A GitHub Environment adds an approval gate AND the
#    "environment" claim ──────────────────────────────────
environment:
  name: production

jobs:
  deploy:
    runs-on: ubuntu-latest
    environment: production

    steps:
      - uses: actions/checkout@v4

      - name: Exchange GitHub OIDC for an OpenBao token
        id: bao
        env:
          BAO_ADDR: https://openbao.corp.internal:8200
          BAO_CACERT: ${{ secrets.BAO_CACERT }}
          # MUST match OpenBao's bound_audiences
```

```
          BAO_AUDIENCE: openbao
        run: |
          set -euo pipefail

          # 1. Ask GitHub for an OIDC token
          OIDC_TOKEN=$(curl -sS -H "Authorization: bearer $ACTIONS_ID_TOKEN_REQUEST_URL" \
                            "$ACTIONS_ID_TOKEN_REQUEST_URL&audience=$BAO_AUDIENCE" \
                            | jq -r .value)

          if [ "$OIDC_TOKEN" = "null" ] || [ -z "$OIDC_TOKEN" ]; then
            echo "::error::could not obtain an OIDC token — is id-token: write set?"
            exit 1
          fi

          # 2. Exchange it for an OpenBao token
          BAO_TOKEN=$(curl -sS --cacert "$BAO_CACERT" \
            -H "X-Vault-Request: true" \
            -H "Content-Type: application/json" \
            -d "{\"role\":\"github\",\"jwt\":\"$OIDC_TOKEN\"}" \
            "$BAO_ADDR/v1/auth/github-actions/login" | jq -r .auth.client_token)

          # 3. Never let it reach the log
          echo "::add-mask::$BAO_TOKEN"
          echo "token=$BAO_TOKEN" >> "$GITHUB_OUTPUT"

      - name: Fetch app config from OpenBao
        env:
          BAO_TOKEN: ${{ steps.bao.outputs.token }}
        run: |
          set -euo pipefail
          # Write to a file, never to stdout — a printed secret lands in the log
          curl -sS --cacert "$BAO_CACERT" \
            -H "X-Vault-Token: $BAO_TOKEN" \
            "$BAO_ADDR/v1/secret/data/myapp" > app-config.json
          echo "::group::config keys"
          jq -r '.data.data | keys[]' app-config.json
          echo "::endgroup::"

      - name: Deploy
        run: ./deploy.sh
```

What is and isn't a secret here

Item	Secret?	Where it lives
`BAO_CACERT`	**No** — it's a public CA certificate	GitHub Secrets (for easy rotation) or inline in the re
The GitHub OIDC token	Yes, transient	In the runner's memory, ~5 min
The OpenBao token	Yes, transient	In the runner's memory, 15 min
`DB_PASSWORD`	**Doesn't exist**	Never fetched into the repo at all

[KEY] **The `aud` must match `bound_audiences` exactly.** In the workflow: `audience=openbao`. In OpenBao: `"bound_audiences": ["openbao"]`. Change one, change both. This is the #1 cause of a 401 here.

11.5 Using the bao CLI in CI

```
      - name: Install the bao CLI
        run: |
          set -euo pipefail
          curl -sSL -o /tmp/bao.tgz \
            https://github.com/openbao/openbao/releases/download/v2.7.1/openbao_2.7.1_linux_a
          tar -xzf /tmp/bao.tgz -C /usr/local/bin bao
          chmod +x /usr/local/bin/bao
          bao version

      - name: Fetch secrets with bao
        env:
          BAO_ADDR: https://openbao.corp.internal:8200
          BAO_CACERT: ${{ secrets.BAO_CACERT }}
          OIDC_TOKEN: ${{ steps.oidc.outputs.token }}
        run: |
          set -euo pipefail
          export BAO_TOKEN=$(curl -sS --cacert "$BAO_CACERT" \
            -H "Content-Type: application/json" \
            -d "{\"role\":\"github\",\"jwt\":\"$OIDC_TOKEN\"}" \
            "$BAO_ADDR/v1/auth/github-actions/login" | jq -r .auth.client_token)
          echo "::add-mask::$BAO_TOKEN"
          bao kv get -mount=secret -field=api_key myapp/deploy > /tmp/deploy-key
```

※ **Use the CLI, not raw curl, for anything non-trivial.** It handles namespaces, wrapping, format parsing and error messages properly. Raw curl is fine for the one `login` call.

11.6 Hardening the pipeline

GitHub Environment = approval gate + claim

```
environment:
  name: production
```

In the repository settings, configure the `production` environment with **Required reviewers**. Now:

- a human approves the deploy, **and**
- the OIDC token carries `environment: "production"`.

That gives you a two-factor control on a secret read.

Bind the role to the environment

```
bao write auth/github-actions/role/github - <<'EOF'
{
  "role_type": "jwt",
  "user_claim": "actor",
  "bound_audiences": ["openbao"],
  "bound_claims": {
    "repository": "your-org/your-repo",
    "environment": "production"
  },
  "token_ttl": "15m",
  "token_max_ttl": "30m",
```

```
  "token_policies": ["ci-deploy"]
}
EOF
```

Now a token minted for a `staging` environment deployment **cannot** use the `production` role.

Bind to a workflow file

```
"bound_claims": {
  "repository": "your-org/your-repo",
  "workflow": "deploy.yml"
}
```

Prevents any other workflow in the repo from minting a usable token.

Bind to the SHA / ref

```
"bound_claims": { "ref": "refs/heads/main" }
```

For a release branch, use the tag name. For maximum strictness, bind `repository_owner` as well.

The `environment` claim in `token_policies_template_claims`

This is the pattern that gives you per-environment authorisation from a single role:

```
bao write auth/github-actions/role/github - <<'EOF'
{
  "role_type": "jwt",
  "user_claim": "actor",
  "bound_audiences": ["openbao"],
  "bound_claims": { "repository": "your-org/your-repo" },
  "token_policies_template_claims": true,
  "token_policies": [
    "ci-{{.repository_owner}}",
    "{{if eq \"production\" .environment}}ci-prod{{end}}",
    "{{if eq \"staging\" .environment}}ci-staging{{end}}"
  ],
  "token_ttl": "15m",
  "token_max_ttl": "30m"
}
EOF
```

Verified output (a token with `environment: production`, `repository_owner: your-org`):

```
token_policies       ["ci-prod" "ci-your-org" "default"]
```

One role. A `staging` run gets `ci-staging` and **not** `ci-prod`. A run from a fork gets neither.

> ※ **Guard optional claims.** If `environment` might be absent (a workflow with no environment), a template referencing a missing claim **fails the whole login**. Use `index`: `{{if index . "environment" "none"}}ci-none{{end}}`.

Never print a secret in CI

```
# ✗
echo "password is $DB_PASSWORD"
```

```
# ✗
cat app-config.json          # ← prints every secret into the log

# ✓
jq -r '.data.data | keys[]' app-config.json   # print the KEYS only
```

OpenBao's own audit log HMACs secret values, but the **CI log** does not. `echo "::add-mask::$BAO_TOKEN"` masks the token; the secrets you fetch with it are not masked. Mask them explicitly if you must print anything:

```
echo "::add-mask::$(jq -r '.data.data.db_password' app-config.json)"
```

11.7 The reverse direction: OpenBao as GitHub's OIDC provider

You can also make **OpenBao** the IdP that GitHub Actions logs in to. This requires an `oidc` mount in OpenBao plus an OpenBao OIDC provider configured with a GitHub client.

```
# 1. An OIDC provider in OpenBao
bao write sys/oidc/providers/default \
    client_type="public" \
    allowed_redirect_uris="https://github.com/your-org/your-repo" \
    allowed_scopes="openid profile email" \
    default_scopes="openid profile email" \
    issuer="https://openbao.corp.internal:8200"
```

```
# 2. A client for GitHub
bao write sys/oidc/providers/default/clients/github-actions \
    client_type="public" \
    allowed_redirect_uris="https://github.com/your-org/your-repo" \
    allowed_scopes="openid profile email" \
    default_scopes="openid profile email"
```

```
# 3. An oidc mount in OpenBao that trusts OpenBao's own provider
bao auth enable -path=self-oidc oidc
bao write auth/self-oidc/config \
    oidc_discovery_url="https://openbao.corp.internal:8200" \
    oidc_client_id="github-actions" \
    oidc_client_secret="<from the client you created>" \
    default_role="github"
```

> ⚠ **2.7.0 removed `prompt=none` redirection support in the OIDC provider** (GHSA-2cjw-94fw-wqjx) — a deliberate security fix. Flows that relied on silent re-authorization will now fail.
>
> **2.6.0** added a **Client Credentials flow** to the OIDC provider, so OpenBao can issue tokens for machine-to-machine flows directly.

Which direction should you use?

	GitHub → OpenBao (this chapter)	OpenBao → GitHub (provider)
Complexity	Low — one `jwt` role	Higher — provider + client + mount
GitHub-native	✓ Uses the built-in `id-token: write`	Needs custom setup
Token lifetime	GitHub's ~5 min → yours	OpenBao's
Revocation	Revoke the role	Revoke the client
Recommendation	✓ **Use this**	Only if OpenBao is your system-wide IdP

11.8 Patterns for a real AI-harness repo

Pattern A — deploy-time fetch (recommended)

```
git push → CI runs → OIDC login → fetch secret → deploy → token expires
```

The secret is fetched at the moment it's needed, used in memory, and gone within 15 minutes. **This is the default and it is correct.**

Pattern B — the application pulls its own secrets

Don't have CI fetch the app's runtime secrets at all. Let the app authenticate to OpenBao itself at startup (AppRole or OIDC) — see Level 7.

Use this when the app is long-lived. CI handing a secret to a long-running process means the secret sits in that process's memory for hours, and re-deploying means re-fetching.

Pattern C — a `bao agent` sidecar in the runner

For workflows that need many secrets and want them cached:

```
# agent.hcl in the repo
vault {
  address = "https://openbao.corp.internal:8200"
  ca_cert = "/etc/ssl/certs/openbao-ca.crt"
}

auto_auth {
  method "jwt" {
    mount_path = "auth/github-actions"
    config = { role = "github" }
  }
  sink "file" {
    config = { path = "/tmp/openbao-token", mode = 0600 }
  }
}
```

The agent handles renewal and the token never appears in your script.

Pattern D — monorepo, multiple services

One role per service, each bound to the same repo but with different `bound_claims` on the `workflow` field:

```
bao write auth/github-actions/role/deploy-api   - <<'EOF'
{ "bound_claims": { "repository": "your-org/your-repo", "workflow": "deploy-api.yml" } }
EOF

bao write auth/github-actions/role/deploy-worker - <<'EOF'
{ "bound_claims": { "repository": "your-org/your-repo", "workflow": "deploy-worker.yml" } }
EOF
```

A compromised `deploy-worker.yml` cannot use the `deploy-api` role.

Pattern E — pull-request validation

```
bao write auth/github-actions/role/pr-check - <<'EOF'
{
  "role_type": "jwt",
  "user_claim": "actor",
  "bound_audiences": ["openbao"],
  "bound_claims": {
    "repository": "your-org/your-repo",
    "ref": "refs/pull/1/merge"
  },
  "token_policies": ["ci-lint-only"]
}
EOF
```

But bind on a claim that is actually stable. A `ref` of `refs/pull/1/merge` changes with every PR — instead, bind on `repository` and use a **read-only, low-value policy**, and rely on GitHub's own `pull_request` trigger + fork restrictions (secrets are **not** passed to workflows triggered by a fork).

11.9 Verified behaviour — what actually happens

I set up a real OpenBao 2.7.1 `jwt` mount, pointed it at a local JWKS server, signed a JWT with the matching RSA key (carrying genuine GitHub-shaped claims), and exercised it:

Test	Result
Valid token, correct audience + repo + ref	✓ token issued, `ci-deploy` policy, 15m T
Token with the **wrong audience**	✗ rejected — `bound_audiences` enforced
Token from a **different repository**	✗ rejected — `bound_claims` enforced
Token from a **different branch**	✗ rejected — `bound_claims` enforced
Tampered payload (signature broken)	✗ rejected — signature verified against t
JWKS entry **without a matching `kid`**	✗ rejected — "failed to verify id token sig
`token_policies_template_claims` with `environment=production`	✓ got `ci-prod` **and** `ci-your-org`

> ※ **The `kid` finding is worth knowing.** The JWKS must contain an entry whose `kid` matches the JWT header's `kid`. If your IdP's JWKS omits `kid`, every login fails with a signature error that looks like a network problem.

11.10 Troubleshooting

Symptom	Cause
`ACTIONS_ID_TOKEN_REQUEST_URL` is empty	`id-token: write` permission missing
The OIDC request returns `{"value": null}`	No `audience` parameter
`permission denied` on the OpenBao login	`aud` doesn't match `bound_audiences`
`bound_claims` mismatch	Repo/ref/environment claim differs
Signature verification fails	Wrong `jwks_url`, or the JWKS has no matching `kid`
`issuer` mismatch	`bound_issuer` wrong
Login works, secret read denied	The role's `token_policies` doesn't include the read policy
Works for `main`, fails for a PR	`bound_claims` on `ref` doesn't match PR refs
`group` claim missing	Keycloak mapper / scope
Template claim missing	`token_policies_template_claims` references an absent claim

Symptom	Cause
CI log shows the secret	You printed it

Inspect the claims in a failing workflow

```
      - name: Debug claims
        run: |
          echo "${{ steps.oidc.outputs.token }}" | cut -d. -f2 | base64 -d | jq .
```

[SEC] Only in a branch you control. In a PR from a fork, the OIDC token isn't issued at all (GitHub withholds it) — which is the correct behaviour.

11.11 The CI security checklist

```
□ id-token: write permission set, and nothing more than needed
□ audience= matches OpenBao's bound_audiences
□ Role bound to repository (and workflow, and environment where it matters)
□ Token TTL short (15m), max TTL short (30m)
□ CI policy has no `list` on secret mounts
□ GitHub Environment with required reviewers for production
□ Secrets are fetched at use time, not checked in or written to disk
□ No secret is ever echoed to the log
□ ::add-mask:: on the OpenBao token
□ BAO_CACERT is a public CA cert, so its presence in GitHub Secrets is not a risk
□ OIDC is the ONLY credential the workflow uses
□ The role is reviewed when the team structure changes
```

11.12 Run it for real

There is a complete, runnable lab for this chapter:

```
cd labs/lab04-ci-oidc
docker compose up -d
./setup.sh          # init, unseal, configure the jwt mount + role
./simulate-ci.sh    # the CI flow + the audit trail + a negative test
```

It runs the **same step scripts** the GitHub Action uses, against a stand-in IdP that mints GitHub-shaped OIDC tokens (identical claims, local key). The OpenBao side is **byte-for-byte the same** as production — only the token source differs.

The audit entry it produces:

```
{
  "op": "read",
  "path": "secret/data/myapp",
  "policies": [ "ci-deploy", "default" ],
  "entity": "885a6cd0-4018-049a-e753-99ec00b7bc39",
  "metadata": {
    "actor": "octocat",
    "environment": "production",
    "ref": "refs/heads/main",
    "repository": "your-org/your-repo",
    "role": "github",
```

```
    "run_id": "1792485478",
    "workflow": "deploy.yml"
  }
}
```

and the negative test:

```
✓ correctly refused:
   error validating claims: claim "repository" does not match any
   associated bound claim values
```

※ The `metadata` block comes from `claim_mappings` on the role. The claims are always validated; `claim_mappings` is what makes them **visible** in the audit log. Without it you get `{"role": "github"}` and nothing else.

 Two bugs this lab caught (both documented in Level 9 §"Scripting gotchas", because they will hit you in any script):

1. `bao status` **exits 2** when the server is sealed or uninitialised — which breaks `set -euo pipefail` scripts that pipe it into `jq`.
2. jq's `//` operator treats `false` as empty, so `jq -r '.sealed // "true"'` reports an **unsealed** server as sealed.

11.13 Checkpoint

1. Why is OIDC better than a stored CI secret, in one sentence each for lifetime, scope and revocation?
2. What must match between the workflow's `audience=` and OpenBao's `bound_audiences`?
3. What is the config path for the `jwt` mount, and the role path?
4. Why must `oidc_discovery_url` omit `.well-known` while `jwks_url` keeps it?
5. Which claim carries the GitHub Environment, and how do you bind it?
6. What does `token_policies_template_claims` let you avoid writing?
7. Why should the CI policy not have `list` on `secret/data/*`?
8. What happens if the JWKS entries don't carry a `kid`?
9. When would you prefer the app to pull its own secrets instead of CI fetching them?

Next → back to Level 9: REFERENCE.md for the full command and API cheatsheet, now updated with everything verified in Levels 10 and 11.

11.14 Scratchpad

```
OpenBao side:
  bao auth enable -path=github-actions jwt
 bao write auth/github-actions/config oidc_discovery_url="https://token.actions.githubusercon
                                  bound_issuer="https://token.actions.githubusercontent.com"
 bao write auth/github-actions/role/github bound_audiences=["openbao"] bound_claims={...}
  bao write auth/github-actions/login role=github jwt="$OIDC_TOKEN"

GitHub side:
  permissions: { contents: read, id-token: write }
  OIDC=$(curl -sS -H "Authorization: bearer $ACTIONS_ID_TOKEN_REQUEST_URL" \
        "$ACTIONS_ID_TOKEN_REQUEST_URL&audience=openbao" | jq -r .value)
 BAO_TOKEN=$(curl -sS --cacert "$BAO_CACERT" -d "{\"role\":\"github\",\"jwt\":\"$OIDC_TOKEN\"}
        "$BAO_ADDR/v1/auth/github-actions/login" | jq -r .auth.client_token)
```

```
Claims:     iss aud sub repository repository_owner ref workflow environment actor sha
Redirects:     none (jwt flow)
Base URL:      https://token.actions.githubusercontent.com      (no .well-known)
JWKS literal:  https://token.actions.githubusercontent.com/.well-known/jwks
```